“AI Based Helmet Violation Detection And E-Challan Forwarding”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

CH . YASWANTH **2520030259**

V . BHARATH KUMAR **2520030169**

Under the Esteemed Guidance of

Dr.D.Jyosthna
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



Declaration

The Project Report entitled **"RideSafe AI: An AI-Driven Framework for Multi-Occupant Motorcycle Violation Detection and Automated E-Challan Generation"** is a record of the bonafide work carried out by **Ch. Yaswanth, V. Bharath** submitted in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY in Computer Science and Information Technology** at **K L (Deemed to be) University**. The work presented in this report focuses on artificial intelligence and computer vision techniques for motorcycle violation detection, including helmet compliance analysis, multi-occupant violation detection, number-plate recognition, and automated e-challan support. The results and contents embodied in this report have been prepared by the project team and have not been copied from any other department, university, or institute.

CH. YASWANTH – 2520030259

V. BHARATH KUMAR – 2520030169

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **"RideSafe AI: An AI-Driven Framework for Multi-Occupant Motorcycle Violation Detection and Automated E-Challan Generation"** is a bonafide work done and submitted by **Ch. Yaswanth, V. Bharath** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the **Department of Computer Science and Information Technology**, K L (Deemed to be) University, during the academic year **2026–2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the valuable support, guidance, and encouragement received from many people. We sincerely express our gratitude to everyone who assisted us directly or indirectly throughout the development and completion of this project.

We would like to express our deepest appreciation to our **Project Guide, Dr.D.Jyothsna**, for her valuable guidance, continuous support, encouragement, and constructive suggestions throughout the project. Her guidance helped us to understand the technical requirements and successfully develop the project.

We express our gratitude to the **Department of Computer Science and Information Technology, K L (Deemed to be) University**, for providing us with the necessary facilities, resources, and academic environment required to complete this project.

We also thank all the teaching and non-teaching staff members who supported us directly or indirectly during the development of this project. Finally, we sincerely thank our parents, friends, and classmates for their continuous encouragement, cooperation, and support throughout our project work.

CH . YASWANTH-2520030259

V. BHARATH - 2520030169

Abstract

Road safety is a major concern in modern transportation systems, particularly with the increasing number of motorcycles on urban roads. Monitoring motorcycle violations manually through CCTV footage is time-consuming, difficult to scale, and dependent on continuous human observation. To address these challenges, our project "RideSafe AI: An AI-Driven Framework for Multi-Occupant Motorcycle Violation Detection and Automated E-Challan Generation" focuses on developing an intelligent computer-vision-based system capable of detecting motorcycle-related traffic violations from images and video footage. The primary aim of the system is to automate the identification of helmet violations, multi-occupant violations, and other relevant traffic violations while supporting number-plate recognition for generating structured e-challan records.

The proposed system combines multiple artificial intelligence and computer vision techniques to perform the complete violation-detection workflow. YOLO11 is used for object detection to identify motorcycles, persons, helmets, and other relevant objects from traffic scenes. SAM2 is incorporated for instance-level segmentation where more precise object regions are required. The system associates detected occupants with their respective motorcycles and performs helmet compliance analysis and occupant counting. This enables the system to identify cases such as riding without a helmet and triple riding. For detected violations, the system proceeds to number-plate detection and OCR/ANPR to extract the vehicle registration number whenever the plate is sufficiently visible.

The generated violation information is organized and stored using a MySQL database, including details such as violation type, timestamp, detected number plate, confidence information, and supporting evidence. A backend service and administrative monitoring dashboard provide a centralized interface for viewing AI detections, violation records, and system activity. The modular architecture allows individual components such as detection, segmentation, OCR, database management, and dashboard functionality to be developed and improved independently. Overall, RideSafe AI provides an integrated framework for AI-assisted traffic monitoring and automated violation record generation, offering a foundation for efficient, scalable, and technology-driven road-safety management.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	9-24
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	25-34
3	Technology Stack	3.1 Artificial Intelligence and Computer Vision 3.2 Object Detection using YOLO11 3.3 Image Segmentation using SAM2 3.4 Backend Technologies 3.5 Database – MySQL 3.6 Front-End Technologies 3.7 Version Control	35-59
4	System Architecture	4.1 High-Level System Architecture 4.2 AI Processing Pipeline 4.3 Description of System Layers 4.4 Deployment Architecture	60-71
5	Design	5.1 System Workflow Design 5.2 Database Design	72-74
6	Implementation	6.1 AI Model and Dataset Implementation 6.2 Violation Detection Implementation 6.3 Application, Backend and Database Implementation	75-88
7	Features	7.1 Main Features of RideSafe AI 7.2 AI-Based Motorcycle Violation Detection 7.3 Violation Records and Admin Monitoring	89-91
8	Testing	8.1 AI Model and Module Testing 8.2 Integration and Database Testing 8.3 Test Cases, Results and Performance Evaluation	92-95
9	Deployment	9.1 Deployment Requirements and Configuration 9.2 Backend and Database Deployment	96-99
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations / Known Issues	100-104
11	Future Enhancements	11. 1 AI and System Enhancements 11.2 Advanced Features and Future Integration	105-107
12	Conclusion	12.1 Summary of the Project 12.2 Achievements 12.3 Skills Learned During Development	108-111
13	References	Research papers, datasets	112-116

14	Appendices	Screenshots of the Application AI Detection Outputs SAM2 Segmentation Outputs Database Screenshots Geo Tag photos Review forms with guide signatures	117-125

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Road transportation is an essential part of modern society, providing connectivity for people, businesses, and communities. Among the different types of vehicles used on roads, motorcycles are widely preferred because of their affordability, convenience, and suitability for navigating congested urban environments. However, the increasing number of motorcycles on roads also creates significant road-safety challenges. Violations such as riding without a helmet and carrying more occupants than permitted can increase the risk of serious injuries during road accidents. Effective monitoring of such violations is therefore an important part of improving road safety and encouraging responsible riding practices.

Traditionally, traffic-rule enforcement relies heavily on police personnel and surveillance operators who monitor roads and CCTV camera feeds manually. Although human monitoring can identify violations, it becomes difficult when the number of vehicles and camera feeds increases. Continuous observation requires considerable time and attention, and violations may be missed because of traffic density, poor visibility, overlapping vehicles, or the large volume of video footage that needs to be reviewed. These limitations create an opportunity for computer-vision-based systems that can automatically analyze traffic scenes and assist authorities in identifying potential violations.

Recent developments in Artificial Intelligence (AI), Machine Learning (ML), and Computer Vision have made automated traffic monitoring increasingly practical. Deep-learning-based object detection models can identify objects such as motorcycles, persons, helmets, and other road elements directly from images and video frames. These models can process large numbers of frames and provide bounding boxes, class labels, and confidence values that can be used by subsequent stages of an intelligent traffic-monitoring pipeline. Such capabilities provide the foundation for developing systems that can assist in identifying motorcycle-related violations from CCTV or other traffic-camera footage.

The proposed project, **“RideSafe AI: An AI-Driven Framework for Multi-Occupant Motorcycle Violation Detection and Automated E-Challan Generation,”** is developed with the objective of creating a modular computer-vision framework for motorcycle violation monitoring. The system is designed to analyze traffic images or video, detect motorcycles and their occupants, evaluate helmet compliance, identify multi-occupant situations, and support further processing of violation cases. Instead of treating traffic monitoring as a single detection task, the proposed framework divides the process into multiple interconnected stages so that each stage can perform a specific responsibility.

The first major component of the system is object detection. The project uses **YOLO11** as part of the computer-vision pipeline to identify relevant objects within traffic scenes. Motorcycle and person detections form an important foundation because subsequent violation analysis depends on understanding the relationship between vehicles and their occupants. Detecting these objects individually is not sufficient for a complete traffic-violation system; the system must also determine which detected occupants belong to which motorcycle. Therefore, motorcycle–occupant association is an important stage of the proposed architecture.

Another important component of the project is helmet compliance analysis. Helmet usage is a critical factor in motorcycle safety, and identifying whether relevant occupants are wearing helmets is one of the primary objectives of RideSafe AI. The system is designed to analyze detected motorcycle occupants and determine their helmet status. This allows potential helmet violations to be identified automatically from the processed traffic footage. The system is also designed to handle situations involving multiple occupants rather than restricting analysis to motorcycles carrying only a single rider.

Multi-occupant analysis is particularly important for identifying cases such as **triple riding**. In a traffic scene, a motorcycle may contain more than one occupant, and the system therefore needs to count and associate occupants correctly before making a violation decision. The proposed framework considers occupant detection and association as separate stages so that the number of people associated with a motorcycle can be analyzed independently from helmet compliance. This provides a foundation for detecting different types of motorcycle violations through a common processing pipeline.

The project also incorporates **SAM2 (Segment Anything Model 2)** as part of its computer-vision technology stack. Segmentation can provide more detailed information about the region occupied by an object than a simple bounding box. This can be useful in traffic scenes where objects overlap or where more precise object regions are required. A dedicated helmet segmentation dataset was prepared for the project by collecting, converting, merging, and organizing multiple datasets. The final prepared SAM2 dataset contains **1,027 unique images**, divided into **720 training images, 205 validation images, and 102 test images**, with a total of **1,570 annotated object instances**. These include helmet and without-helmet instances and provide a dedicated dataset for developing the segmentation component of the system.

The development of the project involves several stages of dataset preparation and model development. Different datasets may use different annotation structures, class names, image distributions, and segmentation formats. Therefore, dataset preprocessing, annotation conversion, merging, and duplicate removal are important parts of the project. The prepared datasets provide the foundation for training and evaluating the AI components while allowing the system to be developed in a structured manner.

Another planned component of the RideSafe AI architecture is **number-plate detection and Optical Character Recognition (OCR)/Automatic Number Plate Recognition (ANPR)**. Once a potential violation is identified, the system is intended to locate the vehicle's registration plate and extract the registration number where the plate is sufficiently visible. This information can then be associated with the detected violation event. The number-plate recognition stage is currently part of the planned and developing pipeline rather than being treated as a fully completed component. Its integration will allow the system to move from simply identifying a potential violation toward creating a structured vehicle-specific violation record.

The proposed system also includes a database layer for maintaining violation information. **MySQL** is used as the planned relational database for storing structured information associated with the system. Depending on the implementation stage, records can include violation type, timestamp, camera information, detected vehicle information, confidence values, number-plate information when available, and references to supporting evidence. Maintaining these records in a structured database makes it possible to retrieve, review, and manage detected violations through an administrative interface.

A monitoring and administration interface is also an important part of the overall system architecture. The dashboard is intended to provide a centralized view of the AI pipeline and its generated records. Through the interface, administrators can monitor detection results, review suspected violations, inspect available evidence, and access stored records. The frontend is based on web technologies such as **HTML, CSS, and JavaScript**, while the backend can provide the communication layer between the AI processing modules, database, and dashboard.

The overall architecture of RideSafe AI is therefore designed as a sequence of interconnected modules. Traffic video or images are provided as input, after which frames are processed by the detection stage. Motorcycle and person detections are then used for occupant association and counting. Helmet analysis is performed for relevant occupants, while multi-occupant analysis provides additional violation information. When a potential violation is identified, the system is designed to pass the relevant frame to the number-plate detection and OCR/ANPR stages as they are developed. The resulting information can then be stored in the MySQL database and presented through the monitoring dashboard.

From an engineering perspective, the project combines several important areas of computer science and artificial intelligence:

1. **Computer Vision:** Processing traffic images and video to identify motorcycles, people, helmets, and other relevant objects.
2. **Object Detection:** Using YOLO11 to detect and localize objects within traffic scenes.
3. **Image Segmentation:** Using SAM2 to obtain more detailed object-level regions where required.
4. **Data Preparation:** Collecting, preprocessing, converting, merging, and organizing datasets for AI model development.
5. **OCR/ANPR:** Providing a planned mechanism for extracting vehicle registration information from number plates.
6. **Backend Development:** Connecting AI processing with application services and database operations.
7. **Database Management:** Using MySQL to maintain structured violation and system records.
8. **Web Development:** Providing an administrative interface for monitoring and reviewing system output.
9. **System Integration:** Combining independent AI and software components into a single modular traffic-monitoring framework.

The modular nature of RideSafe AI is an important aspect of its design. Computer-vision systems are affected by factors such as camera position, lighting conditions, vehicle size, image quality, occlusion, and traffic density. A modular architecture allows individual components to be improved without requiring the entire system to be redesigned. For example, the detection model can be improved independently from the OCR module, while the database and dashboard can continue to use structured outputs from the AI pipeline.

The project also provides an academic and practical foundation for intelligent transportation systems. It demonstrates how artificial intelligence can be combined with conventional software engineering components to address a real-world traffic-monitoring problem. Rather than depending entirely on manual inspection, the proposed system aims to assist monitoring personnel by automatically processing large amounts of visual information and presenting potentially relevant violations in a structured form.

1.2 Problem Statement

The increasing number of motorcycles on urban roads has created a significant need for efficient and reliable traffic monitoring systems. Motorcycles provide an economical and convenient means of transportation, but violations such as riding without a helmet and carrying more occupants than permitted can create serious road-safety concerns. At present, the identification of such violations is often dependent on traffic police personnel and manual monitoring of CCTV footage. Although human supervision can be effective in individual situations, continuously monitoring a large number of roads and camera feeds is difficult, time-consuming, and prone to missed violations, particularly during periods of heavy traffic.

One of the major problems in conventional traffic surveillance is the large volume of visual information that needs to be analyzed. A CCTV camera may continuously capture thousands of frames containing different vehicles, pedestrians, and road activities. Manually reviewing every frame to identify motorcycles, their occupants, helmet usage, and other violations require considerable human effort. When multiple cameras are operating simultaneously, the workload increases further. Important violations may be overlooked because of crowded traffic scenes, limited visibility, camera positioning, motion blur, or the attention limitations of human operators. Therefore, there is a need for an automated system that can assist in analyzing traffic footage and identifying potentially relevant motorcycle violations.

Another challenge is that motorcycle violation detection is not simply an object-detection problem. Detecting a motorcycle and detecting a person separately does not automatically establish which person is riding which motorcycle. A traffic scene may contain several motorcycles and multiple people in close proximity to one another. The system therefore needs to associate detected occupants with the correct motorcycle before making a violation decision. This becomes particularly challenging when motorcycles are close together, when occupants overlap with other objects, or when the camera captures the scene from a distant or angled viewpoint. Reliable motorcycle–occupant association is therefore an important part of an automated violation-detection system.

Helmet detection introduces another layer of complexity. The system must determine whether the relevant motorcycle occupant is wearing a helmet rather than simply detecting the presence of a helmet somewhere within the image. Helmets can appear at different scales and angles, and their appearance can change depending on lighting conditions, camera resolution, rider orientation, and occlusion. In crowded traffic scenes, the heads of riders may also overlap with other objects or with nearby motorcycles. These conditions make automated helmet-compliance analysis more challenging than simple object recognition.

The problem becomes more complex when a motorcycle carries multiple occupants. A system designed only for single-rider helmet detection may not correctly handle situations involving two or more people on the same motorcycle. The system must first identify and count the occupants associated with a particular motorcycle and then perform the appropriate helmet analysis. Cases involving three occupants, commonly referred to as triple riding, require the system to distinguish between legitimate nearby pedestrians and the actual occupants of the motorcycle. Therefore, occupant counting and motorcycle–occupant association need to be treated as important components of the violation-detection pipeline.

Traffic scenes also contain several sources of visual uncertainty. Illumination may vary between daytime and nighttime conditions, while weather conditions can affect image quality. Vehicles may partially block one another, and motorcycle riders may be partially hidden by helmets, clothing, other vehicles, or road infrastructure. Motion blur can reduce the clarity of small objects, while low-resolution CCTV footage can make it difficult to identify helmets and other visual details. These challenges can affect the performance of computer-vision models and make it necessary to design the system as a combination of multiple processing stages rather than relying on a single prediction.

Another important problem is the preparation of suitable datasets for training computer-vision models. AI models require appropriately labeled data to learn the visual characteristics of objects and conditions relevant to the target application. Datasets obtained from different sources may contain different annotation formats, class names, image distributions, and labeling standards. For the RideSafe AI project, multiple helmet-related datasets were therefore collected and processed to create a more suitable training resource. Dataset conversion, merging, duplicate removal, and organization are necessary steps before model training can be performed effectively. The segmentation component also requires appropriate mask annotations for training SAM2.

The project addresses these challenges through a multi-stage artificial-intelligence pipeline. The proposed system uses **YOLO11** for object detection and **SAM2** for segmentation-related tasks. Instead of attempting to identify every type of violation through a single model, the system separates the process into object detection, motorcycle–occupant association, helmet analysis, occupant counting, and violation decision stages. This modular approach makes it possible to process the information generated by one stage before passing it to the next stage. It also allows individual components to be improved independently as the project develops.

A further problem arises after a potential violation has been identified: connecting the visual violation to the corresponding vehicle. For this purpose, the complete RideSafe AI architecture includes a planned number-plate detection and **OCR/ANPR** stage. The purpose of this stage is to identify the registration plate associated with a suspected violation and extract the registration number when the plate is sufficiently visible. However, this component is currently under development and is not considered a fully completed part of the present implementation. Its inclusion in the architecture represents the intended next stage of the system rather than a claim that complete number-plate recognition has already been achieved.

Similarly, the overall project is intended to support automated e-challan generation. A structured violation record can contain information such as the detected violation type, timestamp, camera information, vehicle information, confidence values, evidence references, and number-plate information when available. The current project architecture therefore provides a foundation for such functionality, while the complete integration of number-plate recognition and automated e-challan processing remains part of the continuing development of the system.

Data management is another important consideration. A traffic-monitoring system can generate a large number of detection and violation events, and these records need to be stored in an organized manner. Without a centralized database, it becomes difficult to search, review, update, or manage previously detected violations. RideSafe AI therefore incorporates **MySQL** as the relational database layer for structured information. The database is intended to maintain relevant records and provide the backend with a reliable mechanism for retrieving information for the administrative monitoring interface.

The problem also extends to monitoring and reviewing AI-generated results. An automated system should not only process video but should also provide a convenient method for administrators or authorized users to inspect the results. For this reason, the proposed system includes a web-based monitoring and administration interface. The dashboard is intended to display detection information, violation records, timestamps, available vehicle information, and supporting evidence. This provides a centralized view of the processing pipeline and makes it easier to review system output.

Therefore, the central problem addressed by RideSafe AI is the lack of a unified and modular computer-vision framework capable of assisting with the automated detection and management of motorcycle-related traffic violations from visual traffic data. The system must address several interconnected challenges, including object detection, motorcycle–occupant association, helmet compliance analysis, multi-occupant detection, segmentation, number-plate recognition, database management, and administrative monitoring.

The proposed project aims to address these challenges through the following problem resolutions:

1. **Reduce dependence on continuous manual monitoring** by automatically analyzing traffic images and video.
2. **Detect motorcycles and relevant occupants** using AI-based object detection techniques.
3. **Associate occupants with motorcycles** to ensure that violation analysis is performed for the correct vehicle.

4. **Identify helmet-compliance violations** by analyzing the helmet status of relevant motorcycle occupants.
5. **Detect multi-occupant situations**, including potential triple-riding cases, using occupant counting and association.
6. **Use image segmentation where required** to obtain more precise object information in challenging traffic scenes.
7. **Provide a planned number-plate recognition stage** for connecting detected violations with vehicle registration information.
8. **Maintain structured violation records** using a MySQL database for storage and retrieval.
9. **Provide an administrative monitoring interface** through which detected events and stored records can be reviewed.
10. **Create a modular foundation for future e-challan integration**, allowing the system to progress from violation detection toward automated violation-record processing.

1.3 Scope of the Project

The scope of **RideSafe AI: An AI-Driven Framework for Multi-Occupant Motorcycle Violation Detection and Automated E-Challan Generation** covers the design and development of an artificial-intelligence-based traffic monitoring framework capable of analyzing motorcycle-related violations from images and video footage. The project focuses primarily on using computer vision to identify motorcycles and their occupants, analyze helmet compliance, detect multi-occupant situations, and organize the detected information into structured violation records. The system is designed as a modular framework so that individual components can be developed, tested, and improved independently.

The primary scope of the project begins with the acquisition of traffic images or video footage from suitable sources such as CCTV cameras, recorded traffic videos, or other supported visual inputs. The input data is processed frame by frame so that the AI models can analyze individual traffic scenes. The system is intended to support traffic environments in which multiple motorcycles and other road users may appear simultaneously. The quality of the input footage, camera position, lighting conditions, and visibility of the vehicles can influence the resulting AI predictions.

A major part of the project scope is **motorcycle and object detection**. The system uses **YOLO11** as part of the detection pipeline to identify motorcycles, persons, helmets, and other relevant objects required for violation analysis. The detection stage provides the initial information needed by subsequent modules. Instead of treating the complete traffic scene as a single classification problem, the project separates the relevant objects and processes them according to their roles in the traffic-violation workflow.

The scope also includes **motorcycle–occupant association**. Detecting motorcycles and people independently is not sufficient because the system needs to determine which people belong to each motorcycle. The association stage is therefore responsible for relating detected occupants to the corresponding motorcycle based on their spatial relationship and other available information. This functionality is particularly important in crowded traffic scenes where multiple motorcycles and pedestrians may be located close to one another.

Another major area within the project scope is **helmet compliance detection**. The system is designed to determine whether relevant motorcycle occupants are wearing helmets. The helmet-analysis stage receives information from the detection and association modules and uses it to identify potential helmet violations. The system is intended to consider the relevant occupants of each motorcycle rather than simply detecting whether a helmet exists somewhere in the image. This approach provides a more meaningful basis for motorcycle-specific violation analysis.

The project also covers **multi-occupant detection and occupant counting**. Motorcycles may carry more than one person, and the system therefore needs to identify and count the occupants associated with each motorcycle. This enables the framework to identify situations involving multiple occupants and provide a separate signal for potential **triple-riding violations**. The occupant-counting component is intended to work together with the motorcycle-association stage so that people belonging to different motorcycles are not incorrectly combined.

The scope further includes the use of **SAM2 (Segment Anything Model 2)** for segmentation-related tasks. Segmentation can provide more precise information about object regions than conventional bounding boxes and can be useful in situations where object boundaries are important. A dedicated helmet segmentation dataset has been prepared for this purpose. The dataset preparation process includes collecting relevant data, converting annotation formats, merging datasets, removing duplicates, and dividing the final dataset into training, validation, and testing subsets. The final prepared dataset contains 1,027 unique images and 1,570 annotated object instances.

Dataset preparation itself forms an important part of the project scope. The project uses multiple datasets related to helmet detection and segmentation, and these datasets may differ in their annotation formats and organization. Therefore, preprocessing and standardization are required before they can be used effectively for model development. The scope includes dataset organization, annotation conversion, merging, duplicate removal, and preparation of appropriate training and validation structures for the AI models.

The project also includes a **violation-decision layer** that combines the outputs of different computer-vision modules. Rather than generating a violation based on a single detection, the framework is designed to consider information such as the motorcycle detected, its associated occupants, helmet status, and occupant count. Based on the configured rules, the system can generate a potential violation event. This modular decision process allows different types of motorcycle violations to be handled within a common framework.

Another component included in the overall scope is **number-plate detection and OCR/ANPR**. The purpose of this module is to identify the registration plate of a vehicle associated with a detected violation and extract the registration number. However, this component is currently **under development**. Therefore, the scope of the present project includes its planned integration and architectural design, but the documentation does not treat complete number-plate recognition as an already finished and fully validated feature. Future implementation and testing will determine the final performance of this component.

The project also has scope for **structured violation-data management** using a MySQL database. The database is intended to store information generated by the AI pipeline, such as violation type, timestamp, camera information, confidence values, available vehicle information, and evidence references. Number-plate information can also be associated with the violation record once the OCR/ANPR component is implemented and validated. The database provides a centralized method for storing and retrieving information generated by the system.

A further area within the scope is the development of an **administrative monitoring dashboard**. The dashboard is intended to provide authorized users with a centralized interface for viewing system activity and reviewing detected violations. It can include information such as recent detection events, violation types, timestamps, available number-plate information, and supporting evidence. The dashboard also provides a foundation for future system-management features such as camera monitoring, filtering, reporting, and review workflows.

The scope of the project extends beyond individual AI models to include **integration between artificial intelligence and conventional software components**. The project combines computer vision, model inference, backend services, database management, and web-based visualization into a single system. This integration demonstrates how AI models can be incorporated into a complete application rather than being used only as independent prediction scripts.

From a deployment perspective, the current project scope includes development and testing in a local GPU-enabled environment. The AI models can be executed on a system equipped with a suitable NVIDIA GPU, while the backend, database, and dashboard can operate as connected software components. Future versions can extend the architecture toward multiple camera streams, edge devices, cloud infrastructure, or distributed inference systems depending on performance and deployment requirements.

The project is also intended to provide a foundation for **future e-challan integration**. Once violation detection, vehicle identification, and supporting evidence are sufficiently reliable, the structured violation record can potentially be connected to an authorized e-challan service or workflow. However, complete automated e-challan generation is considered a future stage of the project and is not represented as a fully completed feature in the current implementation. Any real-world enforcement integration would also require appropriate authorization, validation, security, and compliance procedures.

The scope of RideSafe AI can therefore be divided into the following major areas:

1. **Traffic Input Processing** – Processing images and video footage obtained from suitable traffic-monitoring sources.
2. **Object Detection** – Detecting motorcycles, persons, helmets, and other relevant objects using YOLO11.
3. **Object Segmentation** – Applying SAM2-based segmentation where more detailed object regions are required.
4. **Motorcycle–Occupant Association** – Establishing the relationship between motorcycles and their detected occupants.
5. **Helmet Compliance Analysis** – Identifying potential cases where relevant motorcycle occupants are not wearing helmets.
6. **Occupant Counting** – Counting occupants associated with each motorcycle and identifying potential multi-occupant violations.
7. **Violation Classification** – Combining detection results and predefined rules to generate potential violation events.
8. **Number-Plate Recognition** – Developing the planned number-plate detection and OCR/ANPR stage for vehicle identification.
9. **Database Management** – Storing structured violation and system information using MySQL.
10. **Administrative Monitoring** – Providing a web-based interface for reviewing detections and stored violation records.
11. **Evidence Management** – Maintaining references to relevant images or video segments associated with detected events.
12. **Future E-Challan Integration** – Providing a foundation for connecting verified violation records to an authorized e-challan workflow.

The project also has certain boundaries within its scope. It is primarily focused on **motorcycle-related violations** and does not aim to provide a complete solution for every possible traffic violation. The system's performance is dependent on the quality of the input footage and the capabilities of the trained models. Conditions such as severe occlusion, very low-resolution footage, extreme lighting, motion blur, and unusual camera angles may affect detection results. Therefore, the project is intended to provide an AI-assisted monitoring framework rather than claiming that every detected event can automatically be treated as a legally confirmed violation.

1.4 Objectives of the Project

The primary objective of **RideSafe AI** is to design and develop an AI-driven framework capable of assisting in the automated detection and management of motorcycle traffic violations. The system aims to combine computer vision, deep learning, image segmentation, occupant analysis, database management, and web technologies into a unified platform. By reducing the dependence on continuous manual monitoring, the proposed system aims to improve the efficiency of traffic surveillance while providing a structured approach for identifying and managing motorcycle-related violations. The major objectives of the project are as follows:

1. **To design and develop an AI-based traffic monitoring framework for detecting motorcycle-related violations from images and video footage.**
2. **To detect motorcycles, persons, helmets, and other relevant objects using YOLO11-based object detection techniques.**
3. **To identify and associate motorcycle occupants with their corresponding motorcycles for accurate violation analysis.**
4. **To detect helmet compliance and identify potential cases of riding without a helmet using computer vision techniques.**
5. **To count motorcycle occupants and identify potential multi-occupant violations, including triple-riding situations.**
6. **To utilize SAM2-based image segmentation for obtaining more precise object-level information where required by the traffic-analysis pipeline.**
7. **To prepare, preprocess, merge, and organize suitable datasets for training and improving the computer-vision and segmentation components of the system.**
8. **To develop a structured violation-decision mechanism that combines motorcycle detection, occupant association, helmet status, and occupant count.**
9. **To develop and integrate a number-plate detection and OCR/ANPR module for identifying vehicle registration information associated with detected violations.**
10. **To store and manage detected violation information using a MySQL relational database for efficient retrieval and record management.**
11. **To develop a web-based administrative dashboard for monitoring AI detections, reviewing potential violations, and accessing stored violation records.**
12. **To design a modular and extensible architecture that allows future integration of advanced tracking, improved OCR, additional violation categories, and multiple-camera processing.**
13. **To provide a foundation for future integration with authorized automated e-challan generation and traffic-enforcement workflows.**
14. **To evaluate the developed system under different traffic conditions and identify areas for improving detection accuracy, robustness, and processing performance.**

In essence, the objective of RideSafe AI goes beyond simply detecting whether a motorcycle rider is wearing a helmet. It aims to establish a **comprehensive AI-assisted traffic-monitoring framework** that understands motorcycles and their occupants, identifies relevant violations, organizes the detected information, and provides a foundation for vehicle identification and future e-challan processing. By integrating **YOLO11, SAM2, computer vision, occupant analysis, MySQL, backend services, and an administrative monitoring interface**, the project aims to provide a scalable foundation for intelligent motorcycle-violation monitoring and future smart traffic-management applications.

1.5 Importance of the Application

Road safety has become an increasingly important concern with the rapid growth of urban traffic and the rising number of motorcycles on roads. Motorcycles provide an affordable and convenient mode of transportation, but violations such as riding without a helmet and carrying more occupants than permitted can increase the risk of serious road accidents. In conventional traffic-monitoring systems, identifying such violations largely depends on police personnel and manual observation of CCTV footage. This approach requires continuous attention and becomes difficult to manage when a large number of vehicles and camera feeds need to be monitored simultaneously. Hence, the importance of **RideSafe AI** lies in its ability to use artificial intelligence and computer vision to assist in monitoring motorcycle-related traffic violations in a more systematic and scalable manner.

The proposed system is not limited to a simple helmet-detection application; it is designed as an integrated traffic-monitoring framework that combines multiple stages of computer vision and software processing. By bringing together motorcycle detection, occupant detection, motorcycle–occupant association, helmet analysis, multi-occupant detection, database management, and administrative monitoring, the system provides a structured approach to analyzing motorcycle violations. This integration reduces the amount of manual effort required for initial identification and allows potentially relevant traffic events to be organized for further review.

One of the most significant aspects of the project is its focus on **helmet compliance and motorcycle safety**. Wearing a helmet is an important safety practice for motorcycle riders and occupants. However, manually identifying helmet violations across continuous traffic footage is difficult, especially when traffic volume is high. RideSafe AI uses AI-based object detection and computer-vision techniques to identify relevant motorcycles and occupants and analyze their helmet status. This allows potential helmet violations to be identified automatically and provides a foundation for more efficient traffic monitoring.

Another important aspect of the application is its ability to consider **multiple occupants on a motorcycle**. A motorcycle may carry more than one person, and identifying the number of occupants is important for detecting situations such as triple riding. A conventional system that only checks whether a helmet is present may fail to understand the complete situation. RideSafe AI therefore considers motorcycle–occupant association and occupant counting as important stages of its processing pipeline. This allows the system to analyze the motorcycle as a complete unit rather than treating individual people and vehicles independently.

The application is also significant from a technological perspective because it demonstrates the practical use of modern artificial-intelligence techniques in traffic monitoring. **YOLO11** is incorporated for object detection, while **SAM2** is used for segmentation-related tasks where more detailed object-level information is required. The project also involves the preparation and organization of dedicated datasets for model development. These technologies demonstrate how deep learning, computer vision, and image segmentation can be combined to address a practical real-world problem.

The importance of the application is further enhanced by its modular architecture. Traffic environments are highly variable, and a computer-vision system may need to be improved as new datasets, models, and processing techniques become available. By separating detection, segmentation, occupant association, violation analysis, database management, and monitoring into different stages, RideSafe AI allows individual components to be improved without redesigning the entire system. This makes the framework suitable for future expansion and experimentation.

Another important aspect of the proposed system is its planned **number-plate recognition capability**. After a potential violation is identified, the complete architecture is intended to detect the corresponding vehicle's registration plate and extract the registration number using OCR/ANPR techniques. This can help connect a detected violation with a specific vehicle record. The number-plate recognition component is currently under development, so its importance in the present project lies primarily in providing the next stage of the violation-processing pipeline rather than being treated as a fully completed feature.

The use of a centralized **MySQL database** also adds importance to the application. AI-based detection can generate a large amount of information, and storing this information in an organized manner is necessary for effective management and review. The database can maintain details such as violation type, timestamp, camera or input source, confidence information, vehicle details when available, and references to supporting evidence. This structured storage makes it easier to retrieve and review previous events and provides a foundation for future traffic-analysis and reporting functions.

The project also has significance from the perspective of **traffic-monitoring efficiency**. Human operators can spend considerable time observing CCTV feeds and identifying potential violations. An AI-assisted system can automatically process video frames and highlight events that require attention. This does not eliminate the need for human verification, particularly for uncertain detections, but it can help reduce the amount of footage that requires detailed manual examination. As a result, traffic-monitoring personnel can focus their attention on reviewing potentially significant events rather than continuously observing every frame.

The proposed administrative dashboard further increases the usefulness of the system by providing a centralized interface for monitoring AI-generated information. Instead of keeping model outputs only within individual scripts, the application is designed to present relevant detections and violation records through a web-based interface. Administrators can use the dashboard to review available information, examine evidence, and access stored records. This creates a connection between the AI processing layer and the practical requirements of traffic monitoring.

From a software-engineering perspective, the project demonstrates the integration of multiple technologies into a single application. It combines **Python, PyTorch, YOLO11, SAM2, OpenCV, backend services, MySQL, HTML, CSS, JavaScript, and Git/GitHub** within a common system architecture. This integration demonstrates how artificial intelligence can be incorporated into a complete software solution rather than being developed as an isolated machine-learning model.

The application also has academic and research importance. It provides an opportunity to study several areas of computer science, including deep learning, object detection, image segmentation, dataset preparation, computer vision, OCR concepts, database management, backend development, and web-based system design. The project can serve as a practical example of how multiple technologies can be combined to solve a real-world problem involving intelligent transportation and road-safety monitoring.

The system also provides a foundation for future expansion into more advanced traffic-monitoring capabilities. Once the core modules are sufficiently developed and validated, the architecture can be extended with object tracking, improved number-plate recognition, multi-camera processing, real-time alerts, advanced analytics, and other traffic-violation categories. The planned integration with automated e-challan workflows can further extend the system from violation detection toward structured traffic-enforcement support, subject to appropriate validation and authorization.

To summarize, the importance of the RideSafe AI application can be understood through the following key points:

1. **It assists in automated motorcycle-violation monitoring** by applying artificial intelligence and computer vision to traffic images and videos.
2. **It improves the efficiency of helmet-violation detection** by automatically analyzing motorcycle occupants and their helmet status.
3. **It supports multi-occupant analysis** and provides a foundation for identifying potential triple-riding violations.
4. **It combines multiple AI technologies**, including YOLO11 and SAM2, within a unified computer-vision pipeline.
5. **It reduces the dependency on continuous manual observation** by automatically processing traffic footage and identifying potentially relevant events.
6. **It provides structured violation management** through MySQL-based storage and an administrative monitoring interface.
7. **It establishes a foundation for vehicle identification** through the planned number-plate detection and OCR/ANPR module.
8. **It provides a modular and extensible architecture** that can be improved with better models, tracking techniques, and additional traffic-violation categories.
9. **It demonstrates the practical application of AI and software engineering** to a real-world road-safety problem.

In conclusion, the importance of RideSafe AI extends beyond simply detecting helmet violations. Its significance lies in developing an integrated and extensible framework that can analyze motorcycles and their occupants, identify potential traffic violations, organize the resulting information, and provide a foundation for future vehicle identification and automated enforcement workflows. By combining artificial intelligence, computer vision, segmentation, database management, and web technologies, the project establishes a practical foundation for intelligent motorcycle-violation monitoring and future smart traffic-management applications.

1.6 Target Users / Audience

The target users of **RideSafe AI: An AI-Driven Framework for Multi-Occupant Motorcycle Violation Detection and Automated E-Challan Generation** primarily include organizations, personnel, and individuals involved in traffic monitoring, road-safety management, and intelligent transportation systems. The system is designed to assist users who require efficient identification, review, and management of motorcycle-related traffic violations through AI-based video analysis.

The **primary users** are traffic-monitoring personnel and authorized traffic enforcement officials who need to monitor CCTV footage and identify potential motorcycle violations. Instead of continuously observing every frame manually, they can use the system to assist in detecting motorcycles, occupants, helmet compliance, and multi-occupant situations. The monitoring interface can provide relevant violation information and supporting evidence for further review.

The **secondary users** include system administrators who are responsible for managing the application, database records, camera or input sources, and system configuration. Administrators can use the monitoring dashboard to review detected events, access stored violation records, and monitor the functioning of the AI pipeline. The MySQL database provides a structured mechanism for maintaining the information generated by the system.

Traffic departments and organizations responsible for **road-safety management** can also benefit from the proposed framework. The system can provide an AI-assisted approach to analyzing traffic footage and identifying potential violations. Once the number-plate recognition and other planned components are completed and validated, the framework can provide a foundation for connecting detected violations with vehicle information and future authorized e-challan workflows.

The system can also be useful to **researchers, students, and academic institutions** working in areas such as computer vision, artificial intelligence, intelligent transportation systems, object detection, image segmentation, and automated traffic monitoring. The project demonstrates the integration of technologies such as YOLO11, SAM2, image processing, databases, backend services, and web-based monitoring into a practical application.

In addition, the framework can serve as a foundation for **future traffic-management operators and system developers** who may extend the application with additional violation categories, object tracking, improved OCR/ANPR, multi-camera processing, real-time alerts, and advanced traffic analytics. Its modular architecture allows new technologies and models to be incorporated as the system evolves.

In essence, the target audience of RideSafe AI includes:

1. **Traffic-monitoring personnel** who need assistance in identifying motorcycle violations from CCTV or traffic video.
2. **Traffic enforcement officials** who require structured information for reviewing potential violations.
3. **System administrators** responsible for managing the AI system, database, and monitoring dashboard.
4. **Road-safety organizations and traffic departments** interested in AI-assisted traffic surveillance.
5. **Researchers and academic institutions** studying computer vision, AI, intelligent transportation, and traffic-violation detection.
6. **Developers and future system operators** who may extend the framework with advanced tracking, OCR/ANPR, analytics, and authorized e-challan integration.
7.

Overall, the target audience represents a range of users involved in **traffic monitoring, road safety, system administration, research, and intelligent transportation technology**. RideSafe AI aims to bridge the gap between raw traffic-camera footage and structured, reviewable violation information by providing an AI-assisted platform that can support traffic-monitoring activities while allowing the system to evolve toward more advanced automated traffic-management capabilities.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The development of **RideSafe AI** requires a computing environment capable of handling deep-learning models, image and video processing, dataset preparation, database operations, and web-based application development. Since the project involves AI models such as **YOLO11** and **SAM2**, a GPU-enabled system is particularly useful for model training and inference. In addition to computational resources, sufficient memory and storage are required because computer-vision projects involve large datasets, model checkpoints, processed images, video files, and training outputs.

The hardware environment used for the development of RideSafe AI is based on a laptop equipped with an **AMD Ryzen 9 8945HS processor, 16 GB DDR5-5600 RAM, NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM, and 1 TB SSD storage**. This configuration provides the computational resources required for developing and testing the project's AI components and supporting software modules.

The major hardware requirements of the project are described below:

1. Processor

A powerful processor is required for dataset preprocessing, image and video processing, model preparation, application development, database operations, and general system execution. The project development environment uses an **AMD Ryzen 9 8945HS** processor. Its multi-core processing capability supports computational tasks that occur alongside GPU-based AI processing, including data loading, preprocessing, application execution, and backend operations.

2. Graphics Processing Unit (GPU)

A dedicated GPU is one of the most important hardware components for the project because YOLO11 and SAM2 are deep-learning-based computer-vision models. GPU acceleration significantly improves the speed of model training and inference compared with relying only on CPU processing.

The development system uses an **NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB of VRAM**. The GPU supports CUDA-based acceleration and is used for PyTorch-based model development and AI inference. The available VRAM also provides a practical environment for working with computer-vision models and training configurations suitable for the available hardware resources.

3. **Random Access Memory (RAM)**

Sufficient RAM is required for loading datasets, running development tools, executing AI frameworks, processing images and videos, and operating the backend and database components. The project development system contains **16 GB of DDR5-5600 RAM**. The available memory allows the development environment to run Python, PyTorch, model-development tools, database services, browsers, and other supporting applications. RAM requirements may increase when processing larger datasets, multiple video streams, or more computationally demanding models.

4. **Storage**

Storage is required for maintaining the project's source code, datasets, trained model checkpoints, segmentation annotations, processed images, video files, logs, database files, and application resources. The development system uses a **1 TB SSD**, providing sufficient high-speed storage for the current development environment.

SSD storage is particularly useful for AI projects because model training and inference frequently require reading large numbers of images and annotation files. Faster storage can reduce data-loading delays and improve the overall development workflow.

5. **Input Devices**

The project requires standard input devices such as a **keyboard and mouse** for software development, dataset management, database administration, model configuration, and dashboard development. These devices are also required for interacting with development environments and monitoring the system during testing.

6. **Display**

A suitable display is required for developing the web dashboard, inspecting detection results, reviewing segmentation outputs, examining datasets, and monitoring AI inference. A higher-resolution display can be useful when examining small objects such as helmets and number plates in traffic footage.

7. **Camera / Video Input Source**

The system is designed to process **traffic images and video footage**, which can be obtained from CCTV cameras, recorded traffic videos, or other supported visual sources. The quality of the camera input has a direct influence on the performance of the computer-vision models. Clear footage with adequate resolution, suitable lighting, and an

appropriate camera angle provides better information for detecting motorcycles, occupants, helmets, and other objects.

8. Network Connectivity

Internet connectivity is useful during development for downloading datasets, Python packages, model repositories, framework updates, documentation, and other required resources. Network access may also be required for transferring project files to repositories such as GitHub and for future deployment or remote camera-stream integration.

The principal hardware configuration used for the project can be summarized as follows:

Hardware Component	Specification
Processor	AMD Ryzen 9 8945HS
RAM	16 GB DDR5-5600
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
GPU Memory	8 GB VRAM
Storage	1 TB SSD
Input	Keyboard, Mouse, Traffic Images / Video / CCTV Input
Display	Laptop Display / External Display
Network	Internet Connectivity for Development and Resource Access

Overall, the hardware configuration provides a suitable development environment for the current RideSafe AI project. The combination of a high-performance processor, **16 GB RAM**, dedicated **RTX 4060 8 GB GPU**, and **1 TB SSD** supports the project's AI model development, dataset preparation, image and video processing, backend development, database operations, and dashboard implementation. For future expansion involving multiple simultaneous camera streams or larger and more computationally demanding models, additional computational resources or dedicated server/edge hardware may be considered.

2.2 Software Requirements

The development of **RideSafe AI** requires a combination of programming languages, artificial-intelligence frameworks, computer-vision libraries, database software, web-development technologies, and development tools. Since the project involves training and executing deep-learning models, processing traffic images and videos, managing datasets, storing violation records, and providing an administrative interface, the software environment must support both AI-based processing and conventional application development.

The project is primarily developed using **Python** because of its extensive support for machine learning, deep learning, computer vision, image processing, and backend development. A dedicated Python environment is used to maintain the required dependencies and prevent conflicts between different packages. The AI components are developed using **PyTorch**, **Ultralytics YOLO11**, and the **SAM2** framework.

The major software requirements of the project are described below:

1. **Operating System**

A Windows-based development environment is used for the project. The operating system provides the environment required for installing Python, CUDA-supported AI frameworks, development tools, database software, Git, and other supporting applications.

2. **Python**

Python 3.11.15 is used as the primary programming language for the AI and backend components of RideSafe AI. Python provides extensive libraries for deep learning, computer vision, data processing, model training, inference, and API development. The project uses a dedicated Python/Conda environment named **helmet** to manage the dependencies required for the computer-vision pipeline.

3. **PyTorch**

PyTorch is used as the primary deep-learning framework for training and executing the project's AI models. It provides GPU acceleration through CUDA and supports the development of neural-network-based computer-vision applications. The development environment uses **PyTorch 2.13.0 with CUDA 12.6 support**, allowing the NVIDIA RTX 4060 GPU to be utilized for computationally intensive model operations.

4. Ultralytics YOLO11

Ultralytics YOLO11 is used for object detection within the project. The framework provides the functionality required to train, validate, and perform inference using YOLO-based detection models.

The project uses the YOLO11 model family for identifying relevant objects such as motorcycles, persons, helmets, and other objects required by the traffic-monitoring pipeline.

5. SAM2

Segment Anything Model 2 (SAM2) is incorporated for image-segmentation-related tasks. The project uses the SAM2 repository and a **SAM2.1 Hiera Small** configuration as part of the segmentation development process.

SAM2 is particularly useful where more detailed object-level regions are required beyond conventional bounding-box detection. The project includes a dedicated helmet segmentation dataset prepared for this purpose.

6. OpenCV

OpenCV is used for image and video processing. It provides functionality for reading video files, extracting frames, resizing and preprocessing images, drawing detection results, manipulating images, and preparing visual data for AI models. OpenCV also provides the basic processing capabilities required for integrating AI inference with traffic-video input.

7. OCR / ANPR Software Components

The complete system architecture includes an **OCR/ANPR component** for number-plate recognition. This component is currently under development and is intended to process detected number-plate regions and extract registration information. Therefore, OCR/ANPR is considered part of the software requirements and planned technology stack, but its complete implementation and final software configuration will be documented after the module is developed and tested.

8. MySQL

MySQL is used as the relational database management system for the project. It is intended to store structured information related to users, cameras, vehicles, violation events, timestamps, available number-plate information, and supporting evidence. The database provides a centralized mechanism for storing and retrieving records generated by the AI pipeline.

9. FastAPI and Uvicorn

FastAPI is used as the planned backend framework for providing application programming interfaces between the AI-processing components, database, and frontend. **Uvicorn** is used as the ASGI server for running the FastAPI application. This backend layer provides the communication mechanism required to transfer structured detection and violation information between different components of the system.

10. HTML, CSS and JavaScript

The administrative monitoring interface is developed using standard web technologies including **HTML, CSS, and JavaScript**. These technologies are used to create the user interface, display detection and violation information, communicate with backend services, and provide interactive dashboard functionality.

11. Git and GitHub

Git is used for source-code version control, while **GitHub** is used as the remote repository for maintaining and managing the project source code. Version control helps track changes, maintain different versions of the project, and support collaborative development.

12. Development Environment

A code editor/development environment such as **Visual Studio Code** can be used for writing Python, HTML, CSS, JavaScript, configuration files, and other project resources. It also provides useful features for debugging, terminal access, project navigation, and source-code management.

13. Conda Environment Management

Conda is used to manage the Python development environment and dependencies. The project uses a dedicated environment named **helmet**, which isolates the required packages and helps maintain a consistent environment for AI model development.

Software Environment Summary

Software / Technology	Purpose
Windows	Development operating system
Python 3.11.15	Primary programming language

Software / Technology	Purpose
Conda – helmet environment	Python environment and dependency management
PyTorch 2.13.0 + CUDA 12.6	Deep-learning model development and GPU acceleration
Ultralytics YOLO11	Object detection
SAM2 / SAM2.1 Hiera Small	Image segmentation
OpenCV	Image and video processing
MySQL	Relational database management
FastAPI	Backend API development
Uvicorn	Backend application server
HTML / CSS / JavaScript	Web-based monitoring dashboard
Git	Version control
GitHub	Remote source-code repository
Visual Studio Code	Development and code editing environment
OCR/ANPR	Planned number-plate recognition component

Overall, the software environment of RideSafe AI combines deep-learning frameworks, computer-vision libraries, backend technologies, database management, and web-development tools to support the complete project lifecycle. The current environment is primarily focused on **YOLO11-based detection, SAM2 segmentation, dataset preparation, and core system development**, while components such as **OCR/ANPR and complete e-challan integration remain under development** and will be finalized as the corresponding modules are implemented and tested.

2.3 Development Tools and Frameworks

The development of **RideSafe AI** requires a combination of development tools, artificial intelligence frameworks, computer-vision libraries, database technologies, and source-code management tools. These tools support different stages of the project, including dataset preparation, AI model development, training, testing, backend development, database management, and administrative dashboard development. The selected technologies provide the required environment for developing a modular traffic-monitoring system while allowing individual components to be improved independently.

The major development tools and frameworks used or planned for the RideSafe AI project are described below:

1.Visual Studio Code

Visual Studio Code (VS Code) is used as the primary development environment for writing and managing project source code. It provides support for Python, JavaScript, HTML, CSS, configuration files, and other project resources. Its integrated terminal, debugging features, extensions, and project navigation capabilities make it suitable for managing the different components of the RideSafe AI system

2. Anaconda / Conda

Conda is used for managing the Python environment and project dependencies. A dedicated environment named **helmet** is used for the AI development workflow. Isolating the project dependencies within a separate environment helps avoid conflicts between different Python packages and provides a consistent environment for model training and testing.

3. PyTorch

PyTorch is the primary deep-learning framework used for developing and executing the AI components of the project. It provides tensor computation, neural-network functionality, model training, and GPU acceleration. The project environment uses PyTorch with CUDA support to utilize the NVIDIA RTX 4060 GPU during computationally intensive operations.

4. Ultralytics YOLO11

The **Ultralytics YOLO11 framework** is used for object-detection development. It provides tools for training, validating, and performing inference with YOLO models. In RideSafe AI, YOLO11 forms an important part of the detection pipeline for identifying motorcycles, persons, helmets, and other relevant objects required for traffic-violation analysis.

5. SAM2

SAM2 (Segment Anything Model 2) is used for image-segmentation-related development. The project uses the SAM2 framework and **SAM2.1 Hiera Small** configuration for developing the segmentation component. SAM2 provides more detailed object regions and complements the bounding-box information obtained through object detection.

6. OpenCV

OpenCV is used for computer-vision and image/video processing operations. It supports tasks such as reading video files, extracting frames, resizing images, preprocessing visual inputs, drawing detection results, and handling processed frames. It acts as an important supporting library between the raw traffic footage and the AI models.

7. FastAPI

FastAPI is used for backend API development. It provides a lightweight framework for connecting the AI-processing components with the database and frontend application. The backend can expose endpoints for submitting inputs, retrieving detection information, managing records, and communicating with the administrative dashboard.

8. Uvicorn

Uvicorn is used as the server for running the FastAPI backend. It provides an ASGI-compatible environment through which the backend application can receive and process requests from the frontend and other system components.

9. MySQL

MySQL is used as the relational database technology for the project. It provides structured storage for application and violation-related information. The database can maintain records such as detected violation types, timestamps, available vehicle information, confidence values, and supporting evidence references.

10. HTML, CSS and JavaScript

HTML, CSS, and JavaScript are used for developing the web-based administrative dashboard. HTML provides the structure of the interface, CSS controls its presentation and layout, and JavaScript provides interactive functionality and communication with backend APIs.

11. Git and GitHub

Git is used for source-code version control and maintaining different stages of the project. **GitHub** is used as the remote repository for storing and managing the project source code. These tools also support collaboration, backup, change tracking, and organized project development.

12. Label Studio

Label Studio is used as an annotation tool during the dataset-development process. It supports the creation and management of image annotations required for computer-vision and segmentation tasks. The project used annotation workflows for preparing and validating segmentation-related data.

13. Roboflow and Dataset Processing Tools

Roboflow and related dataset-processing workflows are used as sources and utilities for working with computer-vision datasets. Dataset preparation involves downloading, organizing, converting annotation formats, merging compatible datasets, removing duplicates, and preparing the final data for model development.

Development Tool Summary

Tool / Framework	**Purpose**
Visual Studio Code	Project development and source-code management
Conda	Python environment and dependency management
PyTorch	Deep-learning model development
Ultralytics YOLO11	Object detection
SAM2 / SAM2.1	Image segmentation
OpenCV	Image and video processing
FastAPI	Backend API development
Uvicorn	FastAPI application server
MySQL	Database management
HTML, CSS, JavaScript	Web dashboard development
Git	Version control

Tool / Framework	Purpose
GitHub	Remote code repository
Label Studio	Dataset annotation
Roboflow	Dataset management and preparation

Overall, these development tools and frameworks provide the technical foundation required for building RideSafe AI. The combination of **YOLO11, SAM2, PyTorch, OpenCV, FastAPI, MySQL, and web technologies** enables the project to connect AI-based traffic analysis with a complete software application. The current development focuses on the core detection, segmentation, dataset, backend, and database components, while technologies related to **OCR/ANPR and complete e-challan processing** can be integrated as those modules are developed and validated.

CHAPTER-3 TECHNOLOGY STACK

3.1 Artificial Intelligence and Computer Vision

Artificial Intelligence (AI) and **Computer Vision** form the core technological foundation of the **RideSafe AI** project. The proposed system uses AI-based techniques to analyze traffic images and video footage and identify objects and situations relevant to motorcycle-violation monitoring. Instead of depending entirely on continuous manual observation, computer-vision models can process visual information automatically and provide structured outputs that can be used by subsequent stages of the system.

Artificial Intelligence refers to the use of computational methods that enable systems to perform tasks that normally require human intelligence, such as recognition, classification, decision-making, and pattern analysis. In RideSafe AI, AI techniques are applied to traffic-scene understanding. The system is designed to recognize motorcycles, persons, helmets, and other relevant objects and use the detected information to identify potential traffic violations.

Computer Vision is a major field of AI concerned with enabling computers to interpret and understand information contained in images and videos. In a traffic-monitoring application, computer vision can be used to locate vehicles, identify people, analyze object relationships, and extract useful information from video frames. RideSafe AI applies these capabilities to motorcycle-focused traffic analysis.

The computer-vision pipeline begins with an input image or video frame. The input may contain several motorcycles, riders, pedestrians, vehicles, and other objects. The system processes the frame using trained AI models to identify objects of interest. Each detected object can be represented using information such as its class, location, and confidence value. These outputs provide the basic information required for further analysis.

A major requirement of the project is the detection of **motorcycles and persons**. Identifying these objects allows the system to establish which vehicles are present and which people may be associated with them. This is particularly important because the project does not simply aim to detect individual objects; it also aims to understand relationships between motorcycles and their occupants.

The system therefore uses computer-vision processing as a sequence of interconnected stages. First, relevant objects are detected. Next, the detected persons can be associated with motorcycles based on the system's processing logic. The associated occupants can then be analyzed for helmet compliance and occupant count. This approach allows the system to consider the complete motorcycle scene rather than making an isolated decision based on a single detected object.

One of the key advantages of AI-based computer vision is its ability to process large amounts of visual information. A CCTV camera can continuously generate video containing a large number of frames. Manually examining every frame is time-consuming and may result in important events being missed. An automated computer-vision system can process these frames and identify potentially relevant situations, thereby assisting human operators in reviewing traffic footage more efficiently.

However, computer vision in real-world traffic environments also presents several challenges. Objects may appear at different sizes and viewing angles, while lighting conditions can vary significantly. Motorcycles and riders may be partially hidden by other vehicles, pedestrians, or road infrastructure. Motion blur, low-resolution footage, overlapping objects, and crowded traffic conditions can also affect detection performance. Therefore, the project uses multiple computer-vision components rather than depending on a single processing technique.

Deep learning plays an important role in the computer-vision component of RideSafe AI. Deep-learning models can learn visual patterns from labeled datasets and use the learned representations to identify objects in previously unseen images. The project uses **PyTorch** as the primary deep-learning framework and incorporates **YOLO11** for object detection.

YOLO11 provides the object-detection capability required to identify relevant objects in traffic scenes. The model processes an image and produces detection results containing object classes, bounding-box coordinates, and confidence scores. These results can then be passed to subsequent stages of the RideSafe AI pipeline.

The project also incorporates **SAM2 (Segment Anything Model 2)** for segmentation-related tasks. Object detection generally represents objects using rectangular bounding boxes, whereas segmentation can provide a more detailed representation of the region occupied by an object. This can be useful when objects overlap or when more precise object-level information is required.

For the segmentation component, a dedicated dataset was prepared using multiple sources and annotation-processing steps. The datasets were converted into compatible formats, merged, and deduplicated before being organized into training, validation, and testing subsets. The final prepared dataset contains **1,027 unique images and 1,570 annotated object instances**, providing a foundation for developing the project's segmentation component.

Another important aspect of AI-based computer vision is **dataset preparation**. The performance of a model depends significantly on the quality and relevance of its training data. Images need to contain suitable examples of the objects and situations that the model is expected to recognize. Appropriate annotations are also required so that the model can learn the desired visual patterns.

In RideSafe AI, dataset preparation includes collecting relevant images, organizing datasets, converting annotation formats, merging compatible datasets, removing duplicate images, and creating appropriate training and validation structures. These processes are important for developing models that are better suited to the project's target traffic scenarios.

The AI and computer-vision layer also provide the foundation for the project's violation-analysis logic. Once motorcycles and occupants have been identified, the system can use the resulting information to analyze helmet compliance and occupant count. A potential helmet violation can be identified when the relevant occupant is determined to be without a helmet, while a multi-occupant condition can be identified by analyzing the number of occupants associated with a motorcycle.

The computer-vision architecture is designed to remain modular so that individual models and processing techniques can be improved independently. For example, the object-detection model can be retrained with additional data without requiring the database or dashboard to be completely redesigned. Similarly, improved segmentation or tracking techniques can be incorporated into later versions of the system.

The broader RideSafe AI architecture also includes planned **number-plate detection and OCR/ANPR** functionality. Computer vision will be used to locate the number-plate region associated with a detected vehicle, after which OCR techniques can be applied to extract the registration information. This component is currently under development and is therefore treated as a planned extension of the computer-vision pipeline rather than a fully completed module.

Overall, Artificial Intelligence and Computer Vision provide the foundation for transforming raw traffic footage into meaningful information for RideSafe AI. Through the combination of **YOLO11-based object detection, SAM2-based segmentation, image processing, dataset preparation, and multi-stage analysis**, the project establishes a framework capable of assisting in motorcycle-violation monitoring. The AI layer connects the visual input with subsequent occupant analysis, violation detection, database storage, and administrative monitoring, forming the central intelligence component of the overall system.

3.2 Object Detection using YOLO11

YOLO11 (You Only Look Once version 11) is one of the primary computer-vision technologies used in the **RideSafe AI** project for detecting objects present in traffic scenes. Object detection is an important stage of the proposed system because the subsequent modules depend on identifying motorcycles, persons, helmets, and other relevant objects before performing motorcycle–occupant association and violation analysis.

The main purpose of YOLO11 in RideSafe AI is to process an input image or video frame and identify objects of interest within that scene. Unlike traditional image-processing approaches that may require manually designed rules for identifying objects, YOLO11 uses a deep-learning-based approach in which the model learns visual patterns from annotated training data. During inference, the model provides information about the detected objects, including their class, location, and confidence score.

The YOLO11-based detection stage acts as an important entry point into the RideSafe AI processing pipeline. When a traffic image or video frame is provided as input, the frame is processed by the trained detection model. The model analyzes the visual content and generates bounding boxes around objects that match the classes for which it has been trained. These detections are then made available to the subsequent stages of the system for further processing.

For the RideSafe AI application, object detection is particularly important because traffic scenes may contain several motorcycles, riders, passengers, pedestrians, cars, and other objects simultaneously. The model must therefore identify the relevant objects while distinguishing them from unrelated elements in the scene. Motorcycle and person detection are especially important because they provide the basic information required for occupant association and

motorcycle-specific violation analysis.

The output of YOLO11 can be represented using several important attributes. The **bounding box** indicates the location of the detected object within the image. The **class label** identifies what type of object has been detected, while the **confidence score** represents the model's confidence in the corresponding prediction. These outputs can be used by the application to determine which detections should be passed to later processing stages.

The detection process can be represented as follows:

Input Image / Video Frame → YOLO11 Model → Object Detection → Bounding Boxes + Class Labels + Confidence Scores → Further Violation Analysis

One of the important advantages of using YOLO-based object detection is its suitability for processing images and video frames efficiently. Since traffic monitoring can involve a continuous stream of visual data, the ability to perform detection on successive frames is important for developing an AI-assisted monitoring system. The actual processing speed, however, depends on factors such as input resolution, model configuration, GPU resources, number of detected objects, and other processing stages in the pipeline.

YOLO11 in the RideSafe AI Pipeline

In the proposed RideSafe AI architecture, YOLO11 is used as an initial detection component. The output generated by the detector can be passed to other modules according to the requirements of the processing pipeline.

The general workflow is:

1. A traffic image or video frame is provided as input.
2. The input frame is preprocessed as required by the detection model.
3. YOLO11 analyzes the frame using its trained neural-network architecture.
4. Relevant objects such as motorcycles, persons, and helmets are detected.
5. Bounding boxes, class labels, and confidence values are generated.
6. Motorcycle and person detections are used for occupant-association processing.
7. Helmet-related detections can be used for helmet-compliance analysis.
8. The resulting information is passed to the violation-decision stage.
9. Potential violation information can then be stored or displayed through the application.

Training and Dataset Preparation

The performance of the YOLO11 detector depends on the quality and relevance of the training data. Therefore, dataset preparation is an important part of the object-detection component. The training data needs to contain representative examples of the objects that the model is expected to detect. The project involves collecting and organizing relevant traffic and helmet-related datasets and preparing them for model development. Dataset preparation may include image organization, annotation verification, class standardization, preprocessing, and separation of data into appropriate training, validation, and testing subsets. The purpose of these steps is to provide the model with suitable examples during training and to evaluate its performance on images that were not used during training.

Motorcycle Detection

Motorcycle detection is one of the most important functions of YOLO11 within RideSafe AI. The detected motorcycle provides the reference object around which subsequent occupant analysis is performed. Once a motorcycle is identified, the system can examine nearby person detections and determine which occupants are associated with that motorcycle. Accurate motorcycle detection is therefore important for reducing incorrect associations and ensuring that subsequent helmet and occupant analysis is performed on the appropriate vehicle.

Person Detection

Person detection is another important function because the system needs to identify the people associated with each detected motorcycle. A traffic scene may contain riders, passengers, pedestrians, and people associated with nearby vehicles. The person-detection output is therefore not treated as a final violation decision. Instead, it becomes an input to the motorcycle–occupant association stage. The association logic is responsible for determining which detected persons should be considered occupants of a particular motorcycle.

Helmet Detection

Helmet detection can provide additional information required for helmet-compliance analysis. The system can use helmet-related detections together with motorcycle and person information to determine whether the relevant occupant appears to be wearing a helmet. This multi-stage approach is important because a helmet detected somewhere in the frame should not automatically be interpreted as evidence that a particular motorcycle occupant is wearing it. The relationship between the detected helmet, person, and motorcycle must be considered during the subsequent processing stages.

Confidence-Based Processing

YOLO11 generates confidence information along with object detections. Confidence values can be used to establish thresholds for accepting or rejecting detections. A suitable threshold can help reduce low-confidence detections, although setting the threshold too high may also cause valid objects to be missed. The appropriate confidence threshold depends on the dataset, model training, camera conditions, and application requirements. Therefore, threshold selection and model evaluation form part of the testing and optimization process.

Integration with Other AI Components

YOLO11 is not intended to operate independently within RideSafe AI. Its output is combined with other components such as **SAM2**, occupant-association logic, helmet analysis, and the violation-decision layer.

Where more precise object regions are required, segmentation techniques can complement the bounding boxes generated by YOLO11. Similarly, the person and motorcycle detections can be processed together to estimate the number of occupants associated with a particular motorcycle. This modular integration allows the object-detection model to perform its specific role while other components handle segmentation, association, decision-making, database storage, and visualization.

Advantages of YOLO11 for the Project

The use of YOLO11 provides several practical advantages for RideSafe AI:

1. **Automated object detection:** Relevant traffic objects can be identified automatically from images and video.
2. **Multiple-object detection:** Several objects can be detected within the same traffic scene.
3. **Bounding-box localization:** The approximate location of each detected object is provided.
4. **Confidence information:** Each prediction can be evaluated using its associated confidence score.
5. **Integration capability:** Detection outputs can be passed to other AI and software modules.
6. **GPU acceleration:** The model can utilize the project's NVIDIA GPU through the PyTorch environment.
7. **Extensibility:** Additional object classes can be incorporated through suitable datasets and model training.

Limitations and Considerations

Although YOLO11 provides the primary object-detection capability, its performance can be affected by real-world traffic conditions. Small or distant motorcycles may be difficult to detect, while occlusion, motion blur, poor lighting, unusual camera angles, and low-resolution footage can reduce detection reliability. Crowded scenes can also make it difficult to correctly associate detected persons with the appropriate motorcycle.

For this reason, YOLO11 is treated as one component of a larger processing pipeline rather than as the complete solution to traffic-violation detection. Additional processing stages are required to interpret the detections and determine potential violations. Overall, **YOLO11 provides the fundamental object-detection layer of RideSafe AI**. It converts raw traffic imagery into structured object information that can be used by the subsequent motorcycle–occupant association, helmet analysis, multi-occupant detection, and violation-decision components. Its integration with SAM2, backend services, MySQL, and the monitoring dashboard form an important part of the overall AI-based traffic-monitoring architecture.

3.3 Image Segmentation using SAM2

Segment Anything Model 2 (SAM2) is an important component of the computer-vision technology stack used in the **RideSafe AI** project. While object detection models such as YOLO11 identify objects using bounding boxes, image segmentation provides a more detailed representation of the actual region occupied by an object. This additional information can be useful when objects overlap or when accurate object boundaries are required for further analysis.

In RideSafe AI, SAM2 is incorporated primarily for **helmet-related segmentation and computer-vision analysis**. The objective is to use segmentation information to obtain more precise object regions and complement the object-detection results generated by YOLO11. The combination of detection and segmentation provides a more comprehensive representation of the traffic scene.

SAM2 in the RideSafe AI Pipeline

The general role of SAM2 in the proposed system can be represented as:

Traffic Image / Video Frame → Object Detection → Relevant Object Region → SAM2 Segmentation → Precise Object Mask → Further Analysis

YOLO11 can initially identify the relevant object and provide its approximate location through a bounding box. SAM2 can then be used to obtain a more detailed segmentation mask corresponding to the required object region. This separation of responsibilities allows the detection and segmentation components to work together rather than requiring a single model to perform every task.

Importance of Segmentation

A bounding box represents an object using a rectangular region. However, the actual object may occupy only part of that rectangle. In crowded traffic scenes, neighboring objects can overlap with one another, making it difficult to determine the exact boundary of a motorcycle, rider, or helmet using only rectangular detections.

Segmentation provides information at the pixel or region level and can therefore provide a more accurate representation of the object. This can be useful when the system needs to distinguish an object from its surrounding background or from nearby objects.

For RideSafe AI, segmentation can be particularly useful in situations where helmets, riders, and motorcycles appear close together. More precise regions can support subsequent image analysis and provide additional information for improving the overall computer-vision pipeline.

SAM2 Dataset Preparation

A dedicated dataset was prepared for the SAM2 component of the project. Since segmentation models require appropriate mask annotations, the project involved collecting relevant datasets, processing their annotations, converting compatible segmentation formats, merging datasets, and removing duplicate images.

The final prepared SAM2 dataset contains:

- **1,027 unique images**
- **720 training images**
- **205 validation images**
- **102 test images**
- **1,570 total annotated object instances**
- **673 helmet instances**
- **897 without-helmet instances**

This dataset provides a structured foundation for developing and evaluating the segmentation component of RideSafe AI.

Dataset Organization

The dataset is divided into separate training, validation, and testing subsets. The training set is used for model development, while the validation set can be used to monitor model performance during development and assist with configuration selection. The test set is reserved for evaluating the developed model on data that is not used during training.

This separation is important because evaluating a model only on the images used during training does not provide a reliable indication of how well it can generalize to new traffic scenes.

SAM2.1 Hiera Small Configuration

The project uses the **SAM2.1 Hiera Small** configuration as part of the segmentation development. The corresponding SAM2 repository and configuration files are maintained within the project environment.

The training configuration was prepared according to the available GPU resources. The documented configuration includes a training batch size of **1**, one frame per sample, a base learning rate of $\mathbf{5.0 \times 10^{-6}}$, a vision learning rate of $\mathbf{3.0 \times 10^{-6}}$, and a planned training duration of **40 epochs**.

These parameters are selected with consideration for the available **NVIDIA RTX 4060 Laptop GPU with 8 GB VRAM** and the computational requirements of the segmentation model.

Training Process

The SAM2 training process involves providing the model with images and their corresponding segmentation annotations. During training, the model learns visual characteristics that allow it to distinguish the target regions from their surrounding areas.

The general training workflow consists of:

1. Preparing the annotated segmentation dataset.
2. Dividing the dataset into training, validation, and testing subsets.
3. Configuring the SAM2.1 Hiera Small model.
4. Loading the appropriate model checkpoint and configuration.
5. Training the model using the prepared dataset.
6. Monitoring the training process and generated checkpoints.
7. Evaluating the trained model using validation and test data.
8. Integrating the resulting segmentation capability into the overall computer-vision pipeline.

Integration with YOLO11

SAM2 and YOLO11 have different responsibilities within RideSafe AI. YOLO11 primarily provides object detection, while SAM2 provides more detailed segmentation information.

The two components can therefore work together in the following manner:

YOLO11 → Detect Relevant Object → Identify Region of Interest → SAM2 → Generate Segmentation Mask

This approach allows YOLO11 to provide an efficient initial localization while SAM2 can provide more detailed information about the selected object region.

Role in Helmet Analysis

Helmet-related analysis is an important part of the RideSafe AI project. The SAM2 component provides a way to obtain detailed regions associated with helmet-related objects and can complement the detection results generated by YOLO11.

The segmentation output can be used as an additional source of visual information when analyzing helmet-related regions. This can be particularly useful in cases where bounding boxes contain significant background information or where objects overlap.

However, segmentation alone does not determine whether a traffic violation has occurred. The final violation decision depends on the combined information from motorcycle detection, occupant association, helmet analysis, and the configured violation rules.

Advantages of SAM2

The use of SAM2 provides several potential advantages for the project:

1. **Detailed object representation:** Segmentation provides more precise object regions than simple bounding boxes.
2. **Improved region analysis:** Precise masks can assist subsequent image-processing operations.
3. **Handling of complex object boundaries:** Segmentation can better represent irregular object shapes.
4. **Complementary to object detection:** SAM2 can work alongside YOLO11 rather than replacing the detection model.
5. **Potential improvement in crowded scenes:** More detailed object regions can assist with separating relevant objects from their surroundings.
6. **Extensible architecture:** The segmentation component can be further improved or extended as additional datasets become available.

Limitations and Considerations

The performance of a segmentation model depends heavily on the quality of the training data and annotations. Incorrect or inconsistent masks can negatively affect model learning. Similarly, difficult traffic conditions such as severe occlusion, low-resolution footage, motion blur, and poor lighting can affect segmentation quality.

The computational requirements of SAM2 must also be considered. Segmentation can require significant GPU memory and processing resources, particularly when processing high-resolution images or multiple frames. Therefore, the current project configuration is designed with the available RTX 4060 8 GB GPU in mind.

Current Development Status

The SAM2 component is part of the project's current AI development work, including dataset preparation, configuration, and model training. The segmentation component is being developed alongside the YOLO11 detection pipeline.

The project does not treat SAM2 as a standalone solution for complete traffic-violation detection. Instead, it forms one component of the larger RideSafe AI architecture, where its output can complement object detection and contribute to more detailed visual analysis.

Overall, **SAM2 provides the segmentation layer of the RideSafe AI computer-vision pipeline**. By combining detailed segmentation with YOLO11-based object detection, the project aims to obtain more useful visual information for helmet and motorcycle-related analysis. The prepared segmentation dataset and SAM2.1 Hiera Small configuration provide the foundation for continuing development and evaluation of this component within the overall intelligent traffic-monitoring system.

3.4 Backend Technologies

The backend is an important part of the RideSafe AI system because it provides the communication layer between the artificial intelligence modules, database, and application interface. While the AI components are responsible for processing traffic images and identifying potential motorcycle violations, the backend is responsible for receiving the processing results, organizing the information, communicating with the database, and making the required information available to the frontend and administrative dashboard.

The RideSafe AI system uses **Python-based backend technologies**, with **FastAPI** serving as the primary backend framework and **Uvicorn** used as the application server. This combination provides a lightweight and efficient environment for developing APIs that can communicate with the AI processing modules and database components.

Role of the Backend

The backend acts as an intermediate layer between the AI processing system and the application interface. When a traffic image or video frame is processed, the AI modules generate detection information such as detected motorcycles, persons, helmets, occupant associations, and potential violation details. The backend can receive these outputs and organize them into structured information for further processing.

The general backend communication flow can be represented as:

AI Processing Modules → Backend API → Database → Frontend / Admin Dashboard

This architecture separates AI processing from application-level operations. Such separation makes the system easier to maintain and allows individual modules to be modified or extended without requiring major changes to the complete application.

FastAPI

FastAPI is used as the backend web framework for developing application programming interfaces. It is a Python-based framework designed for building APIs and is suitable for applications that require communication between machine-learning components and web interfaces.

In RideSafe AI, FastAPI can provide endpoints through which the frontend or other application components communicate with the backend. These endpoints can be used for operations such as submitting data, retrieving violation records, obtaining detection results, and communicating with database operations.

The use of FastAPI also allows the AI models and backend services to remain within the Python ecosystem. This is useful because the project's computer-vision and machine-learning components are also implemented using Python-based technologies such as PyTorch, Ultralytics YOLO11, and SAM2.

Uvicorn

Uvicorn is used as the application server for running the FastAPI backend. It provides the environment required to start and serve the FastAPI application during development and deployment.

The basic architecture can therefore be represented as:

Client / Frontend → Uvicorn → FastAPI Application → AI / Database Services

During development, Uvicorn allows the backend application to be started locally so that API endpoints can be tested before integrating them with the complete dashboard and deployment environment.

Communication with AI Modules

One of the important responsibilities of the backend is to provide communication between the application layer and the AI modules. The AI pipeline may generate structured results from traffic images or video frames, including detected objects and potential violation information.

The backend can organize these outputs into application-level records. For example, a processed traffic event may contain information such as:

- Detection identifier
- Motorcycle detection
- Associated occupants

- Helmet status
- Multi-occupant status
- Violation type
- Confidence information
- Processing timestamp
- Additional event information

This structured approach makes the AI results easier to store, display, and process.

API-Based Architecture

The use of APIs provides a standardized method for communication between different parts of the RideSafe AI application. Instead of directly connecting the frontend to the AI models or database, the frontend can communicate with the backend through defined API endpoints.

A simplified architecture is:

Frontend_Interface
↓
FastAPI
↓
AIProcessing/BusinessLogic
↓
MySQL Database

This architecture improves modularity because the frontend, backend, AI models, and database can be developed and tested independently.

Backend and Database Integration

The backend also provides the connection between the application and the MySQL database. When a violation-related event is generated, the backend can process the information and store the required details in the database.

Similarly, when the administrator requests previously generated records through the dashboard, the backend can retrieve the corresponding information from MySQL and return it to the frontend.

This creates a two-way communication process:

Frontend → Backend → MySQL

And

MySQL → Backend → Frontend

Such an architecture helps maintain a centralized and structured storage system for application records.

Backend Security and Validation

The backend layer can also provide validation of incoming data before it is processed or stored. API requests can be checked to ensure that required fields are present and that the received information follows the expected format.

Authentication and authorization mechanisms can also be incorporated into the backend for administrative functions. This is particularly relevant for a traffic-monitoring system where access to violation records and administrative information should be controlled.

Advantages of the Selected Backend Technologies

The selected backend technologies provide several benefits for the RideSafe AI project:

1. **Python Compatibility:** FastAPI integrates naturally with the Python-based AI and computer-vision modules.
2. **API-Based Communication:** REST APIs provide a structured communication mechanism between application components.
3. **Modular Architecture:** AI, database, backend, and frontend components can be developed separately.
4. **Lightweight Development:** FastAPI and Uvicorn are suitable for rapid development and testing.
5. **Database Integration:** The backend can communicate with the MySQL database for storing and retrieving records.
6. **Future Scalability:** Additional APIs and services can be added as the project grows.
7. **AI Integration:** Python-based AI models can be integrated with backend processing without requiring a separate programming environment.
8.

Role in RideSafe AI

The backend provides the foundation for connecting the AI processing pipeline with the remaining application components. YOLO11 and SAM2 perform the major computer-vision operations, while the backend organizes their outputs and communicates them to the database and frontend.

The architecture also supports future development of additional modules such as number-plate recognition, OCR/ANPR processing, advanced violation records, notifications, and authorized e-challan integration. These features are part of the intended system expansion and are not being presented as fully completed components at the current stage.

Overall, the backend technologies provide the communication and service layer required to connect the AI models,

database, and user interface of RideSafe AI. The use of **FastAPI and Uvicorn** provides a suitable Python-based foundation for developing the application's APIs and integrating the AI-driven traffic monitoring pipeline with the rest of the system.

3.5 Database – MySQL

Database management is an important part of the RideSafe AI system because the application generates and processes different types of information during traffic monitoring and violation analysis. The artificial intelligence modules identify objects and analyze traffic scenes, while the database provides a structured and persistent method for storing the relevant information. For the RideSafe AI project, **MySQL** is used as the relational database management system for maintaining structured application and violation-related records.

MySQL organizes information using tables, rows, and columns, making it suitable for applications where different categories of information need to be stored and related systematically. In RideSafe AI, the database is intended to maintain information generated by the AI processing modules and make these records available to the backend and administrative dashboard.

Role of MySQL in RideSafe AI

The primary role of MySQL in the project is to provide persistent storage for information generated during the processing of traffic images and video. The AI modules may identify motorcycles, persons, helmet conditions, occupant information, and potential traffic violations. The relevant information can then be processed by the backend and stored as structured database records.

The database also allows previously stored information to be retrieved when required. This is particularly useful for administrative monitoring, reviewing detected violations, maintaining historical records, and displaying information through the dashboard.

The database can maintain information such as:

1. Violation identification details.
2. Date and time of the detected event.
3. Type of detected violation.
4. Motorcycle and occupant-related information.
5. Helmet compliance information.
6. Multi-occupant or triple-riding information.
7. AI detection confidence information.
8. Evidence or image references.
9. Processing or review status.
10. Administrative information where required.

The database structure can be extended as additional modules are developed during the project.

Relational Database Structure

MySQL follows a relational database model in which information is stored in organized tables. Related information can be maintained in separate tables and connected through appropriate relationships. This approach helps avoid unnecessary duplication and makes the stored information easier to manage.

For RideSafe AI, the database can logically maintain different categories of information related to the system. Violation records can contain details about detected events, while detection-related information can maintain relevant AI outputs. Occupant-related information can be associated with the corresponding motorcycle and violation record.

A structured relational approach is particularly useful because the project involves multiple stages of processing. The information generated during detection and violation analysis can be organized into meaningful records instead of being stored as unstructured data.

Storage of Violation Information

One of the major applications of MySQL in RideSafe AI is the storage of traffic violation information. When the system identifies a potential violation, the relevant details can be stored in the database for further review and management.

A violation record may contain information such as the unique record identifier, violation type, date and time, occupant count, helmet status, detection confidence, evidence reference, and current record status.

A typical record structure can be represented as follows:

Field	Description
Violation ID	Unique identification number for the violation record
Date and Time	Time associated with the detected event
Violation Type	Category of the detected violation
Occupant Count	Number of occupants associated with the motorcycle
Helmet Status	Helmet compliance information
Confidence	Relevant AI detection confidence
Evidence Reference	Reference to the associated image or evidence
Status	Current processing or review status

The exact database fields can be modified according to the requirements of the final implementation.

Backend and Database Integration

MySQL is integrated with the backend layer of the RideSafe AI application. The backend provides the communication interface through which application components can interact with the database. This prevents the frontend from having to directly access the database and provides a separate application layer for handling database operations.

When the AI processing system produces relevant information, the backend can validate and organize the information before storing it in MySQL. Similarly, when the administrative dashboard requests stored records, the backend can retrieve the required information from the database and provide it to the frontend.

This separation between the database, backend, and frontend improves the modularity of the application and makes the individual components easier to maintain.

Database Operations

The RideSafe AI application requires several standard database operations for maintaining its records. These operations can be performed through SQL queries and backend database functions.

The major operations include:

1. **Insert Operation:** Used to add newly generated violation or detection records to the database.
2. **Select Operation:** Used to retrieve stored records for viewing and analysis.
3. **Update Operation:** Used to modify information such as processing status or other editable fields.
4. **Delete Operation:** Used to remove records when permitted by the system's administrative requirements.
5. **Search and Filtering:** Used to retrieve records based on criteria such as violation type, date, or status.

These operations provide the basic database functionality required by the monitoring and administrative components of the project.

Database and AI Modules

The database is responsible for storing the outputs and application information generated by the AI pipeline, while the actual computer-vision processing is performed by the AI models. YOLO11 is used for object detection and SAM2 is used for image segmentation and related analysis.

The separation of responsibilities allows the AI models and database to operate as independent components. Changes to the detection or segmentation models can therefore be made without requiring the complete database system to be redesigned.

For example, if the AI model is retrained with a new dataset or its detection configuration is modified, the database can continue to store the resulting structured information according to the application's existing data model.

Database Support for the Admin Dashboard

MySQL also supports the administrative monitoring component of RideSafe AI. The dashboard can retrieve stored information from the database and present it to authorized users in an organized format.

The information displayed through the dashboard may include stored violation records, violation categories, detection dates, occupant information, helmet-related results, and available evidence references. This provides a centralized method for reviewing information generated by the AI-based monitoring system.

The database therefore acts as a persistent source of information for the dashboard rather than requiring all information to be generated again whenever the administrator opens the application.

Advantages of MySQL

The use of MySQL provides several advantages for the RideSafe AI project:

1. **Structured Data Management:** Information can be organized into well-defined relational tables.
2. **Persistent Storage:** Records can be maintained after individual traffic-processing operations are completed.
3. **Efficient Querying:** SQL queries allow stored information to be searched, filtered, and retrieved.
4. **Data Consistency:** Relational database constraints can help maintain consistent information.
5. **Backend Compatibility:** MySQL can be integrated with the Python-based backend used by the project.
6. **Scalability:** Additional tables and relationships can be introduced as the project expands.
7. **Administrative Support:** Stored records can be retrieved and displayed through the monitoring dashboard.
8. **Easy Management:** Database records can be managed using standard SQL operations and database-management tools.

Future Database Expansion

The current database architecture provides a foundation for extending RideSafe AI with additional functionality. As the project progresses, the database can be expanded to accommodate more detailed vehicle information, number-plate information, OCR/ANPR results, evidence management, user roles, notifications, and other application-related records.

The **number-plate recognition, OCR/ANPR, and automated e-challan generation components are still under development** in the current project stage. Therefore, these functionalities should not be considered fully implemented database features at present. Their corresponding information can be incorporated into the database as those modules are developed and integrated into the complete system.

Summary

MySQL provides the structured data-management foundation of the RideSafe AI application. It enables the system to store, retrieve, update, and manage information generated during AI-based traffic monitoring and violation analysis. Its relational structure is suitable for maintaining motorcycle, occupant, helmet, violation, and administrative information in an organized manner.

The integration of MySQL with the backend and administrative dashboard also supports the modular architecture of the project. By maintaining a centralized database, RideSafe AI can preserve processed information for later review and management. The database can further be extended as additional modules, including number-plate recognition, OCR/ANPR, and future authorized e-challan support, are developed and integrated into the system.

3.6 Front-End Technologies

The front-end of the RideSafe AI system provides the interface through which users can interact with the developed application and view the results generated by the AI-based traffic monitoring modules. Unlike conventional web applications that require separate HTML, CSS, and JavaScript development, the RideSafe AI application uses **Python and Streamlit** to create and run its interactive user interface. The application is launched using the streamlit run app.py command, which starts the Streamlit application and provides the interface through a web browser.

Streamlit is a Python-based framework designed for developing interactive data and machine-learning applications. Its integration with Python makes it particularly suitable for RideSafe AI because the project's major AI and computer-vision components are also developed using Python. This allows the user interface and AI processing components to be integrated within a common development environment.

Streamlit

Streamlit is the primary front-end framework used in the RideSafe AI application. It provides the components required to build an interactive interface without requiring the project to separately implement a traditional HTML, CSS, and JavaScript front-end.

The application interface is developed through Python code in the app.py file. Streamlit converts the Python instructions into an interactive browser-based interface that can be used to interact with the application.
The application is started using the following command:

streamlit run app.py

After execution, Streamlit starts the application and makes the interface available through a web browser. This approach simplifies the development process because the same Python environment can be used for the AI models, image processing, application logic, and user interface.

Python-Based User Interface

Python plays a central role in the front-end development of RideSafe AI because Streamlit applications are written directly in Python. The interface can contain different components such as headings, buttons, file upload controls, images, tables, status information, and other interactive elements.

This approach is particularly useful for the project because the AI processing pipeline already depends on Python-based technologies such as **PyTorch, Ultralytics YOLO11, SAM2, and OpenCV**. The Streamlit interface can therefore directly interact with the processing functions implemented within the same Python application environment.

The use of Python for both AI processing and interface development also reduces the need for additional front-end programming languages and frameworks.

Application Interface

The Streamlit interface provides a way for users to interact with the RideSafe AI system. Depending on the current implementation, the interface can be used to provide input to the system, execute the required processing operations, and display the resulting information.

The interface can include components such as:

1. **Application Title and Information:** Provides the name and purpose of the RideSafe AI application.
2. **Input Controls:** Allows users to provide the required image, video, or other input to the application.
3. **Processing Controls:** Provides buttons or other controls for initiating AI-based processing.
4. **Detection Results:** Displays the results generated by the AI modules.
5. **Images and Visual Results:** Allows processed images or detection outputs to be displayed.
6. **Violation Information:** Presents relevant violation-related information generated by the system.
7. **Status Information:** Provides feedback regarding the current processing operation.

The exact interface components can be expanded as the project continues to develop.

Integration with AI Models

One of the major advantages of using Streamlit is its ability to work directly with Python-based machine-learning and computer-vision components. In RideSafe AI, the interface can communicate with the AI processing functions responsible for traffic-scene analysis.

The YOLO11 model can be used for object detection, while SAM2 can provide segmentation-related processing. Streamlit provides the user-facing layer through which these processing operations can be initiated and their results displayed.

This integration reduces the separation between the AI development environment and the application interface. The models and interface can operate within the same Python project structure, making development and testing more convenient.

Display of Detection Results

The Streamlit interface can be used to present the output generated by the AI processing pipeline. Depending on the processing stage, the application may display detected motorcycles, persons, helmet-related results, occupant information, and potential violation information.

Visual results are particularly useful for the RideSafe AI project because computer-vision systems are easier to verify when the user can inspect the processed image or video output. The interface therefore provides a practical method for testing and demonstrating the results of the AI models.

The displayed information can also help during development by allowing the project team to inspect whether the detection and segmentation components are producing the expected results.

Streamlit and Database Integration

The Streamlit interface can also be integrated with the backend and MySQL database components of the system. Information stored in the database can be retrieved and displayed through the application interface when the corresponding functionality is implemented.

This allows the application to provide a centralized interface for viewing stored information generated during traffic violation analysis. The database remains responsible for persistent storage, while Streamlit provides the user-facing mechanism for accessing and presenting the information.

Advantages of Streamlit

The use of Streamlit provides several advantages for the RideSafe AI project:

1. **Python-Based Development:** The interface can be developed using the same Python environment used for the AI models.
2. **Simple Implementation:** Interactive interfaces can be created without developing a separate traditional frontend application.
3. **AI Integration:** Streamlit can directly interact with Python-based AI and computer-vision libraries.
4. **Rapid Development:** Interface components can be developed and modified quickly.
5. **Interactive Visualization:** Images, tables, results, and status information can be displayed directly.
6. **Easy Execution:** The application can be started using the streamlit run app.py command.
7. **Suitable for AI Demonstration:** The framework is well suited for demonstrating machine-learning and computer-vision applications.

Current Application Status

The current RideSafe AI application uses **Streamlit as the primary front-end framework**, with the main application being executed through app.py. Therefore, the project documentation should not describe HTML, CSS, and JavaScript as the technologies used to build the primary interface.

The current interface is focused on supporting the AI-based traffic-monitoring workflow and displaying the results generated by the developed modules. Additional interface functionality can be incorporated as the remaining system components are completed.

The **number-plate recognition, OCR/ANPR, and automated e-challan generation modules are still under development**. Consequently, the current Streamlit interface should not be described as providing a fully completed automated e-challan workflow.

Summary

Streamlit provides the primary front-end technology for the RideSafe AI application. By using a Python-based interface, the project can directly connect the user interface with its AI and computer-vision components without requiring a separate HTML, CSS, and JavaScript application.

The use of Streamlit also simplifies application execution, testing, and demonstration. The app.py application provides the main entry point for the interface and can be launched using the streamlit run app.py command. This Python-centered architecture is well suited to the current development of RideSafe AI and provides a foundation for incorporating additional monitoring, visualization, database, and future enforcement-support features as the project progresses.

3.7 Version Control

Version control is an important part of the development process of the RideSafe AI project because the system consists of multiple components, including AI models, datasets, Python application files, configuration files, database-related components, and documentation. During development, these components may undergo frequent modifications as new features are implemented, existing modules are improved, and errors are corrected. Version control provides a systematic way to manage these changes and maintain the project source code in an organized manner.

For the RideSafe AI project, Git is used as the version control system, while GitHub is used as the remote repository for storing and managing the project source code. The combination of Git and GitHub provides a structured development workflow and allows the project files to be maintained in a centralized repository.

Git

Git is a distributed version control system used to track changes made to project files. It allows developers to maintain different versions of the source code and provides a record of modifications made during development.

In the RideSafe AI project, Git can be used to track changes to important project components such as Python source files, Streamlit application files, configuration files, backend components, and other relevant project resources.

Git also allows developers to return to previous versions of files when required. This is useful during AI application development because modifications to model-processing code or application logic may sometimes introduce unexpected errors.

GitHub

GitHub is used as the remote repository platform for the project. It provides online storage for the Git repository and allows the project source code to be maintained remotely.

The RideSafe AI project repository can contain the relevant source-code structure required for the application, including the Streamlit application, Python modules, configuration files, and supporting project resources.

GitHub also provides a convenient way for project members to access the latest version of the source code and maintain the project as a collaborative software-development activity.

Git Repository

A Git repository maintains the tracked files and their change history. When a project is initialized as a Git repository, Git begins tracking the files that are added to version control.

Common Git operations used during development include:

1. git init – Initializes a new Git repository.
2. git status – Displays the current state of tracked and untracked files.
3. git add – Adds selected files to the staging area.
4. git commit – Saves a set of changes as a version in the repository.
5. git branch – Manages different development branches.
6. git remote – Manages the connection to a remote repository.
7. git push – Uploads committed changes to the remote GitHub repository.
8. git pull – Retrieves changes from the remote repository.

These operations provide the basic workflow required for maintaining the project's source code.

Project Development and Change Tracking

During the development of RideSafe AI, different components may require repeated modifications. For example, changes may be made to the YOLO11 detection process, SAM2 segmentation configuration, Streamlit application, backend functionality, or database integration.

Git allows these changes to be recorded as separate commits. Each commit represents a particular state of the

project at a specific point in development. This provides a useful history of the development process and makes it easier to identify when particular changes were introduced.

Version control is particularly useful for an AI-based project because model-development code and application code can change frequently during experimentation and testing.

Collaboration

GitHub can also support collaboration among the project team members. Since RideSafe AI is developed as a team project, maintaining the source code in a shared repository provides a centralized location for the project files.

Team members can work on different components of the application and synchronize their changes through Git. Proper repository organization and commit practices can reduce the possibility of accidentally losing important source-code changes.

GitHub can also provide a record of contributions made to the repository, which is useful for maintaining the development history of a team-based academic project.

Repository Organization

A well-organized repository is important for maintaining the RideSafe AI project. The repository can contain separate directories for application code, models, configuration files, datasets or dataset-related files, and supporting resources according to the requirements of the project.

Large model checkpoints and datasets may require separate storage or appropriate repository-management practices because they can occupy significant amounts of storage space. Only the files required for source-code management and reproducibility should be maintained in the Git repository according to the project's storage requirements.

Benefits of Version Control

The use of Git and GitHub provides several benefits to the RideSafe AI project:

1. Change Tracking: Changes made to project files can be recorded over time.
2. Version Management: Previous versions of the source code can be referenced when required.
3. Collaboration: Multiple team members can work on the project using a shared repository.
4. Remote Backup: The GitHub repository provides remote storage for the project source code.
5. Error Recovery: Earlier committed versions can help recover from unwanted changes.
6. Project Organization: Source-code changes can be managed systematically.

7. Development History: Commits provide a record of the project's development process.
8. Future Maintenance: The repository provides a useful starting point for future improvements and extensions.

Version Control in the Current Project

Git and GitHub are used to manage the development of the RideSafe AI project and its application source code. The project is developed incrementally, with different components being implemented and tested during the development process. Version control helps maintain the source code while these components evolve.

The use of GitHub also provides a centralized repository for the project and makes it easier to maintain the latest version of the application source code. As the project continues to develop, additional modules can be incorporated into the repository while maintaining the existing development history.

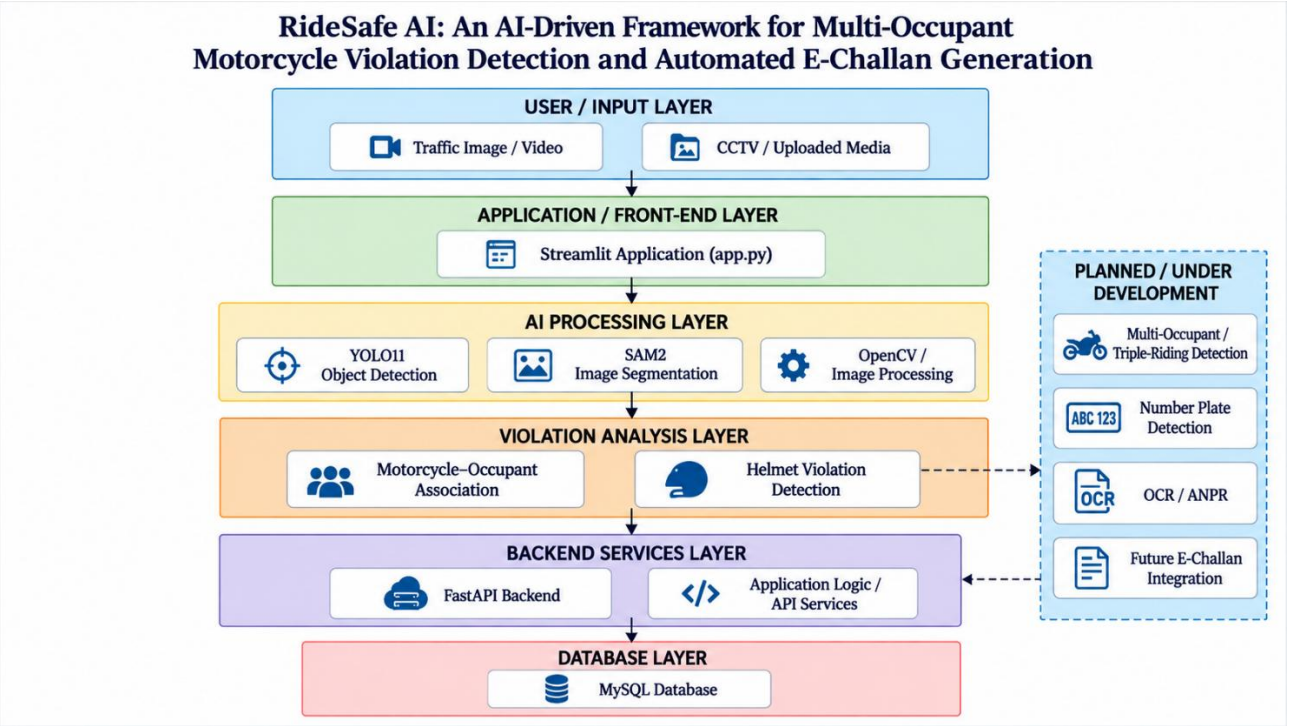
Summary

Version control provides an important foundation for managing the RideSafe AI software-development process. Git is used to track changes and maintain versions of the project source code, while GitHub provides the remote repository for storing and managing the project.

By maintaining the project under version control, the development team can organize changes, preserve previous versions, collaborate more effectively, and maintain a history of the software-development process. This provides a reliable foundation for continuing the development and future enhancement of the RideSafe AI system.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



4.2 AI Processing Pipeline

The AI processing pipeline is the central processing component of the RideSafe AI system. It is responsible for analyzing traffic images or video frames and extracting the visual information required for motorcycle violation analysis. The pipeline combines object detection, image processing, segmentation, motorcycle–occupant association, and helmet-related analysis to identify potential violations from the available traffic footage.

The processing pipeline is designed as a sequence of interconnected AI operations in which the output generated by one stage provides information required by the following stage. The use of multiple computer-vision techniques allows the system to analyze traffic scenes in greater detail than would be possible using a single detection model.

Input Processing

The pipeline begins with a traffic image or video provided to the application. The input may contain one or more motorcycles, riders, passengers, helmets, and other objects present in the surrounding environment.

Before performing AI analysis, the input is processed into a suitable format for the detection models. In the case of video input, individual frames can be obtained for analysis. Image-processing operations can also be applied where required to ensure that the input is suitable for further processing.

The quality of the input is an important factor in the overall performance of the system. Poor lighting, motion blur, low resolution, occlusion, and unsuitable camera angles can affect the ability of the AI models to identify relevant objects.

Object Detection using YOLO11

The first major AI operation is object detection using **YOLO11**. The model is used to identify relevant objects present in the traffic scene and provide their corresponding detection information.

The detection stage is particularly important because the subsequent processing depends on identifying motorcycles and persons within the input scene. The model provides bounding-box information and confidence values for detected objects.

The YOLO11 detection results provide the initial information required for further analysis, including motorcycle identification, person detection, and relevant helmet-related detection.

Image Segmentation using SAM2

After the relevant regions have been identified, **SAM2** is used to provide detailed segmentation information where required. Object detection provides rectangular bounding regions, while segmentation provides more precise information about the visible region of an object.

SAM2 is particularly useful for analyzing complex traffic scenes in which motorcycles, riders, and other objects may overlap. More precise object regions can help the system perform detailed visual analysis and support the subsequent violation-detection stages.

The project uses the **SAM2.1 Hiera Small** architecture for the segmentation component. A dedicated helmet-related segmentation dataset was prepared for model development, consisting of **1,027 unique images**, with **720 training images, 205 validation images, and 102 test images**. The dataset contains **1,570 annotated object instances**, including **673 helmet instances and 897 without-helmet instances**.

Motorcycle and Occupant Analysis

After the detection and segmentation stages, the system analyzes the relationship between motorcycles and detected persons. This stage is necessary because a traffic scene may contain several motorcycles and people at the same time.

The system attempts to determine which detected persons are associated with each motorcycle by considering their spatial relationship and position within the traffic scene. The associated occupants can then be analyzed individually.

This stage provides the information required to determine the number of occupants associated with a motorcycle and supports further analysis of helmet compliance and potential multi-occupant violations.

Helmet Violation Analysis

Helmet analysis is performed using the information obtained from the object detection, segmentation, and occupant-association stages. The system examines the relevant occupant regions to determine whether helmet compliance can be identified from the available visual information.

The analysis is performed in the context of the associated motorcycle rather than simply considering every helmet-related detection in the complete image. This helps prevent unrelated objects or people in the surrounding traffic scene from being incorrectly considered part of the same motorcycle event.

The output of this stage contributes to the identification of potential helmet-related violations.

Multi-Occupant and Triple-Riding Analysis

The occupant information obtained during the motorcycle–occupant association stage can also be used to analyze multi-occupant conditions. The number of persons associated with an individual motorcycle can be considered during violation analysis.

Cases involving more than the permitted number of occupants can be identified as potential multi-occupant or triple-riding violations. However, this component is currently **under development** and should not be presented as a completely finalized automated enforcement module.

Factors such as occlusion, overlapping riders, low-resolution footage, and incorrect person–motorcycle association can affect the accuracy of this analysis. Therefore, further development and testing are required to improve its reliability under different traffic conditions.

Violation Decision Layer

The results obtained from the different AI-processing stages are combined during violation analysis. Instead of relying on a single detection, the system considers information such as motorcycle detection, associated occupants, helmet status, and occupant count.

The violation decision layer is intended to distinguish between normal motorcycle activity and potential violation conditions. This provides a structured mechanism for converting raw computer-vision predictions into application-level information.

The resulting information can subsequently be passed to the backend for storage and presentation through the Streamlit application.

Processing Confidence and Visual Conditions

AI-based traffic analysis is affected by the quality and characteristics of the input data. The system therefore needs to consider detection confidence and visual conditions when interpreting AI results.

Important factors include:

1. **Lighting:** Bright sunlight, shadows, and low-light conditions can affect object visibility.
2. **Occlusion:** Riders or helmets may be partially hidden by other objects.
3. **Motion Blur:** Moving motorcycles can produce blurred images.
4. **Camera Angle:** Different camera positions can change the appearance and relative positions of objects.
5. **Image Resolution:** Distant objects may contain insufficient visual information.
6. **Traffic Density:** Crowded scenes can increase the difficulty of object association.
7. **Object Overlap:** Multiple motorcycles and persons may overlap within the same image region.

These factors are considered important during model evaluation and system testing because they directly influence the quality of the AI-generated results.

Backend Integration

After the AI processing and violation-analysis stages, the relevant information can be passed to the backend application. The backend is responsible for organizing the information and communicating with the MySQL database and application interface.

The AI processing layer and backend are maintained as separate logical components so that changes to the AI models can be made without requiring a complete redesign of the application layer.

The processed information can be used by the Streamlit interface for displaying detection results and by the database for maintaining structured records.

Current Development Status

The current AI processing pipeline focuses primarily on the core computer-vision components of RideSafe AI, including **YOLO11-based object detection, SAM2-based segmentation, motorcycle–occupant analysis, and helmet violation analysis**.

The **multi-occupant/triple-riding detection component is still under development**, while **number-plate detection and OCR/ANPR are also under development**. Automated e-challan generation is a future extension of the system and should not be represented as a completed component at the current stage.

The pipeline has therefore been designed with a modular structure so that these additional components can be incorporated as their development progresses.

Summary

The AI processing pipeline forms the core analytical component of RideSafe AI. It processes traffic inputs using YOLO11 and SAM2 and uses the resulting information for motorcycle detection, occupant association, helmet analysis, and violation assessment.

The modular nature of the pipeline allows individual AI components to be developed, tested, and improved independently. It also provides a foundation for integrating the remaining number-plate recognition, OCR/ANPR, and future e-challan-related functionality as the project continues to develop.

4.3 Description of System Layers

The RideSafe AI system is organized into multiple logical layers, with each layer performing a specific function within the overall application. This layered architecture helps separate artificial intelligence processing, violation analysis, application services, database management, and user interaction. Such separation makes the system easier to develop, test, maintain, and extend as additional functionality is introduced.

The major layers of the RideSafe AI architecture include the **Input Layer, Application/Front-End Layer, AI Processing Layer, Violation Analysis Layer, Backend Services Layer, and Database Layer**. In addition, certain planned modules such as number-plate detection, OCR/ANPR, and future e-challan integration are treated separately because they are still under development.

Input Layer

The Input Layer represents the starting point of the RideSafe AI system. It provides the traffic information that needs to be analyzed by the application. The input may consist of traffic images, video files, or suitable traffic-media data obtained from available sources.

The quality and characteristics of the input have a direct influence on the performance of the AI models. Factors such as image resolution, lighting, camera position, motion blur, and object visibility can affect the accuracy of the subsequent processing stages.

The input layer is therefore responsible for providing suitable visual information to the application so that the AI processing components can analyze the traffic scene.

Application / Front-End Layer

The Application or Front-End Layer provides the user-facing interface of RideSafe AI. The project uses **Streamlit** for developing this interface, with the main application implemented through the Python file app.py.
The application can be started using the following command:

```
streamlit run app.py
```

Streamlit allows the project to provide an interactive interface without requiring a separate HTML, CSS, and JavaScript frontend. Users can interact with the application, provide required inputs, initiate processing, and view the results generated by the AI components.

The application layer therefore acts as the primary interaction point between the user and the underlying processing system.

AI Processing Layer

The AI Processing Layer is responsible for the computer-vision operations performed on the provided traffic data. This layer contains the major AI and image-processing technologies used by the project.

YOLO11 is used for object detection and provides information about relevant objects identified in the traffic scene. This includes detecting motorcycles and persons required for subsequent analysis.

SAM2 is used for image segmentation and provides more detailed object-region information. Segmentation is particularly useful when more precise visual information is required for analyzing objects within complex traffic scenes.

OpenCV supports image and video processing operations required by the application. It can be used for handling images, extracting video frames, preprocessing visual information, and supporting other computer-vision operations. Together, these technologies provide the primary visual-processing capabilities of the RideSafe AI system.

Violation Analysis Layer

The Violation Analysis Layer is responsible for interpreting the information generated by the AI processing components. Detection results alone do not directly represent a traffic violation; they must be analyzed in relation to the motorcycle and its occupants.

This layer includes motorcycle–occupant association and helmet-related violation analysis. The system examines the relationship between detected motorcycles and persons to determine the occupants associated with individual motorcycles. This information can then be used to analyze helmet compliance and other motorcycle-related conditions.

The **multi-occupant/triple-riding detection component is currently under development**. Therefore, it is included as part of the intended violation-analysis architecture but should not be represented as a fully completed module.
The violation-analysis layer provides the application with structured information that can be used for further processing and record management.

Backend Services Layer

The Backend Services Layer provides the application-level services required to connect the AI processing components with the database and user interface. The project uses **Python-based backend technologies**, with

FastAPI used for developing backend APIs where required.

The backend is responsible for handling application logic, processing relevant information received from the AI components, and communicating with the database. It provides a separate service layer so that the AI models do not need to communicate directly with the user interface or database.
This separation also makes it possible to modify the AI processing components without requiring major changes to the rest of the application.

Database Layer
The Database Layer provides persistent storage for information generated and managed by the RideSafe AI application. **MySQL** is used as the relational database management system for the project.

The database can store structured information related to violation records, detection results, occupant information, helmet status, timestamps, confidence information, evidence references, and other relevant application data according to the implemented database structure.

The database layer allows previously generated records to be retrieved and managed through the application. It also provides the foundation for maintaining historical information instead of relying only on temporary processing results.

Planned / Under-Development Layer
Certain components shown in the overall architecture are planned extensions of the RideSafe AI system and are currently under development. These components include **number-plate detection, OCR/ANPR, and future e-challan integration**.

Number-plate detection is intended to identify the vehicle's registration plate from relevant traffic footage. OCR/ANPR is intended to support the extraction and recognition of registration information from the detected number plate.

The automated e-challan component is intended as a future extension that could use verified violation and vehicle-identification information to support authorized digital challan workflows. These capabilities are not presented as fully completed features of the current implementation.

The **multi-occupant/triple-riding analysis** is also currently under development and is therefore treated as an evolving component within the violation-analysis architecture.

Interaction Between the Layers
The layered organization allows each component of RideSafe AI to perform a clearly defined responsibility. The input layer provides the traffic data, while the application layer allows users to interact with the system. The AI processing layer performs object detection, segmentation, and image processing, and the violation-analysis layer interprets the resulting information.

The backend services provide the application logic and communication mechanisms required to manage the processed information. MySQL provides persistent storage for relevant records, while the Streamlit application presents the available results to the user.

This separation reduces dependency between individual components and provides a structured foundation for further development.

Advantages of the Layered Architecture

The layered architecture provides several advantages to the RideSafe AI project:

1. **Modularity:** Each layer has a specific responsibility and can be developed independently.
2. **Maintainability:** Changes to one component can be made without unnecessarily modifying unrelated components.
3. **Scalability:** Additional AI and application modules can be introduced as the system develops.
4. **Testing:** Individual layers can be tested separately before complete system integration.
5. **Integration:** AI models, backend services, database operations, and the user interface can communicate through defined interfaces.
6. **Extensibility:** Future modules such as OCR/ANPR and e-challan support can be integrated into the existing architecture.
7. **Clear Responsibility:** Separating the system into logical layers makes the overall architecture easier to understand and manage.

Summary

The RideSafe AI architecture is divided into logical layers covering input handling, application interaction, AI processing, violation analysis, backend services, and database management. Each layer performs a specific role while contributing to the overall operation of the system.

The current architecture is centered around **YOLO11, SAM2, OpenCV, Python, Streamlit, FastAPI, and MySQL**. The modular design also provides space for the continued development of multi-occupant/triple-riding analysis, number-plate detection, OCR/ANPR, and future e-challan integration. This layered organization provides a structured foundation for developing RideSafe AI into a more comprehensive AI-assisted traffic-monitoring system.

4.4 Deployment Architecture

The deployment architecture of the RideSafe AI system describes the environment in which the different software, artificial intelligence, database, and user-interface components are executed. The project is developed as a Python-based AI application and is designed to run on a system capable of supporting computer-vision and deep-learning workloads. The deployment environment contains the AI models, application files, supporting libraries, database services, and Streamlit interface required for operating the developed system.

The current development environment is based on a Windows operating system and uses a dedicated Python Conda environment named helmet. The system is developed and tested on hardware equipped with an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM, which provides GPU acceleration for deep-learning operations.

Local Development Environment

The current RideSafe AI system is primarily developed and tested in a local computing environment. This approach allows the project team to train, test, and execute the AI models without requiring a continuously available cloud server.

The development system contains the required Python environment, AI libraries, model files, application source code, dataset resources, and supporting tools. The local environment also provides access to the NVIDIA GPU, which is important for computationally intensive model operations.

The major hardware configuration used for development includes:

- Processor: AMD Ryzen 9 8945HS
- RAM: 16 GB DDR5-5600
- GPU: NVIDIA GeForce RTX 4060 Laptop GPU
- GPU Memory: 8 GB VRAM
- Storage: 1 TB SSD
- Operating System: Windows

This configuration provides the computational resources required for the current development and testing of the RideSafe AI application.

Python Environment

Python is the primary programming environment used throughout the project. A dedicated Conda environment named helmet is used to maintain the project dependencies separately from other Python installations.

The project environment uses Python 3.11.15. The environment contains the major libraries required for artificial intelligence, computer vision, application development, and database integration.

The main AI-related software components include:

- PyTorch
- Ultralytics YOLO11
- SAM2
- OpenCV
- Supporting Python libraries

Maintaining a dedicated environment helps reduce dependency conflicts and provides a consistent environment for running the project.

GPU-Based AI Execution

The AI models used by RideSafe AI require significant computational resources, particularly during training and inference. The NVIDIA RTX 4060 Laptop GPU with 8 GB of VRAM is used to provide GPU acceleration for deep-learning operations.

The project environment is configured with PyTorch and CUDA support, allowing compatible AI operations to utilize the NVIDIA GPU rather than relying entirely on CPU processing.
GPU acceleration is particularly useful for YOLO11 and SAM2-related model operations because deep-learning computations can require substantial processing resources.

Application Deployment

The user-facing application is implemented using Streamlit. The main application file is app.py, and the application can be launched from the project environment using:

streamlit run app.py

When the command is executed, Streamlit starts the application and provides the interface through a web browser. This allows the developed AI functionality to be accessed through an interactive application interface without requiring a separate frontend deployment framework.

The Streamlit application acts as the primary interface for interacting with the developed system and displaying relevant AI-processing results.

Backend Deployment

The backend components are implemented using Python-based technologies, with FastAPI used for backend API development where required. The backend can operate within the same Python development environment and communicate with the AI processing components and MySQL database.

During development, the backend services can be executed locally for testing and integration. This approach allows the team to verify communication between the AI modules, application interface, and database before considering deployment to a larger environment.

Database Deployment

MySQL provides the database services required by the RideSafe AI application. The database is responsible for maintaining structured information generated during the system's operation.

During the current development stage, the database can be hosted within the local development environment. The backend communicates with MySQL to store and retrieve relevant application records

.

The local database configuration is useful for development and testing because it allows database operations to be verified without depending on an external database server.

Model and Application Files

The deployment environment also contains the trained and configured AI model resources required by the application. YOLO11 model files and SAM2-related model resources are maintained within the project environment and loaded by the application when required.

The project also contains supporting configuration files, application source code, dataset-related resources, and other files required for development and execution.

Proper organization of these resources is important because the application depends on the correct model paths and configuration settings when performing AI processing.

Development and Testing Deployment

The current deployment approach is primarily intended for development, demonstration, and testing rather than large-scale production traffic enforcement. The local environment allows the project team to test individual AI modules and evaluate their interaction with the application.

The development deployment can be used to verify:

1. AI model loading and execution.
2. Traffic image and video processing.
3. YOLO11 object detection.
4. SAM2 segmentation.
5. Motorcycle and occupant analysis.
6. Helmet violation analysis.
7. Streamlit application functionality.
8. Backend communication.
9. MySQL database operations.

This approach allows problems to be identified and corrected before considering a larger deployment environment.

Future Deployment Possibilities

Although the current system is developed and tested locally, the architecture can be extended to other deployment environments in the future. A more advanced deployment could use a dedicated GPU server or cloud-based infrastructure to process larger amounts of traffic footage.

Future deployment could also support centralized database services, multiple monitoring interfaces, multiple traffic-camera sources, and remote administrative access. Such deployment would require additional considerations related to hardware capacity, network communication, security, authentication, data storage, and system availability.The planned number-plate detection, OCR/ANPR, and automated e-challan functionality can also be incorporated into a larger deployment architecture after their respective modules are completed and tested.

Current Deployment Status

The current RideSafe AI system is primarily a local development and demonstration system. The core AI components and application environment are being developed and tested on the project's local hardware.

The system should therefore not be described as a fully deployed city-wide traffic enforcement platform. The current architecture provides the foundation for such future expansion while allowing the developed modules to be evaluated in a controlled environment.

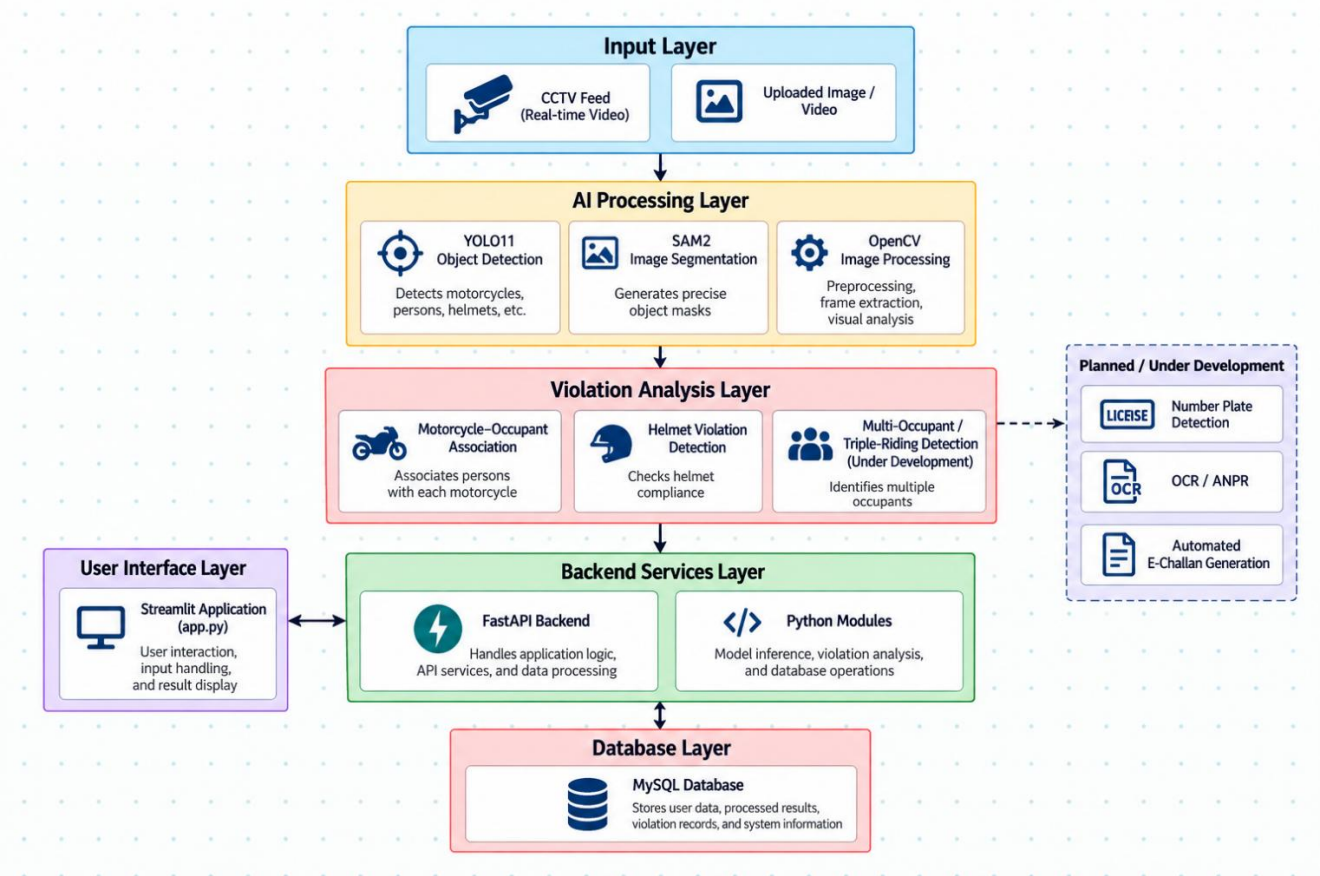
The multi-occupant/triple-riding analysis, number-plate detection, and OCR/ANPR components remain under development, while automated e-challan generation is planned as a future extension. These limitations should be considered when describing the current deployment status.

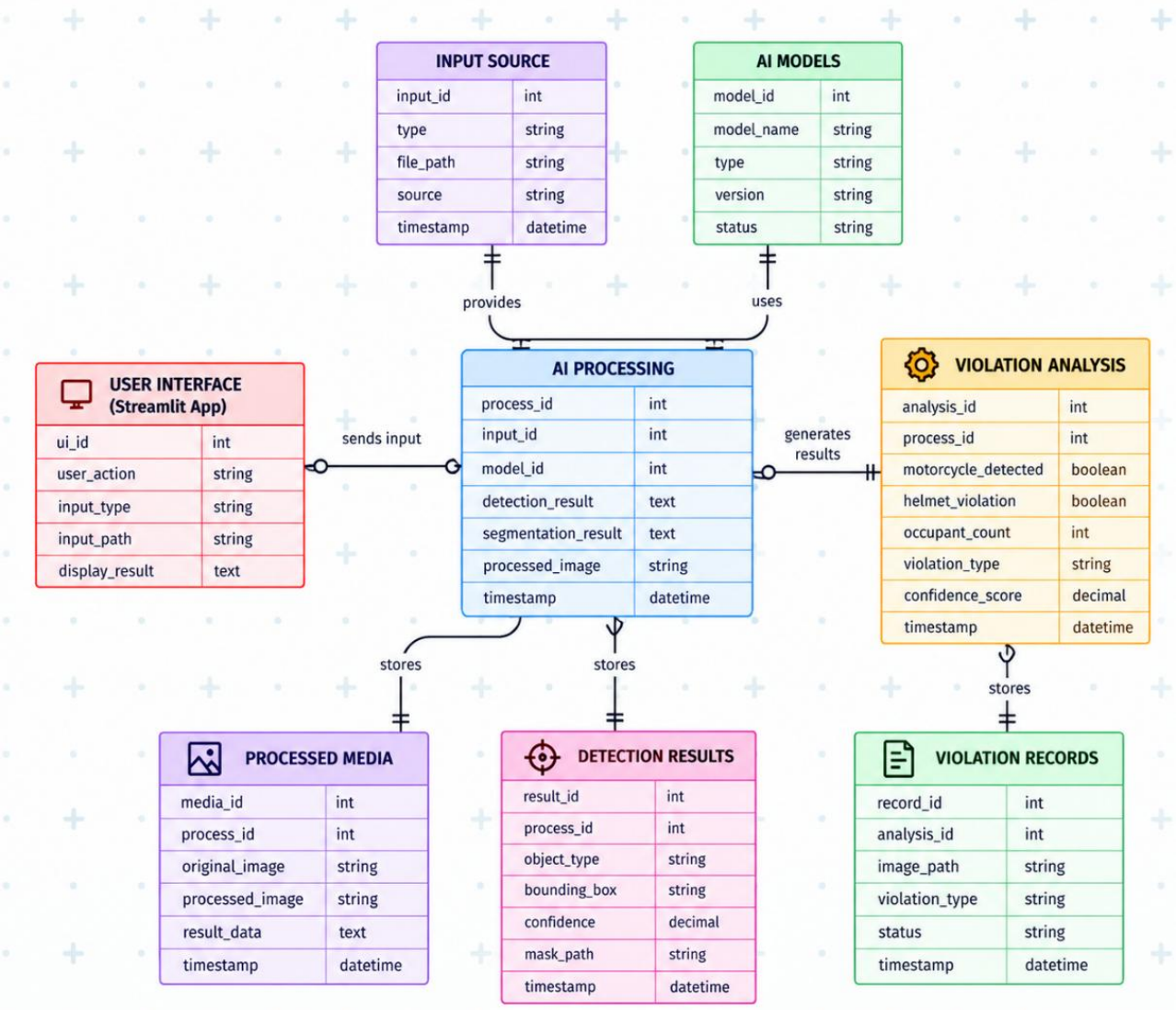
Summary

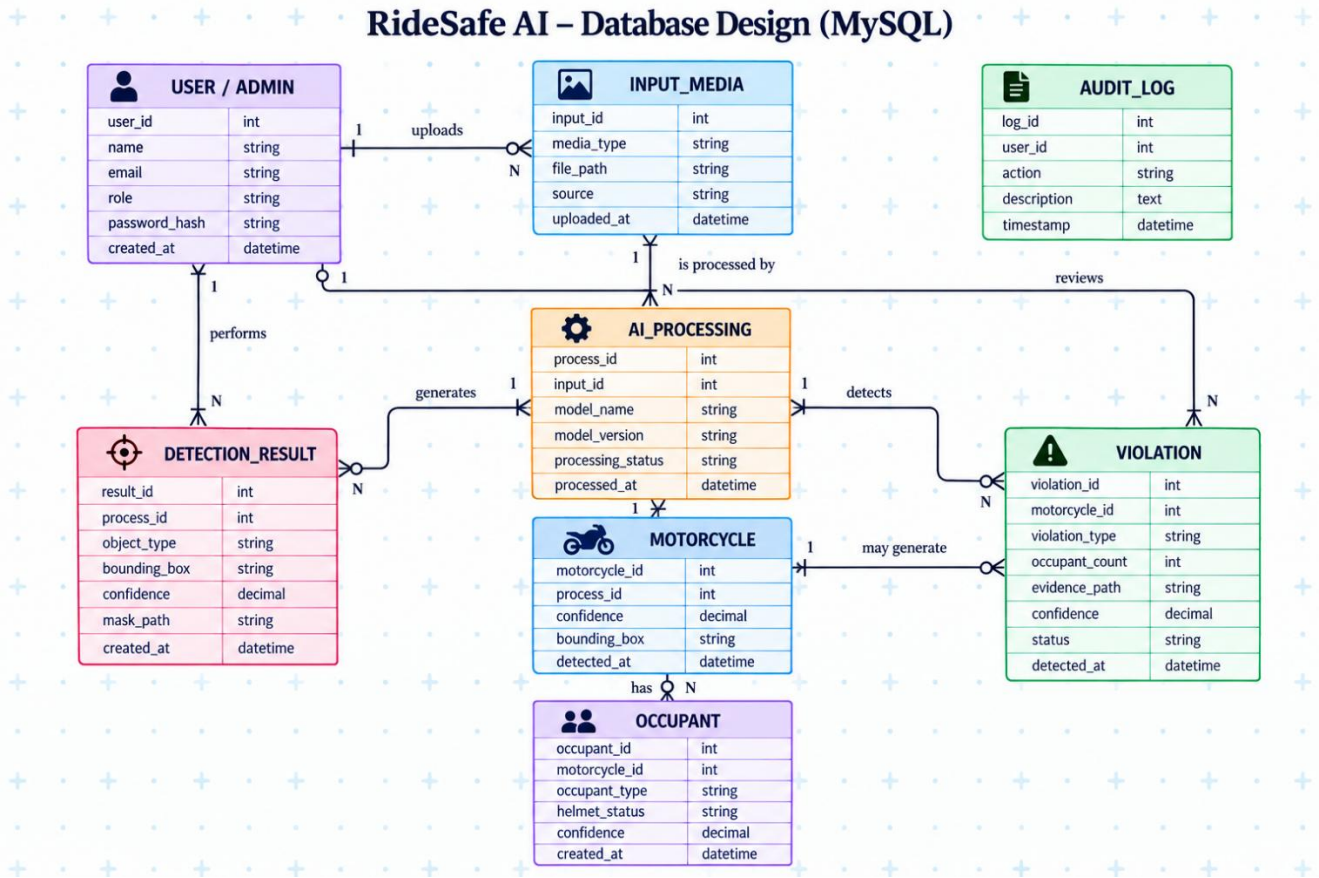
The deployment architecture of RideSafe AI is based on a local Python development environment supported by GPU acceleration. The system uses a dedicated Conda environment, PyTorch with CUDA support, YOLO11, SAM2, OpenCV, FastAPI, Streamlit, and MySQL to provide the required AI processing, application, backend, and database functionality.

The current deployment approach is suitable for development, testing, and project demonstration. Its modular structure also provides a foundation for future deployment on dedicated GPU servers or cloud infrastructure when the remaining AI and enforcement-support modules are completed and the system is prepared for larger-scale operation.

CHAPTER -5 DESIGN




INPUT SOURCE
input_id int
type string
file_path string
source string
timestamp datetime
AI MODELS
model_id int
model_name string
type string
version string
status string
provides
uses
USER INTERFACE (Streamlit App)
ui_id int
user_action string
input_type string
input_path string
display_result text
sends input
AI PROCESSING
process_id int
input_id int
model_id int
detection_result text
segmentation_result text
processed_image string
timestamp datetime
generates results
VIOLATION ANALYSIS
analysis_id int
process_id int
motorcycle_detected boolean
helmet_violation boolean
occupant_count int
violation_type string
confidence_score decimal
timestamp datetime
stores
stores
stores
PROCESSED MEDIA
media_id int
process_id int
original_image string
processed_image string
result_data text
timestamp datetime
DETECTION RESULTS
result_id int
process_id int
object_type string
bounding_box string
confidence decimal
mask_path string
timestamp datetime
VIOLATION RECORDS
record_id int
analysis_id int
image_path string
violation_type string
status string
timestamp datetime


RideSafe AI – Database Design (MySQL)
USER / ADMIN
user_id int
name string
email string
role string
password_hash string
created_at datetime
uploads
1
N
INPUT_MEDIA
input_id int
media_type string
file_path string
source string
uploaded_at datetime
AUDIT_LOG
log_id int
user_id int
action string
description text
timestamp datetime
is processed by
reviews
performs
AI_PROCESSING
process_id int
input_id int
model_name string
model_version string
processing_status string
processed_at datetime
generates
detects
DETECTION_RESULT
result_id int
process_id int
object_type string
bounding_box string
confidence decimal
mask_path string
created_at datetime
VIOLATION
violation_id int
motorcycle_id int
violation_type string
occupant_count int
evidence_path string
confidence decimal
status string
detected_at datetime
MOTORCYCLE
motorcycle_id int
process_id int
confidence decimal
bounding_box string
detected_at datetime
may generate
has
OCCUPANT
occupant_id int
motorcycle_id int
occupant_type string
helmet_status string
confidence decimal
created_at datetime

CHAPTER -6 IMPLEMENTATION

6.1 AI Model and Dataset Implementation

The implementation of RideSafe AI begins with the preparation of the datasets and the development of the artificial intelligence models required for traffic-scene analysis. Since the performance of an AI-based computer-vision system depends significantly on the quality and organization of its training data, considerable attention was given to collecting, preprocessing, organizing, and validating the datasets before model development. The implementation primarily uses YOLO11 for object detection and SAM2.1 Hiera Small for image segmentation.

The AI implementation was developed using Python, with the required machine-learning libraries and GPU-supported environment configured for model training and testing. The project uses PyTorch and CUDA-supported computation to utilize the available NVIDIA RTX 4060 Laptop GPU during deep-learning operations.

Dataset Preparation

Dataset preparation was one of the major stages of the implementation process. Multiple helmet- and traffic-related datasets were collected and organized for use in the project. The datasets were reviewed and processed to ensure that the required images and annotations were available in a consistent structure.

The project included several datasets related to helmet detection, motorcycle traffic scenes, license plates, and segmentation. The relevant datasets were organized under the project dataset directory so that they could be used independently for different stages of model development.

The preparation process involved identifying the required image data, organizing the corresponding annotations, removing duplicate images, and dividing the final dataset into suitable training, validation, and testing subsets.

For the SAM2 segmentation component, the final prepared dataset contains:

- 1,027 unique images
- 720 training images
- 205 validation images
- 102 testing images
- 1,570 annotated object instances
- 673 helmet instances
- 897 without-helmet instances

The removal of duplicate images was important because duplicate samples can unnecessarily influence the training process and may result in similar images appearing across different dataset subsets.

Dataset Organization

The final dataset was organized into separate training, validation, and testing partitions. The training set is used for model learning, the validation set is used for evaluating model behavior during development, and the testing set is reserved for evaluating the developed model on previously unseen data.

The dataset organization also ensures that the model-development process remains reproducible. Each dataset partition has its corresponding image references and annotation information required by the training configuration.

For segmentation development, the final SAM2 dataset was maintained separately from other project datasets. This allowed the segmentation model to use the appropriate image and mask information without interfering with the other model-development resources.

YOLO11 Implementation

YOLO11 is the primary object-detection model used in the RideSafe AI implementation. It is responsible for identifying relevant objects in traffic scenes and providing their locations and detection confidence.

The YOLO11 implementation is integrated into the Python-based application environment. The model receives an image or video frame and processes the visual information to identify objects relevant to the traffic-analysis pipeline.

The detection output contains information such as:

- Object class
- Bounding-box coordinates
- Detection confidence
- Location of the detected object within the frame

The resulting detection information is used by subsequent stages of the system for motorcycle and person analysis.

Motorcycle and Person Detection

The YOLO11-based detection component provides the initial object information required for motorcycle violation analysis. Motorcycle detections provide the reference regions around which occupant analysis can be performed, while person detections provide information about potential riders and passengers.

The detected objects are not treated independently during the later stages of the system. The motorcycle and person detections are analyzed together so that the system can determine which persons are associated with a particular motorcycle.

This provides the foundation for later helmet analysis and multi-occupant analysis.

Helmet-Related Detection

Helmet-related analysis is another important part of the AI implementation. The available visual information is examined to determine helmet compliance for relevant motorcycle occupants.

The model outputs and associated image regions provide the information required for the violation-analysis stage. The system is designed to consider helmet information in relation to the motorcycle and its occupants rather than simply treating every helmet-related detection in a traffic scene as a separate violation.

The performance of helmet-related analysis depends on factors such as image resolution, lighting, rider position, occlusion, camera angle, and motion blur.

SAM2 Implementation

The second major AI technology used in the project is SAM2, which provides image-segmentation capabilities. The project uses the SAM2.1 Hiera Small architecture for its segmentation development.

While YOLO11 provides object detections through bounding boxes, SAM2 provides more detailed segmentation masks for relevant image regions. This allows the system to obtain more precise information about the visible regions of selected objects.

SAM2 was integrated into the project environment and configured using the prepared segmentation dataset. The dataset and configuration were developed specifically to support helmet-related segmentation and subsequent computer-vision analysis.

SAM2 Training Configuration

The SAM2 model was trained using a project-specific configuration based on the SAM2.1 Hiera Small architecture. The training configuration was prepared according to the available computational resources and the characteristics of the prepared dataset.

The primary training settings included:

Parameter	Configuration
Model	SAM2.1 Hiera Small
Batch Size	1
Number of Frames	1
Base Learning Rate	5.0×10^{-6}
Vision Learning Rate	3.0×10^{-6}
Number of Epochs	40
Dataset	sam2_helmet_final

A batch size of one was used considering the computational requirements of the segmentation model and the available GPU memory.

Model Training and Evaluation

The AI models were developed through an iterative process of configuration, training, testing, and observation of the generated results. The training process allows the models to learn visual patterns from the prepared datasets, while validation and testing provide information about their behavior on data that is not directly used for model learning.

During model development, the project team examined the generated outputs to identify problems related to detection quality, segmentation accuracy, object visibility, and difficult traffic conditions.

The development environment uses the available NVIDIA RTX 4060 Laptop GPU with 8 GB of VRAM to accelerate compatible deep-learning operations. This significantly supports the training and testing of the AI components compared with relying only on CPU computation.

Integration of YOLO11 and SAM2

YOLO11 and SAM2 serve different but complementary purposes within the RideSafe AI system. YOLO11 provides object-detection information, while SAM2 provides detailed segmentation information for selected regions.

The combination allows the system to first identify relevant objects and subsequently perform more detailed visual analysis where segmentation is required. This is particularly useful in traffic scenes where motorcycles, riders, and helmets may overlap or where precise object boundaries are difficult to obtain using bounding boxes alone.

The integration of these technologies forms an important part of the computer-vision implementation of RideSafe AI.

Handling Difficult Traffic Conditions

During implementation, several real-world conditions were considered because they can affect AI model performance. Traffic images may contain partially visible riders, overlapping motorcycles, crowded scenes, poor lighting, motion blur, and distant objects.

The system therefore uses a modular approach in which detection, segmentation, and violation analysis are separated. This allows individual components to be improved without requiring the entire application to be redesigned.

The dataset preparation stage also plays an important role in addressing these challenges because a diverse and appropriately annotated dataset can help the models learn a wider range of visual conditions.

Current Implementation Status

The current implementation focuses on the core AI components of RideSafe AI, particularly dataset preparation, YOLO11-based object detection, and SAM2-based image segmentation. These components form the foundation for the subsequent motorcycle, occupant, and helmet violation analysis.

The multi-occupant/triple-riding detection component is still under development. Similarly, number-plate detection and OCR/ANPR are under development and therefore should not be presented as fully completed AI modules in this section.

The automated e-challan functionality is planned as a future extension and depends on the completion and integration of the required vehicle-identification and violation-record components.

Summary

The AI model and dataset implementation provides the foundation for the RideSafe AI system. Multiple datasets were collected, organized, cleaned, and prepared for model development, resulting in a structured SAM2 dataset containing 1,027 unique images and 1,570 annotated object instances.

YOLO11 is used for object detection, while SAM2.1 Hiera Small provides image-segmentation capabilities. The two technologies complement each other by providing object-level detection and more detailed segmentation information. The resulting AI components form the foundation for the subsequent violation-detection and application layers of the RideSafe AI system.

6.2 Violation Detection Implementation

The violation detection implementation is responsible for analyzing the results produced by the AI models and identifying motorcycle-related conditions that may represent traffic violations. The implementation combines object detection, motorcycle–occupant association, helmet analysis, occupant counting, and planned vehicle-identification capabilities into a structured violation-analysis process. This stage is important because detecting individual objects alone is not sufficient to determine whether a traffic violation has occurred.

The violation-detection process uses the information generated by YOLO11, SAM2, OpenCV, and the application logic. The system analyzes detected motorcycles and persons together and uses their spatial relationship and visual information to determine relevant occupants and their helmet status. Additional analysis is intended to identify multi-occupant or triple-riding conditions.

Motorcycle Detection and Association

Motorcycle detection forms the starting point of the violation-analysis stage. YOLO11 identifies motorcycles present in the input image or video frame and provides their corresponding bounding-box information and confidence values.

After motorcycles have been detected, the system analyzes the detected person regions in relation to each motorcycle. This is necessary because a traffic scene may contain multiple motorcycles and multiple persons, and the system needs to determine which persons belong to which motorcycle.

The motorcycle–occupant association process considers the spatial relationship between detected motorcycles and persons. The associated person information is then used by the subsequent helmet and occupant-analysis stages.

Occupant Analysis

Occupant analysis is performed to determine the persons associated with an individual motorcycle. This is particularly important for the RideSafe AI project because helmet compliance and multi-occupant violations depend on identifying the occupants of each motorcycle.

The system considers the detected person regions and their relationship with the corresponding motorcycle. The resulting association allows the application to distinguish relevant motorcycle occupants from unrelated persons who may appear elsewhere in the traffic scene.

Occupant association can become challenging when motorcycles are close to one another or when riders are partially hidden. Therefore, the accuracy of this stage depends on the quality of the object detections and the visual conditions of the input.

Helmet Violation Detection

Helmet violation detection is one of the primary violation-analysis functions of RideSafe AI. After identifying the occupants associated with a motorcycle, the system analyzes the available visual information to determine whether the relevant occupant appears to be wearing a helmet.

The helmet analysis uses the results obtained from the AI detection and segmentation components. YOLO11 provides object-level detection information, while SAM2 can provide more detailed segmentation information for relevant image regions.

The system is designed to evaluate helmet status in relation to the motorcycle occupant rather than considering every helmet-related object in the complete traffic scene. This helps reduce the possibility of treating unrelated objects or persons as part of the same violation event.

Helmet Analysis Challenges

Helmet detection can be affected by several real-world conditions. Riders may be partially occluded by other vehicles, the helmet may be difficult to distinguish because of the camera distance, or the image may contain motion blur and poor lighting.

Important conditions that can affect helmet analysis include:

1. Partial Occlusion: Parts of the rider or helmet may be hidden.
2. Low Resolution: Distant motorcycles may contain insufficient visual information.
3. Motion Blur: Moving motorcycles can produce blurred rider regions.
4. Lighting Conditions: Shadows and low-light scenes can reduce visual clarity.
5. Camera Position: Different camera angles can change the appearance of helmets and riders.
6. Crowded Traffic: Multiple motorcycles and persons may overlap.

These conditions are considered during model development and testing because reliable violation analysis depends on the quality of the available visual information.

Multi-Occupant and Triple-Riding Detection

Occupant counting is used to identify the number of persons associated with an individual motorcycle. This information can be used to identify cases in which a motorcycle appears to have more occupants than permitted.

The system uses motorcycle–occupant association as the basis for this analysis. Once the relevant persons associated with a motorcycle have been identified, the number of associated occupants can be considered during violation analysis.

Cases involving multiple occupants, including potential triple-riding situations, can therefore be classified as potential violations based on the detected occupant count.

However, the multi-occupant/triple-riding detection component is currently under development. It should therefore be considered an evolving part of the system rather than a fully completed automated enforcement feature. Further development and testing are required to improve occupant association and counting under crowded and partially occluded traffic conditions.

Violation Classification

The violation-analysis stage combines the information obtained from different processing components rather than depending on a single detection result.

For a particular motorcycle, the system can consider:

- Motorcycle detection
- Associated occupant information
- Helmet status
- Occupant count
- Detection confidence
- Relevant visual evidence

Based on these results, the application can identify potential violation conditions and prepare the information for further processing.

This approach provides a structured method for converting raw AI predictions into application-level violation information.

Number-Plate Detection

Number-plate detection is intended to provide vehicle-identification information associated with a detected violation. The purpose of this component is to identify the registration-plate region of the relevant motorcycle so that the registration information can subsequently be extracted.

The number-plate module is shown as part of the overall RideSafe AI architecture; however, number-plate detection is currently under development. Therefore, it should not be represented as a fully completed component of the current implementation.

The development of this module will require additional testing under different traffic conditions, particularly for distant motorcycles, angled plates, low-resolution footage, and partially obstructed number plates.

OCR / ANPR

OCR (Optical Character Recognition) and ANPR (Automatic Number Plate Recognition) are intended to support the extraction and recognition of registration information from detected number plates.

The planned process involves identifying the relevant number-plate region and applying character-recognition techniques to obtain the registration information. The recognized information could subsequently be associated with the corresponding violation record.

At the current stage, OCR/ANPR remains under development. Consequently, the documentation does not treat automatic number-plate recognition as a fully implemented feature. Its integration will depend on the successful development and testing of the number-plate detection and recognition components.

Evidence and Violation Information

The violation-analysis process can produce information that is useful for maintaining a structured violation record. Relevant information may include the detected violation type, date and time, occupant count, helmet status, confidence information, and a reference to the processed image or other available evidence.

This information can later be passed to the backend and stored in the MySQL database. Maintaining structured records allows the application to retrieve and display the information through the Streamlit interface.

The database and dashboard integration are described separately under Section 6.3 Application, Backend and Database Implementation.

Current Implementation Status

The current violation-detection implementation focuses primarily on the developed motorcycle detection, occupant-related analysis, and helmet violation analysis components. These form the core of the traffic-violation processing stage.

The multi-occupant/triple-riding detection, number-plate detection, and OCR/ANPR components are still under development. The automated e-challan generation capability is a future extension and is dependent on the completion and integration of the required violation and vehicle-identification modules.

Therefore, the current system should be understood as an AI-assisted motorcycle violation-analysis framework rather than a fully automated traffic-enforcement system.

Summary

The violation detection implementation combines the outputs of the AI models with application-level analysis to identify potential motorcycle violations. Motorcycle detection and occupant association provide the foundation for helmet analysis and occupant counting, while the resulting information can be organized into structured violation records.

The architecture also provides provisions for multi-occupant detection, number-plate recognition, and OCR/ANPR. These components are currently under development and will be further integrated and evaluated as the project progresses. The resulting modular design provides a foundation for future expansion toward advanced traffic-monitoring and authorized e-challan-support functionality.

6.3 Application, Backend and Database Implementation

The application, backend, and database implementation provides the software layer that connects the AI processing components with the user interface and persistent data storage of the RideSafe AI system. While YOLO11 and SAM2 are responsible for the core computer-vision operations, the application layer manages user interaction, processing requests, result presentation, and communication with the database.

The implementation uses **Python** as the primary programming language. **Streamlit** is used to develop the application interface, **FastAPI** is used for backend services and API-based communication, and **MySQL** is used for structured data storage. This combination allows the AI components and application components to operate within a common Python-based development environment.

Streamlit Application Implementation

The user-facing application is developed using **Streamlit**. The main application file is app.py, which acts as the entry point for running the interface.

The application is started using the following command:

```
streamlit run app.py
```

When the command is executed, Streamlit starts the application and makes the interface available through a web browser. This provides a convenient way for users to interact with the developed AI system without requiring a separate traditional frontend framework.

The Streamlit application can provide controls for supplying input, initiating processing, and displaying the results generated by the AI modules.

Application Input Handling

The application provides the interface through which the required traffic data can be supplied for processing. Depending on the implemented workflow, the input may include traffic images or video data.

After the input is provided, the application can pass the information to the appropriate Python processing functions. These functions load the required AI models and perform the corresponding detection, segmentation, and analysis operations.

The application therefore acts as the connection point between the user and the underlying AI-processing modules.

AI Model Integration

The Streamlit application is integrated with the Python-based AI components used in the project. YOLO11 provides object-detection functionality, while SAM2 provides segmentation functionality.

The application can invoke the relevant processing functions when an input is submitted. The generated results can then be presented through the Streamlit interface.

This Python-based integration is useful because the AI models and application interface can operate within the same development environment. It also simplifies testing because changes to the AI-processing code can be evaluated directly through the application.

Backend Implementation using FastAPI

FastAPI is used as the backend technology for developing application programming interfaces and backend services. The backend provides an additional application layer for handling system logic and communication between different components.

The backend can receive application requests, process relevant information, communicate with database services, and return structured results to the application layer.

Using a separate backend layer also helps maintain a modular architecture. The AI models do not need to directly handle database operations or user-interface responsibilities.

Application Logic

The application logic manages the processing of information between the different components of the system. It determines how input is handled, which processing operations are required, and how the resulting information is presented or stored.

The application logic can include operations such as:

1. Receiving user input through the Streamlit interface.
2. Loading the required AI models.
3. Processing images or video frames.
4. Obtaining detection and segmentation results.
5. Performing violation-related analysis.
6. Preparing structured information for storage.
7. Retrieving stored information when required.
8. Presenting relevant results through the application interface.

Separating these responsibilities into appropriate Python modules makes the application easier to maintain and modify.

MySQL Database Integration

MySQL is used as the relational database for storing structured application information. The backend and Python application components can communicate with the database to store and retrieve relevant records.

Information that may be maintained in the database includes:

- Violation records
- Violation type
- Date and time
- Occupant-related information
- Helmet status
- Detection confidence
- Evidence references
- Processing status
- Other application-related information

The database provides persistent storage so that relevant information can be accessed after the completion of an individual processing operation.

Storing Violation Records

After the AI-processing and violation-analysis stages generate relevant information, the application can organize the results into structured records. These records can then be stored in MySQL through the backend or appropriate database operations.

A violation record can contain information required for later review and monitoring. Maintaining structured records also allows the dashboard to retrieve information based on different requirements.

The database structure can be extended as additional project modules are developed.

Retrieving Database Information

The application can also retrieve stored records from MySQL. Database queries can be used to obtain the information required by the monitoring interface.

For example, records can be retrieved for viewing previously processed violations, reviewing detection information, or examining records according to specific conditions such as violation type or date.

This provides a persistent information-management mechanism for the RideSafe AI application.

Admin Dashboard Implementation

The Streamlit application also provides the foundation for the administrative monitoring interface. The dashboard is intended to present information generated by the AI system in an organized and accessible form.

Depending on the implemented functionality, the dashboard can display:

1. Processed traffic images or results.
2. Detected motorcycle information.
3. Helmet-related analysis.
4. Occupant-related information.
5. Potential violation records.
6. Stored database information.
7. Application or processing status.

The dashboard provides a convenient interface for demonstrating and monitoring the developed system.

Backend and Database Communication

The backend provides the communication layer between the application and MySQL database. Instead of requiring the user interface to directly perform database operations, requests can be handled through the backend services.

This approach improves the separation of responsibilities within the application. Streamlit focuses on user interaction and result presentation, FastAPI manages backend services and application communication, and MySQL provides persistent data storage.

The modular structure also allows individual components to be modified without unnecessarily affecting the other layers.

Current Implementation Status

The current application implementation focuses on the integration of the **Streamlit interface, Python-based AI processing, backend services, and MySQL database**.

The Streamlit application is used as the primary user-facing interface and is launched through streamlit run app.py.

FastAPI provides the backend framework for application services, while MySQL provides structured data storage.

The **multi-occupant/triple-riding analysis, number-plate detection, and OCR/ANPR components remain under development**. Automated e-challan generation is also a planned future capability. Therefore, the current application should not be described as providing a completely automated e-challan workflow.

Advantages of the Implementation

The selected application architecture provides several advantages:

1. **Python-Based Integration:** The application, AI models, and processing modules can operate within the same Python environment.
2. **Simple User Interface:** Streamlit allows the project to provide an interactive interface without developing a separate frontend application.
3. **Modular Backend:** FastAPI provides a separate layer for application services and API communication.
4. **Structured Storage:** MySQL provides persistent relational storage for application records.
5. **Easy Testing:** Individual AI and application components can be tested through the interface.
6. **Extensibility:** Additional modules can be integrated as development progresses.
7. **Maintainability:** Separating the interface, processing, backend, and database responsibilities makes the system easier to manage.

Summary

The application, backend, and database implementation connects the AI processing components of RideSafe AI with the user-facing application and persistent storage layer. Streamlit provides the interactive interface, Python manages the application and AI-processing logic, FastAPI provides backend services, and MySQL maintains structured application records.

This implementation provides the software foundation required for operating and demonstrating the current RideSafe AI system. Its modular structure also allows additional functionality, including improved multi-occupant analysis, number-plate recognition, OCR/ANPR, and future authorized e-challan support, to be integrated as those components are further developed.

CHAPTER 7- FEATURES

7.1 List of Main Features

RideSafe AI provides an AI-assisted platform for analyzing motorcycle traffic footage and identifying relevant violation conditions. The application brings together the developed detection, analysis, and monitoring capabilities into a single interface.

The major features of the system include:

1. **Traffic Media Input** – Allows traffic images or video data to be provided for analysis.
2. **Motorcycle Detection** – Identifies motorcycles present in the provided traffic scene.
3. **Occupant Analysis** – Analyzes persons associated with detected motorcycles.
4. **Helmet Violation Detection** – Identifies potential helmet non-compliance among relevant motorcycle occupants.
5. **Multi-Occupant Analysis** – Provides analysis for motorcycles carrying multiple occupants. The triple-riding component is currently under development.
6. **Violation Record Management** – Maintains relevant detected-violation information through the application's database.
7. **Interactive Monitoring Interface** – Provides a Streamlit-based interface for running the application and viewing available results.
8. **Administrative Monitoring** – Allows stored detection and violation information to be viewed and reviewed through the application.
9. **Future Vehicle Identification** – Number-plate detection and OCR/ANPR are under development and are intended to extend the system with vehicle-identification capabilities.
10. **Future E-Challan Support** – Provides a foundation for integrating authorized e-challan functionality after the remaining identification and validation modules are completed.

The features are designed to work together as a unified traffic-monitoring application while keeping the current implementation and future extensions clearly separated.

7.2 AI-Based Motorcycle Violation Detection feature

The AI-based violation detection feature is the core functional capability of RideSafe AI. It is designed to analyze motorcycle traffic scenes and identify potential violation conditions from the available visual information. The feature combines motorcycle detection, occupant analysis, and helmet compliance analysis to provide structured information about detected motorcycle-related violations.

The major violation-analysis capabilities include:

1. **Motorcycle Detection** – Identifies motorcycles present in the input traffic media.
2. **Occupant Association** – Associates detected persons with the corresponding motorcycle to determine relevant occupants.
3. **Helmet Compliance Analysis** – Examines the helmet status of relevant motorcycle occupants and identifies potential helmet violations.
4. **Multi-Occupant Analysis** – Determines the number of occupants associated with a motorcycle to support identification of multiple-occupant cases.
5. **Triple-Riding Detection** – Intended to identify motorcycles carrying more than the permitted number of occupants. This functionality is currently under development.
6. **Violation Classification** – Organizes the available detection results into potential violation categories for further processing and record management.
7. **Visual Evidence** – The processed traffic media and detection results can provide supporting visual information for reviewing identified cases.

The system is designed to consider the relationship between motorcycles and their occupants rather than treating individual object detections as independent violations. This helps the application analyze a complete motorcycle-related event.

Number-plate detection and OCR/ANPR are also under development and are intended to extend the violation-detection capability by associating identified violations with vehicle registration information. Automated e-challan generation is planned as a future extension and is not considered a completed feature of the current system.

Overall, this feature provides the main AI-assisted functionality of RideSafe AI by converting visual traffic information into potential motorcycle violation information for further review and management.

7.3 Violation Records and Admin Monitoring feature

RideSafe AI provides a dedicated mechanism for storing and reviewing the results generated during traffic analysis. The detected information is maintained in the MySQL database so that relevant violation records can be accessed for monitoring and administrative purposes.

The major functions include:

1. **Violation Record Storage** – Stores detected violation information along with relevant details generated by the system.
2. **Result Monitoring** – Allows administrators to view the available detection and violation results through the application interface.
3. **Record Review** – Provides a way to examine stored records and verify the information identified during analysis.
4. **Database Management** – Maintains structured records in MySQL for organized storage and retrieval.
5. **Administrative Access** – Provides an interface for authorized users to monitor the available system records.

The monitoring functionality helps organize the results produced by the AI modules and provides a centralized view of the available violation information. Number-plate identification and automated e-challan processing are planned for future integration as the corresponding modules are completed.

CHAPTER -8 TESTING

8.1 AI Model and Module Testing

Testing of RideSafe AI was carried out to verify whether the developed AI modules produce meaningful and consistent results when provided with traffic images and video inputs. The testing process focused on the individual detection capabilities and their behavior under different traffic conditions.

The motorcycle detection module was tested using different traffic scenes containing motorcycles at varying positions, sizes, and viewing angles. The purpose was to verify that motorcycles could be detected correctly without depending on a fixed camera position. The results were reviewed using the generated detection outputs and visualized bounding regions.

The occupant analysis module was tested by providing scenes containing motorcycles with different numbers of associated persons. This testing was used to verify whether persons appearing near motorcycles could be considered during the violation-analysis process. Particular attention was given to scenes containing single riders and multiple occupants, since correct association is important for further violation analysis.

Helmet compliance analysis was tested using traffic samples containing both helmeted and non-helmeted riders. The results were checked against the visible condition in the input media. Cases where the rider's head was partially hidden, distant, or affected by image quality were also considered during testing.

The segmentation component was evaluated by checking whether the generated masks corresponded to the relevant motorcycle-rider regions. The testing helped verify that the segmentation output could provide useful visual information for the developed analysis workflow.

The multi-occupant and triple-riding functionality was also considered during module testing. Since this component is still under development, its results were treated as developmental outputs rather than a fully completed violation decision. Similarly, number-plate detection and OCR/ANPR were not treated as completed modules during the final testing stage.

Testing was performed using both normal cases and challenging traffic situations to identify limitations such as overlapping objects, small objects, poor visibility, different camera angles, and crowded scenes. These tests helped identify areas requiring further improvement before the system can be used in a fully automated traffic-enforcement environment.

8.2 Integration Database Testing

After testing the individual modules, integration testing was performed to verify communication between the application components. The objective was to ensure that the output generated during traffic analysis could be passed correctly to the application and subsequently handled by the database layer.

The input and processing workflow was tested using suitable traffic media. The application was checked for successful input handling, processing, result display, and storage of relevant information. The Streamlit interface was also tested to ensure that users could interact with the available functionality without disrupting the analysis process.

Database testing focused on the storage and retrieval of violation-related information in MySQL. Records generated by the application were checked to ensure that the information was stored in the appropriate database structure. Retrieval operations were tested to confirm that previously stored information could be accessed for administrative monitoring.

Integration testing also considered cases where no violation was detected. The system was checked to ensure that such cases did not result in incorrect violation records. Similarly, multiple detections from the same input were reviewed to identify potential duplication or inconsistent record creation.

The testing process helped verify the interaction between the AI processing modules, application interface, backend services, and database. It also helped identify areas where additional validation will be required when the number-plate, OCR/ANPR, and automated e-challan modules are completed.

8.3 Tools Used for Testing

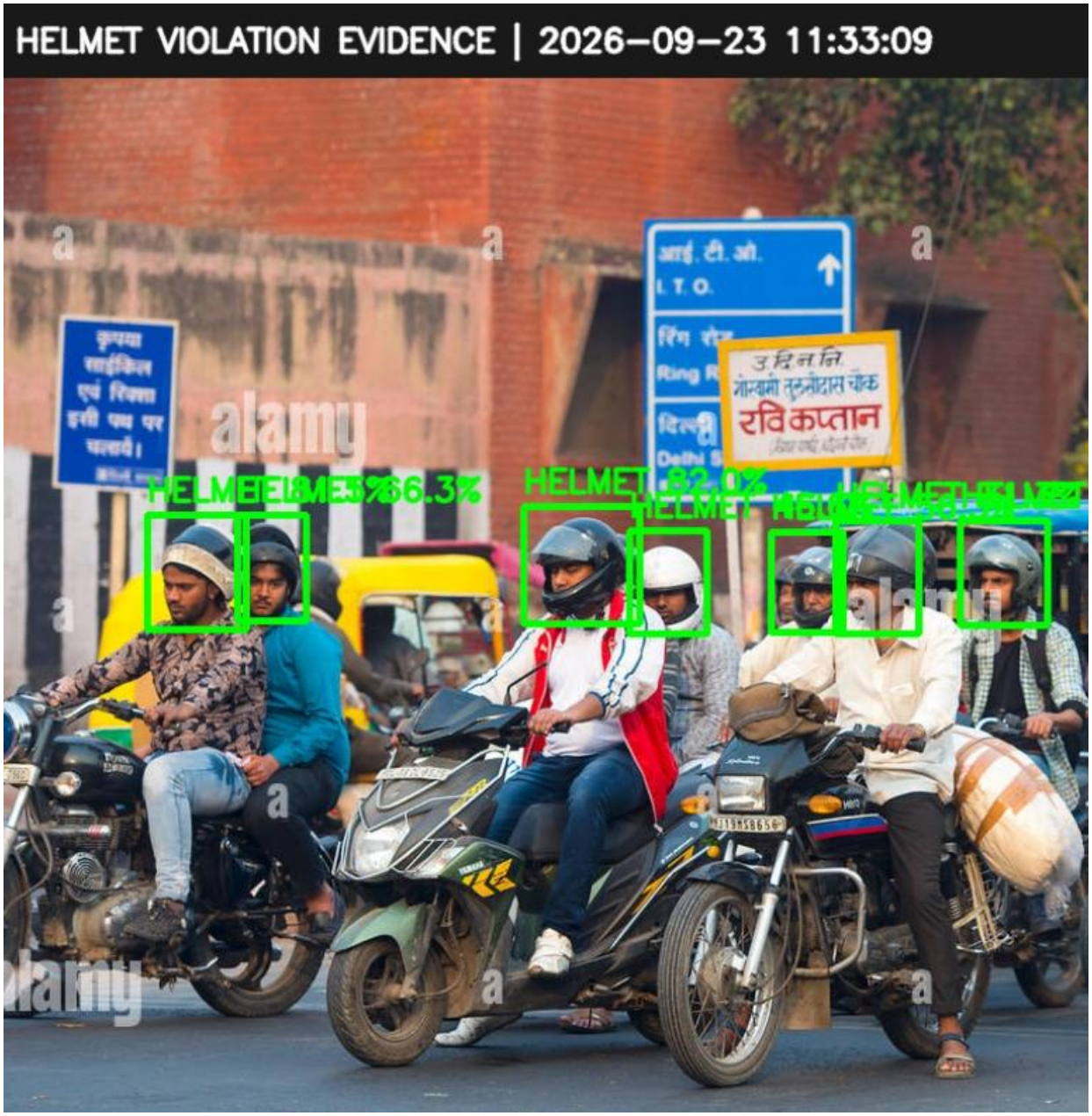
The final testing stage focused on evaluating the overall behavior of RideSafe AI under different input conditions. The system was tested using traffic scenes containing motorcycles, helmeted riders, non-helmeted riders, and multiple occupants. The generated outputs were compared with the visible conditions in the corresponding input media.

The results showed that the currently developed modules can support the intended motorcycle violation-analysis workflow. The application was able to process suitable traffic inputs and present the available results through the monitoring interface. The database functionality was also checked to ensure that relevant information could be stored and retrieved when required.

Performance evaluation considered both functional correctness and practical application behavior. Functional performance was evaluated by checking whether the system generated the expected type of result for each tested condition. Practical performance was considered in terms of processing behavior, application responsiveness, database operations, and consistency across different traffic inputs.

Testing also revealed the effect of challenging conditions such as poor image quality, partially hidden riders, overlapping objects, distant motorcycles, different camera angles, and crowded traffic scenes. These conditions can affect detection accuracy and therefore represent important areas for further improvement.

The testing stage provides a foundation for continued development of RideSafe AI. The completed modules can be further refined using additional training data and challenging traffic samples, while the modules currently under development can be integrated and evaluated after their implementation.




HELMET VIOLATION EVIDENCE | 2026-09-23 11:35:
NO HELMET 82.2%

CHAPTER -9 DEPLOYMENT

9.1 Deployment Requirements and Configuration

The deployment of RideSafe AI requires a properly configured computing environment in which the artificial intelligence models, application components, backend services, and database can operate together. Since the system performs computer vision-based processing, the deployment environment must provide sufficient computational resources for handling image and video data efficiently. The configuration used during development provides the required environment for executing the current version of the application.

The hardware environment consists of a computer equipped with an **AMD Ryzen 9 8945HS processor, 16 GB DDR5 RAM, 1 TB SSD storage, and an NVIDIA GeForce RTX 4060 Laptop GPU with 8 GB VRAM**. The NVIDIA GPU provides hardware acceleration for AI processing and is particularly useful when executing computationally intensive computer vision operations. Adequate storage is also required because the project contains trained model files, datasets, input media, generated results, and application resources.

The software environment is based on **Windows** and **Python 3.11.15**. A dedicated Conda environment named helmet is used to maintain the project dependencies separately from other Python projects. This helps avoid conflicts between different package versions and provides a consistent environment for running the developed modules.

The primary machine-learning framework used in the deployment environment is **PyTorch**, configured with CUDA support for GPU acceleration. The project also contains the required packages for YOLO-based detection, SAM2-based processing, image and video handling, database connectivity, and application development. All required dependencies must be installed before starting the application.

The trained model files and configuration files are placed in their respective project directories. The application also requires access to the appropriate dataset and model resources wherever they are required by the implemented modules. Maintaining the expected project structure is important because the application references these resources during execution.

Before deployment, the environment is checked to confirm that Python is correctly installed, the required Conda environment is available, and the AI processing framework can access the NVIDIA GPU. The availability of CUDA acceleration is important for achieving practical processing performance during model execution.

The application configuration also includes the database connection settings required for communication with MySQL. These settings allow the application to store and retrieve the information generated during traffic analysis. Configuration values should be maintained according to the deployment environment rather than being hard-coded into different parts of the application

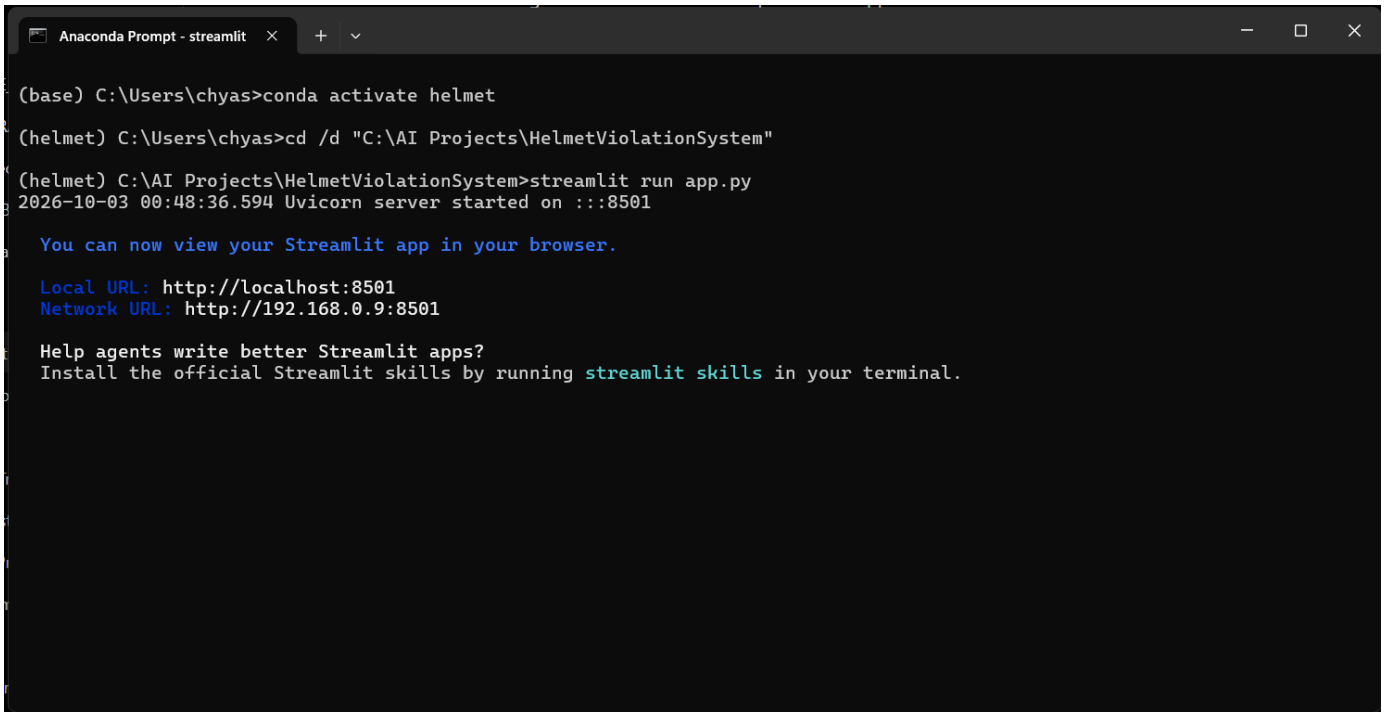
.

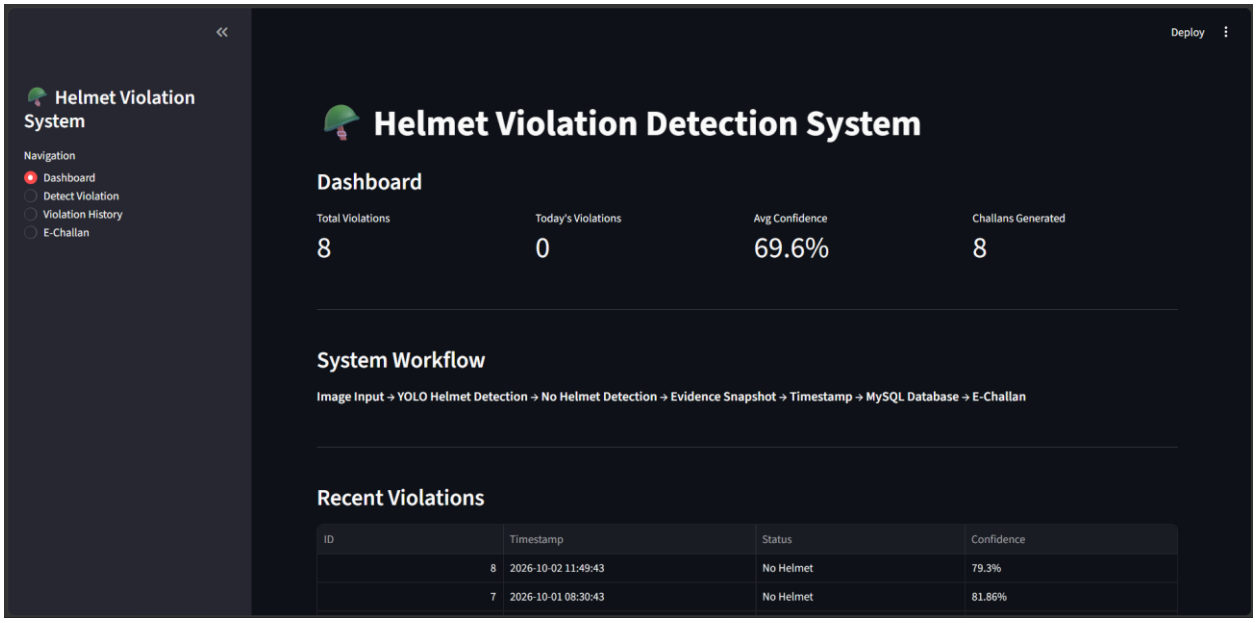
After completing the environment configuration, the application can be started through the Streamlit framework using the following command:

streamlit run app.py

Once the command is executed successfully, Streamlit starts the application and provides a local web address through which the RideSafe AI dashboard can be accessed. The deployment environment can then be tested by providing suitable traffic media and verifying that the application loads correctly and performs the available processing operations.

The deployment configuration is designed to support the current stage of the RideSafe AI project while allowing future modules to be added without changing the complete development environment. The number-plate detection, OCR/ANPR, triple-riding, and automated e-challan components that are still under development can later be integrated into the same environment after their implementation and the testing is completed.





9.2 Backend and Database Deployment

The backend and database deployment of RideSafe AI provides the supporting environment required for processing application requests and maintaining the information generated during traffic analysis. The deployment connects the application interface with the backend services and MySQL database so that detected information can be processed, stored, and retrieved when required.

The backend component is developed using **Python-based services and FastAPI**. During deployment, the required Python environment and project dependencies must be available before starting the backend service. The backend is responsible for providing the supporting application functionality and handling communication between the different components of the system.

Before starting the backend, the project configuration is checked to ensure that the required modules, model resources, and database settings are available. The backend environment must use the same project dependencies that were used during development to avoid compatibility problems during execution.

The database component is deployed using **MySQL**. A dedicated database is configured for storing the information generated by the RideSafe AI application. The database structure contains the required tables for maintaining application and violation-related information. Proper database configuration is necessary so that records generated during processing can be stored and accessed by the application.

Database connectivity is verified during deployment by establishing a connection between the application and MySQL. Basic database operations such as inserting records, retrieving stored information, and checking the availability of required tables are performed to confirm that the connection is functioning correctly.

The deployment process also requires appropriate configuration of database credentials and connection parameters. These values allow the backend to communicate with the MySQL server securely and consistently. The configuration should be updated according to the system on which the application is deployed.

Once the backend and database services are configured, the application can communicate with the database during normal operation. When a relevant detection or violation result is generated, the available information can be passed to the backend and stored in the corresponding database structure. Administrators can subsequently access the stored information through the application's monitoring interface.

The backend and database deployment was also considered from the perspective of future expansion. The current system provides the foundation for storing and managing the available violation information, while additional information can be incorporated when the remaining modules are completed. In particular, **number-plate detection, OCR/ANPR, multi-occupant/triple-riding detection, and automated e-challan processing** can later be connected to the existing backend and database structure.

Proper deployment of these components helps maintain a clear separation between application processing and data management. It also makes the system easier to maintain and extend as additional features are implemented and tested.

For demonstration and development purposes, the backend and MySQL database can be operated on the same local machine as the Streamlit application. In a future production deployment, the backend and database can be hosted on dedicated or cloud infrastructure depending on the required scale, security, and accessibility.

id	violation_id	challan_number	generated_at	fine_amount	status
1	1	ECH-20260915-0001	2026-09-15 09:44:31	500	Generated
2	2	ECH-20260923-0002	2026-09-23 10:23:45	500	Generated
3	6	ECH-20260923-0006	2026-09-23 11:36:27	500	Generated
4	5	ECH-20260923-0005	2026-09-23 21:02:05	500	Generated
5	4	ECH-20260923-0004	2026-09-23 21:02:15	500	Generated
6	3	ECH-20260923-0003	2026-09-23 21:02:23	500	Generated
7	7	ECH-20261001-0007	2026-10-01 08:31:24	500	Generated
8	8	ECH-20261002-0008	2026-10-02 11:51:01	500	Generated
NULL	NULL	NULL	NULL	NULL	NULL

violation_id	timestamp	helmet_status	confidence	evidence_image	challan_number	generated_at	fine_amount	status
1	2026-09-15 09:44:00	No Helmet	0.83284	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260915-0001	2026-09-15 09:44:31	500	Generated
2	2026-09-23 10:22:56	No Helmet	0.398501	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0002	2026-09-23 10:23:45	500	Generated
3	2026-09-23 11:26:58	No Helmet	0.804312	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0003	2026-09-23 21:02:23	500	Generated
4	2026-09-23 11:27:49	No Helmet	0.826479	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0004	2026-09-23 21:02:15	500	Generated
5	2026-09-23 11:33:09	No Helmet	0.269471	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0005	2026-09-23 21:02:05	500	Generated
6	2026-09-23 11:35:58	No Helmet	0.821686	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0006	2026-09-23 11:36:27	500	Generated
7	2026-10-01 08:30:43	No Helmet	0.818649	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20261001-0007	2026-10-01 08:31:24	500	Generated
8	2026-10-02 11:49:43	No Helmet	0.792978	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20261002-0008	2026-10-02 11:51:01	500	Generated

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The development of RideSafe AI involved several technical and practical challenges due to the complexity of analyzing real-world motorcycle traffic scenes. The system combines computer vision, object detection, segmentation, occupant analysis, application development, and database management. Each component required separate testing and refinement before being integrated into the overall application.

One of the major challenges was **collecting and preparing suitable datasets** for motorcycle and helmet-related detection. Traffic datasets obtained from different sources contained variations in image quality, camera angles, object sizes, annotation formats, and class names. Some datasets also contained duplicate or unsuitable images. Preparing the data for training therefore required conversion, organization, verification, and removal of unnecessary or duplicate samples.

Another challenge was **detecting motorcycles and associated occupants in crowded traffic scenes**. In real-world traffic footage, motorcycles can appear close to one another, while riders may overlap with other vehicles or persons. This makes it difficult to establish the correct relationship between a motorcycle and its occupants. Small or partially visible objects also created additional difficulties during detection.

Helmet detection presented another challenge because helmets can appear very small in distant traffic footage. Different helmet designs, colors, lighting conditions, rider positions, and image resolutions can affect the visibility of the helmet region. In some cases, the rider's head may be partially blocked by another object, making reliable classification more difficult.

The development of the **SAM2-based segmentation component** also required considerable effort. Preparing segmentation data involved handling different annotation formats and ensuring that the required masks corresponded correctly to the intended objects. Training and evaluating the segmentation model also required suitable configuration and computational resources.

Another issue encountered during development was **GPU and memory management**. AI model training and inference can require significant computational resources, particularly when processing high-resolution images or video. The available hardware was therefore considered carefully while selecting training parameters and processing configurations.

The project also faced challenges related to **video processing**. Unlike individual images, traffic videos contain continuous frames and require repeated processing. Processing large numbers of frames can increase computational requirements and processing time. Selecting appropriate frames and maintaining practical processing performance therefore became an important consideration.

The **integration of different project components** was another development challenge. The AI models, Python application, Streamlit interface, backend functionality, and MySQL database had to operate together correctly. Changes made to one component sometimes required corresponding adjustments in other parts of the application.

The development of **number-plate detection and OCR/ANPR** introduced additional challenges. Number plates can be small, tilted, partially hidden, or affected by motion blur and lighting conditions. Correctly detecting the plate is necessary before reliable character recognition can be performed. Because of these factors, the number-plate and OCR functionality remains under development.

Similarly, **multi-occupant and triple-riding detection** requires reliable identification and association of multiple persons with the same motorcycle. Crowded traffic scenes and overlapping riders make this particularly challenging. This functionality is therefore being developed and refined separately rather than being treated as a completely finished component.

Finally, converting the AI outputs into a reliable **automated e-challan workflow** requires additional validation and vehicle identification information. Since number-plate recognition and related modules are still under development, complete automated e-challan generation has not been treated as part of the finished implementation.

These challenges provided important practical experience throughout the development of RideSafe AI. They also helped identify the areas that require further refinement before the system can be extended into a more comprehensive real-world traffic monitoring and enforcement platform.

10.2 Solutions Applied

Several solutions were applied during the development of RideSafe AI to address the technical and practical issues encountered during implementation. The solutions focused on improving the reliability of the developed modules, maintaining an organized development environment, and making the system easier to test and extend.

For dataset-related issues, the collected datasets were **organized, converted, merged, and reviewed** before being used for model development. Duplicate and unsuitable samples were removed where required, while the datasets were arranged into appropriate training, validation, and testing groups. This helped maintain a more consistent dataset structure for model training and evaluation.

To address differences between annotation formats, the required data-conversion procedures were applied before preparing the final training data. The segmentation datasets were converted into the format required by the SAM2 training workflow. The final prepared SAM2 dataset contained **1,027 unique images**, divided into training, validation, and testing sets.

For object-detection challenges, the system was tested using traffic scenes with different object sizes, positions, and levels of crowding. Multiple testing scripts and sample images were used to examine the detection outputs. This helped identify cases where motorcycles, riders, or helmets were difficult to detect and provided information for further model refinement.

The helmet-analysis problem was addressed by using dedicated helmet-related training data and combining detection with the developed segmentation-based processing. Testing was performed on both helmeted and non-helmeted riders to verify that the system could distinguish the relevant conditions in suitable traffic scenes.

To handle the computational requirements of model training and inference, the available **NVIDIA RTX 4060 GPU** was utilized for accelerated processing. Training configurations such as batch size and learning rates were selected according to the available hardware resources. This helped maintain practical resource usage while developing the AI models.

Video-processing challenges were addressed by using frame extraction and selected traffic frames for analysis. This allowed the developed models to be tested on representative portions of traffic footage without unnecessarily processing every frame during development and experimentation.

For application integration, the project components were organized into separate files and directories based on their functionality. The Streamlit application was used as the main interface, while the database functionality was maintained separately through MySQL. This organization made it easier to test individual components and identify problems during integration.

Database-related issues were addressed by establishing a structured MySQL database for storing relevant application and violation information. Database operations were tested by inserting and retrieving records to verify that information generated by the application could be maintained correctly.

For the modules that are still under development, the current implementation was kept separate from the completed functionality. **Multi-occupant/triple-riding detection, number-plate detection, and OCR/ANPR** are being developed and tested independently. This prevents incomplete modules from being treated as fully validated features.

The development process also involved repeated testing and debugging of individual scripts and application components. Sample inputs, intermediate outputs, and generated results were reviewed to identify errors and improve the overall workflow.

These solutions helped establish a stable foundation for the current RideSafe AI implementation while keeping the system flexible enough for the remaining modules to be integrated in future development stages.

10.3 Current Limitations/ Known Issues

Although RideSafe AI provides the foundation for AI-assisted motorcycle violation analysis, the current implementation has several limitations that need to be considered. These limitations mainly result from the challenges of real-world traffic environments and from modules that are still under development.

One limitation is that **detection performance depends on the quality of the input media**. Low-resolution images, motion blur, poor lighting, distant motorcycles, and partially visible objects can affect the quality of the generated results. Traffic scenes with significant object overlap can also make it difficult to distinguish individual motorcycles and occupants correctly.

The current system also has limitations in **occupant association**. When several motorcycles and riders are present close to each other, accurately determining which person belongs to which motorcycle can become difficult. This can affect subsequent helmet and occupant analysis, particularly in highly crowded traffic conditions.

Multi-occupant and triple-riding detection is currently under development. Therefore, the current system should not be considered a fully validated automated solution for identifying every multi-riding violation. Additional development and testing are required to improve the reliability of this functionality across different traffic scenarios.

Another current limitation is **number-plate identification**. Number-plate detection and OCR/ANPR are still under development. Small plates, tilted plates, motion blur, poor illumination, obstruction, and different plate appearances can make accurate vehicle identification difficult. Consequently, complete vehicle identification is not yet treated as a finished component of the system.

The **automated e-challan generation process is also not fully implemented**. A complete e-challan workflow would require reliable vehicle identification, validated violation information, appropriate database records, and integration with an authorized external enforcement or challan service. The current project provides the foundation for such integration but does not claim complete automated challan generation.

Another limitation is the **processing requirement for video-based analysis**. Processing continuous traffic footage can require considerable computational resources and time, especially when multiple AI operations are performed on high-resolution frames. The current implementation is therefore primarily intended for development, demonstration, and controlled testing rather than unrestricted large-scale traffic surveillance.

The system also depends on the **quality and diversity of its training data**. Although the prepared datasets provide a useful foundation, additional traffic samples covering different roads, weather conditions, camera positions, vehicle types, and lighting conditions would help improve the generalization of the models.

Database and administrative functionality are also limited to the features currently implemented. The system maintains and displays relevant records, but a complete production-level enforcement platform would require additional security controls, user-management mechanisms, audit procedures, and integration with authorized traffic-management systems.

These limitations do not prevent the current system from demonstrating its intended AI-assisted traffic-analysis capabilities. Instead, they define the areas that require further development, testing, and validation before RideSafe AI can be considered for deployment in a fully automated real-world traffic enforcement environment.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 AI and System Enhancements

RideSafe AI provides a foundation for AI-assisted motorcycle violation analysis, but several improvements can be introduced to increase its accuracy, reliability, and practical usability. Future development can focus on improving the existing AI models and extending their ability to handle more complex traffic environments.

One important enhancement is the improvement of **motorcycle and occupant detection** using larger and more diverse traffic datasets. Additional images and videos collected from different road conditions, camera positions, lighting environments, and traffic densities can help the models perform more consistently in real-world situations.

The **helmet detection capability** can also be improved by including more variations of helmets, rider appearances, camera angles, and difficult visibility conditions in the training data. This can help reduce incorrect detections in situations where the helmet is partially visible or the rider is located far from the camera.

A major future enhancement is the completion of **multi-occupant and triple-riding detection**. The system can be further developed to reliably identify the number of persons associated with each motorcycle and classify cases where the permitted number of occupants is exceeded. This would allow multi-riding analysis to become a more complete part of the violation-detection workflow.

The **number-plate detection and OCR/ANPR modules** can also be improved and integrated into the main system. A more robust vehicle-identification process would allow detected violations to be associated with the corresponding registration number. This would significantly extend the practical functionality of the application.

Future versions can also incorporate **multi-frame and video-based analysis** rather than relying primarily on individual frames. By considering information across consecutive frames, the system could improve object tracking and maintain more consistent information about motorcycles and their occupants throughout a traffic sequence.

Another enhancement could involve **object tracking** for motorcycles and occupants. Tracking can help maintain the identity of detected objects across multiple frames and reduce repeated processing of the same vehicle. This can also support more reliable violation-event generation from continuous traffic footage.

The system can further be enhanced through **model optimization and hardware acceleration** to reduce processing time. Techniques such as optimized inference configurations, efficient model versions, and improved frame-processing strategies could make the application more suitable for larger traffic datasets and longer video streams.

Additional validation can also be performed using traffic footage from different locations and environmental conditions. This would help evaluate how well the developed system generalizes beyond the datasets used during the current development stage.

These improvements would allow RideSafe AI to progress from a development and demonstration platform toward a more comprehensive traffic-analysis system capable of handling a wider range of real-world conditions.

11.2 Possible Integrations or Optimizations

Future development of RideSafe AI can extend the system beyond helmet and motorcycle-occupancy violations by introducing additional traffic-safety and vehicle-monitoring capabilities. These enhancements can make the platform more comprehensive while allowing the existing AI and database infrastructure to support additional types of traffic events.

One major future feature is **mobile-phone usage detection while driving**. A dedicated computer-vision model can be developed to identify situations where a motorcycle rider is using a mobile phone while operating the vehicle. The system could analyze the rider's hand position and the presence of a mobile device and generate a potential violation record when sufficient visual evidence is available.

Another possible enhancement is **wrong-route or wrong-way driving detection**. By combining vehicle detection with road-lane information and movement tracking, the system could identify motorcycles or other vehicles travelling in a restricted or incorrect direction. This functionality could be particularly useful for monitoring one-way roads, restricted lanes, and other locations were travelling in the wrong direction creates a safety risk.

The system can also be extended to support **additional traffic violations** such as signal violations, illegal lane changes, unsafe overtaking, and other rule-based traffic events. These capabilities can be developed as separate AI modules and integrated into the existing violation-management framework after appropriate training and validation.

The **OCR/ANPR module** provides an important opportunity for future vehicle identification. Once a license plate is detected and its registration number is extracted, the information can be associated with the corresponding traffic event and stored with the relevant timestamp and available evidence. This can provide an additional vehicle-identification layer for authorized monitoring and investigation.

Beyond traffic violations, license-plate information could potentially support **law-and-order investigations** when legally authorized and subject to applicable privacy, data-protection, and evidentiary requirements. For example, if a vehicle involved in a reported **hit-and-run, vehicle theft, or another criminal investigation** is captured by a connected traffic camera, the recorded license plate could provide a lead for authorized investigators. Historical traffic-camera records could potentially be searched for matching vehicle registration numbers and corresponding timestamps or locations.

Such investigative use should be treated separately from automatic violation generation. A detected license plate should not by itself be considered proof that a person committed a crime. Instead, the identified vehicle information can serve as **supporting investigative evidence or a lead**, which can then be verified through appropriate law-enforcement procedures and additional evidence.

Another future enhancement is **real-time CCTV integration**. The system could receive live camera streams and continuously analyze traffic conditions, allowing violations and relevant vehicle information to be identified as events occur. With suitable infrastructure, multiple camera locations could be connected to a centralized monitoring platform.

A more advanced administrative dashboard could also provide **vehicle search, event history, location-based filtering, timestamps, violation categories, and visual evidence**. Authorized users could search recorded events using available vehicle or incident information and review the associated footage or detection results.

The platform could further incorporate **geolocation and temporal analysis** so that each detected event is associated with its approximate camera location and time. This could support both traffic-management analysis and authorized investigations involving specific vehicles or incidents.

With these enhancements, RideSafe AI can evolve from a motorcycle-violation detection system into a broader **AI-based intelligent traffic and vehicle monitoring platform**, supporting traffic-safety analysis, vehicle identification, authorized investigative assistance, and future integration with traffic-management and law-enforcement systems.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

RideSafe AI was developed as an AI-driven framework for analyzing motorcycle traffic scenes and identifying potential traffic violations through computer vision. The primary objective of the project was to develop a practical system that can assist in identifying motorcycle-related safety violations from traffic images and video while providing a structured platform for reviewing and maintaining the generated information.

The project addresses the limitations of relying entirely on manual observation of traffic footage. Continuous traffic monitoring can involve large amounts of visual information, making manual identification of every possible violation difficult and time-consuming. RideSafe AI explores how artificial intelligence can assist this process by automatically analyzing traffic media and identifying relevant motorcycles, riders, and helmet-related conditions.

During development, the project followed a complete workflow beginning with dataset preparation and AI model development, followed by testing, application integration, and database management. The prepared datasets were used to develop the required AI components, while different traffic scenarios were used to examine the behavior of the system under practical conditions. The results were then made accessible through an interactive application interface.

The developed system is capable of supporting motorcycle detection, occupant analysis, and helmet-violation analysis. It also provides a foundation for analyzing motorcycles carrying multiple occupants. The information generated by the application can be maintained through the MySQL database and reviewed through the administrative monitoring functionality.

An important aspect of the project is that the current implementation clearly separates completed functionality from features that require further development. Triple-riding detection, number-plate detection, OCR/ANPR, and automated e-challan processing are not presented as fully completed capabilities. Instead, the current system provides the foundation required for these components to be integrated and validated in subsequent development.

The project demonstrates that AI-based traffic analysis can be used as a practical assistance mechanism for identifying potential motorcycle violations and organizing the resulting information. Rather than replacing human verification or authorized enforcement procedures, the system can provide automated analysis that helps reduce the amount of manual effort required to review traffic footage.

Overall, RideSafe AI represents a practical application of artificial intelligence and computer vision to a real-world traffic-safety problem. The project establishes a working foundation that can be expanded with additional detection capabilities, vehicle identification, real-time monitoring, and authorized external system integration.

12.2 Achievements

The development of RideSafe AI resulted in several significant achievements in the areas of artificial intelligence, computer vision, dataset preparation, application development, and system integration.

One of the major achievements was the development of an **AI-assisted motorcycle violation-analysis workflow**. The system was designed to process traffic media and identify motorcycles and relevant rider conditions, providing a foundation for automated analysis of helmet-related violations.

A structured dataset-development process was also completed for the segmentation component. The final SAM2 dataset contained **1,027 unique images**, divided into training, validation, and testing sets, with **1,570 annotated object instances**. Preparing this dataset required combining available sources, handling annotation formats, removing duplicate data, and organizing the final data for model development.

The project also provided practical experience in developing and working with **YOLO11-based object detection and SAM2-based image segmentation**. The AI components were tested using different traffic scenarios to understand their behavior under variations such as object size, visibility, camera angle, and traffic density.

Another important achievement was the development of the **Streamlit-based RideSafe AI application**. Instead of keeping the AI models as independent scripts, the project brought the developed functionality into an interactive application where traffic media can be processed and the available results can be viewed.

The integration of **MySQL database functionality** was another significant achievement. Relevant information generated by the application can be maintained in a structured database, providing a foundation for record management and administrative monitoring.

The project also established a foundation for future vehicle-identification capabilities. The ongoing development of number-plate detection and OCR/ANPR can eventually allow detected traffic events to be associated with specific vehicle registration information. This can provide additional possibilities for traffic monitoring and, where legally authorized, support investigative workflows involving incidents such as hit-and-run or vehicle theft.

From a development perspective, the project also achieved practical integration of multiple components into a single system. The team gained experience in moving from individual AI experiments toward a structured application involving models, datasets, user interfaces, backend functionality, and database storage.

The project therefore achieved its primary development objective of creating a functional foundation for AI-assisted motorcycle traffic-violation analysis while also identifying the technical areas that require further development before a complete real-world enforcement system can be established.

12.3 Skills Learned During Development

The development of RideSafe AI provided the team with practical experience in a wide range of technical and problem-solving areas. The project required the application of concepts learned through academic coursework as well as independent exploration of modern AI and software-development technologies.

A major area of learning was **computer vision and artificial intelligence**. The team gained practical experience in object detection, image segmentation, model training, inference, dataset preparation, and evaluation. Working with traffic images and videos also provided an understanding of the difficulties involved in applying AI models to real-world visual data.

The team developed experience in **dataset engineering**, including collecting data from different sources, organizing datasets, converting annotation formats, removing duplicate samples, creating training and validation sets, and preparing data for model training. This helped demonstrate the importance of high-quality and properly structured data in machine-learning projects.

The project also strengthened **Python programming and AI development skills**. The team worked with machine-learning frameworks, computer-vision libraries, model-processing scripts, and application components. Debugging model outputs and resolving compatibility or configuration issues provided practical experience beyond theoretical programming exercises.

Experience was also gained in **application development using Streamlit**. The team learned how to connect AI processing functionality with an interactive user interface and how to present generated results in a form that can be demonstrated and reviewed by users.

Working with **MySQL and database integration** provided knowledge of structured data storage, record management, database connectivity, and retrieval operations. This helped the team understand how AI-generated information can be incorporated into a larger software system rather than remaining only as temporary model outputs.

The project also improved the team's understanding of **software testing and debugging**. Different input conditions were examined, unexpected results were investigated, and individual modules were tested before being integrated into the complete application. This process highlighted the importance of systematic testing when developing AI-based systems.

In addition to technical knowledge, the project strengthened **team collaboration, project planning, documentation, problem-solving, and presentation skills**. The team had to divide development activities, track progress, troubleshoot issues, document the implementation, and present the final work in a clear technical format.

Most importantly, the project provided practical insight into the complete lifecycle of an AI application—from data preparation and model development to application integration, testing, deployment, and future expansion. This experience provides a strong foundation for developing more advanced computer-vision and intelligent software systems in future projects.

CHAPTER -13 REFERENCES

Reference Papers

No.	Year	Research Paper	Main Focus	Key Method / Result
1	2025	Automated traffic violation detection and E-challan system using YOLOv3 and IoT	This new traffic violation detection system using YOLOv3 and IoT effectively detects signal jumps and helmet violations in various environmental conditions, enabling intelligent traffic regulation in metropolitan areas.	YOLOv3, Tesseract, OCR, CRNN. This new traffic violation detection system using YOLOv3 and IoT effectively detects signal jumps and helmet violations in various environmental conditions, enabling intelligent tr
2	2026	Automated Road Safety Surveillance: Real-Time Helmet Detection and E-Challan Generation Using YOLO and OCR	This automated surveillance system detects helmetless riders and extracts vehicle registration numbers in real-time, helping authorities enforce helmet laws and promote safer riding behavior.	OCR. This automated surveillance system detects helmetless riders and extracts vehicle registration numbers in real-time, helping authorities enforce helmet laws and promote safer ridin
3	2025	A Deep Learning Approach for Helmet Detection and Fine Generation System	This study proposes a real-time deep learning-based helmet detection and fine generation system with over 95% accuracy, reducing manual workload and supporting road safety enforcement.	YOLOv3, OCR. accuracy of over 95% for helmet detection.
4	2023	Generating Smart E-challan System for Violation of Traffic Rules	This system automatically detects motorcyclists without helmets and retrieves their motorcycle number plates from CCTV footage, generating e-challans and databases for traffic rule violations.	OCR. This system automatically detects motorcyclists without helmets and retrieves their motorcycle number plates from CCTV footage, generating e-challans and databases for traffic rule
5	2025	Development of an AI-Based Smart Traffic Guard System for Real-Time Detection and Automated Penalty Enforcement of Traffic Violations		YOLOv5, OCR, Tkinter.
6	2025	Two-Wheeler Traffic Violations Detection and	This paper proposes an automated system that	This paper proposes an automated system that

		Automated Penalty Issuance System	detects traffic violations, tracks offenders, and issues tickets using vehicle registration numbers, improving enforcement efficiency and road safety.	detects traffic violations, tracks offenders, and issues tickets using vehicle registration numbers, improving enforcement efficiency a
7	2022	Two-Wheeler Vehicle Traffic Violations Detection and Automated Ticketing for Indian Road Scenario	This AI-based traffic violation detection system effectively detects two-wheeler violations and automates ticketing in Indian road scenarios, improving safety and promoting smart city ecosystems.	DeepSORT, Tesseract. precision (mAP) of 98.
8	2025	Computer-vision based automatic rider helmet violation detection and vehicle identification in Indian smart city scenarios using NVIDIA TAO toolkit and YOLOv8	Our two-step deep learning approach accurately detects rider helmet violations and vehicles in Indian smart cities, reducing traffic safety risks.	YOLOv8, OCR. achieved a promising helmet detection accuracy of 98.
9	2026	Modular AI-Based Framework for Two-Wheeler Traffic Violation Detection Using Object Detection, Pose Estimation, and ALPR	This modular AI-based system effectively detects two-wheeler traffic violations in real-time surveillance, generating e-challans, with a best mAP of 95%.	YOLOv8, YOLOv5, ALPR, pose estimation. achieved a best mAP of 95%, while the helmet detection model achieved 98.
10	2026	Real-Time YOLOv8-based Helmet Violation Detection and License Plate Recognition for Intelligent Traffic Monitoring	The proposed real-time traffic monitoring system accurately detects helmet violations and recognizes vehicle plates at a rate of 32 frames per second, providing safety in high-speed automated traffic monitoring.	YOLOv8. accuracy of 96.
11	2025	A Web-Based Ticketing System for Traffic Violations Using YOLO and OCR for Real-Time Detection	This web-based electronic ticketing system effectively detects traffic violations, particularly helmet noncompliance, with 93% helmet detection accuracy and 94% license plate detection accuracy.	OCR. accuracy, $94.
12	2025	RoadEye: A Framework For Detecting Traffic Rule Violations	RoadEye, a framework for detecting traffic rule violations, shows promising accuracy in recognizing license plates, cross-walk violations, and helmetless driving.	mAP (Mean Accuracy Precision) score of 74% in detecting and localizing the license plates, 89% for cross-walk detection and 67% for helmetless driving detection.
13	2024	NUMBERPLATE DETECTION AND CHALLAN GENERATION FOR NON-HELMET RIDERS	This project develops a computerized CNN-based detection device for helmet authentication and a web-based interface for managing and tracking traffic infractions, improving safety and customer satisfaction.	This project develops a computerized CNN-based detection device for helmet authentication and a web-based interface for managing and tracking traffic infractions, improving safety
14	2026	SafeRide- AI Enabled Smart Road Safety Violation Detection and Monitoring System Using Neural Vision	This paper presents a real-time road safety violation detection system using deep learning and computer vision techniques, achieving high precision and recall,	YOLOv8. precision of 78%, recall of 79.

			potentially integrating with smart city infrastructure.	
15	2025	Automated Helmet Detection and Fine Issuance System Using YOLOv8 C OpenCV	The Helmet Detection for Motorcyclists using YOLOv8 and OpenCV system automatically identifies and penalizes violators in real-time, promoting helmet use and improving road safety.	YOLOv8, EasyOCR, OCR. The Helmet Detection for Motorcyclists using YOLOv8 and OpenCV system automatically identifies and penalizes violators in real-time, promoting helmet use and improving road safety.
16	2026	Smart Surveillance System for Triple Riding and No Helmet Violation Detection and Send Notification Using License Plate Recognition	This deep learning-based system effectively detects triple riding and helmetless riders using surveillance cameras, and automatically notifies registered vehicle owners of violations using license plate recognition.	YOLOv8, EasyOCR, OCR, CRNN. achieving robust detection under varying illumination and dense traffic conditions.
17	2025	Smart Traffic Surveillance: Helmet Detection and License Plate Recognition Using Deep Learning	This paper presents an advanced traffic violation detection system that combines helmet detection and license plate recognition using deep learning, enhancing road safety and reducing law enforcement burden.	YOLOv3, EasyOCR, OCR. accuracy by fine-hyperparameter tuning YOLOv3 on a diverse dataset that includes various helmet types, different lighting conditions, and occlusions.
18	2025	AI-Powered Smart Traffic Management System	AI-Powered Smart Traffic Management System effectively detects traffic violations and issues automated challans through Telegram, improving road safety and reducing human dependency in traffic monitoring.	accuracy and real-time performance, highlighting the potential for deployment in smart cities to improve road safety and reduce human dependency in traffic monitoring.
19	2023	Sighted-Helmet Detection and E-Challan Application	Helmet detection technology and e-challan systems can effectively enforce helmet laws and reduce head injuries in motorcycle accidents.	YOLOv8. Helmet detection technology and e-challan systems can effectively enforce helmet laws and reduce head injuries in motorcycle accidents.
20	2026	HELPR: Helmet Enforcement and License Plate Reconnaissance Using YOLOv8 and EasyOCR for Real-Time Traffic Surveillance	The HELPR system accurately detects helmet violations and motorcycle number plates in real-time traffic surveillance, offering scalable, data-driven traffic enforcement in smart cities.	YOLOv8, EasyOCR, OCR. accuracy in helmet detection, 96% accuracy in rider detection, and 88% accuracy in number plate detection, whereas EasyOCR gets a maximum of 96% accuracy in character-level OCR.

The datasets used during the development of RideSafe AI were collected from the following sources:

1. **Kaggle** – Motorcycle and helmet-related datasets were obtained from Kaggle and used for model development and experimentation.

2. **Roboflow Universe** – Multiple motorcycle, helmet, rider, and segmentation datasets were obtained from Roboflow Universe. Roboflow Universe provides publicly available computer-vision datasets in formats suitable for object detection and related tasks.

3. **COCO Dataset** – COCO-format datasets and annotations were used during the segmentation-data preparation process. COCO is a large-scale dataset supporting object detection and segmentation.

4. **Final Combined Dataset** – The datasets obtained from these sources were processed, converted where necessary, cleaned, merged, and deduplicated to prepare the final datasets used in the RideSafe AI project.

For the **SAM2 segmentation component**, the final prepared dataset contained **1,027 unique images**, divided into training, validation, and testing sets.

The following official documentation and technical resources were referred to during the development, implementation, testing, and deployment of RideSafe AI:

1. **Python Documentation** – Used as a reference for Python programming, standard libraries, environment configuration, and application development.

2. **Ultralytics YOLO11 Documentation** – Used for YOLO11 model configuration, training, validation, inference, and object-detection implementation.

3. **Meta AI SAM 2 Documentation** – Used as a technical reference for SAM2-based image and video segmentation and model implementation.

4. **PyTorch Documentation** – Used for deep-learning model development, training, inference, tensor operations, and GPU-accelerated processing.

5. **OpenCV Documentation** – Used for image processing, video processing, computer-vision operations, and handling traffic-media inputs.

6. **Streamlit Documentation** – Used for developing and running the interactive RideSafe AI application and its user interface.

7. **FastAPI Documentation** – Used as a technical reference for Python-based backend and API development where applicable in the project environment.

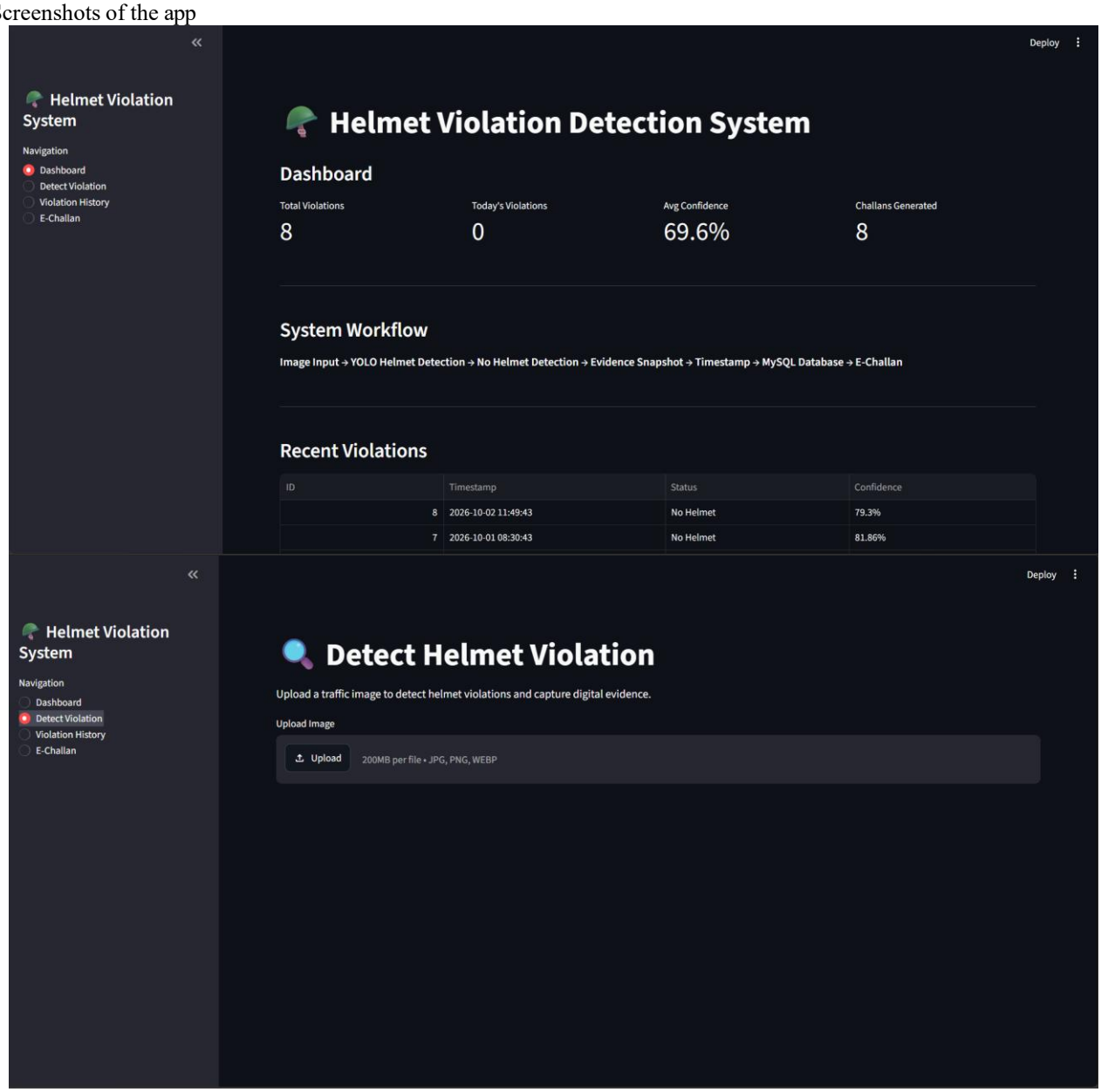
8. **MySQL Reference Documentation** – Used for database creation, table management, SQL operations, database configuration, and storage of application records.

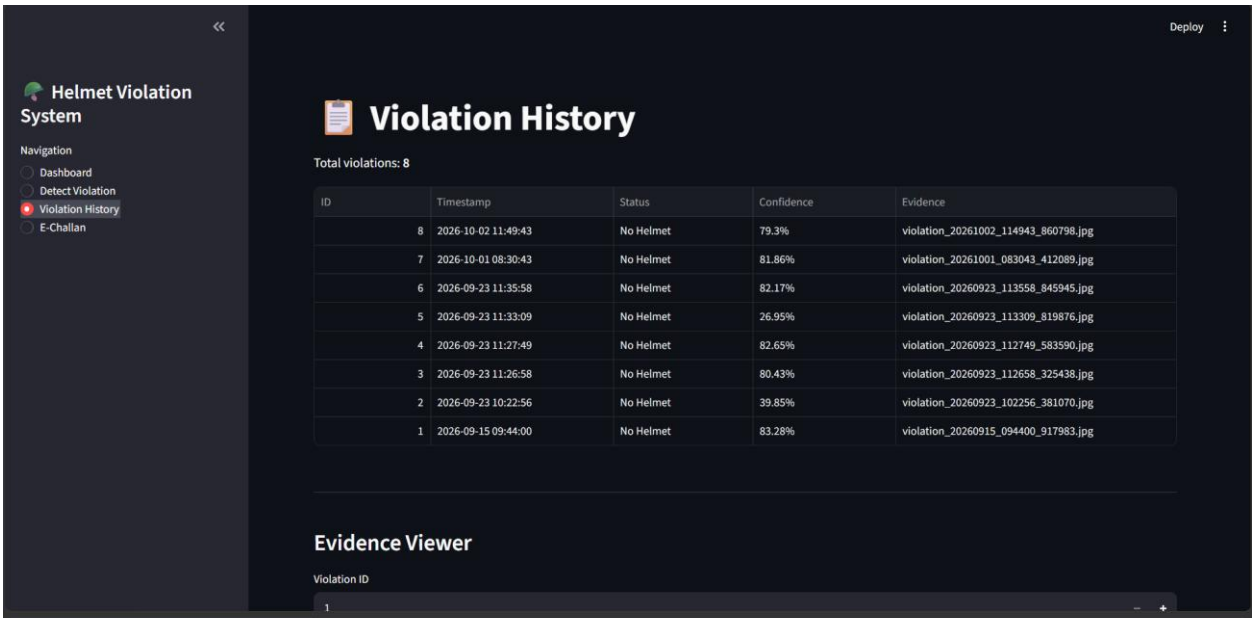
9. **Git and GitHub Documentation** – Used for source-code version control, repository management, commits, branches, and project collaboration.

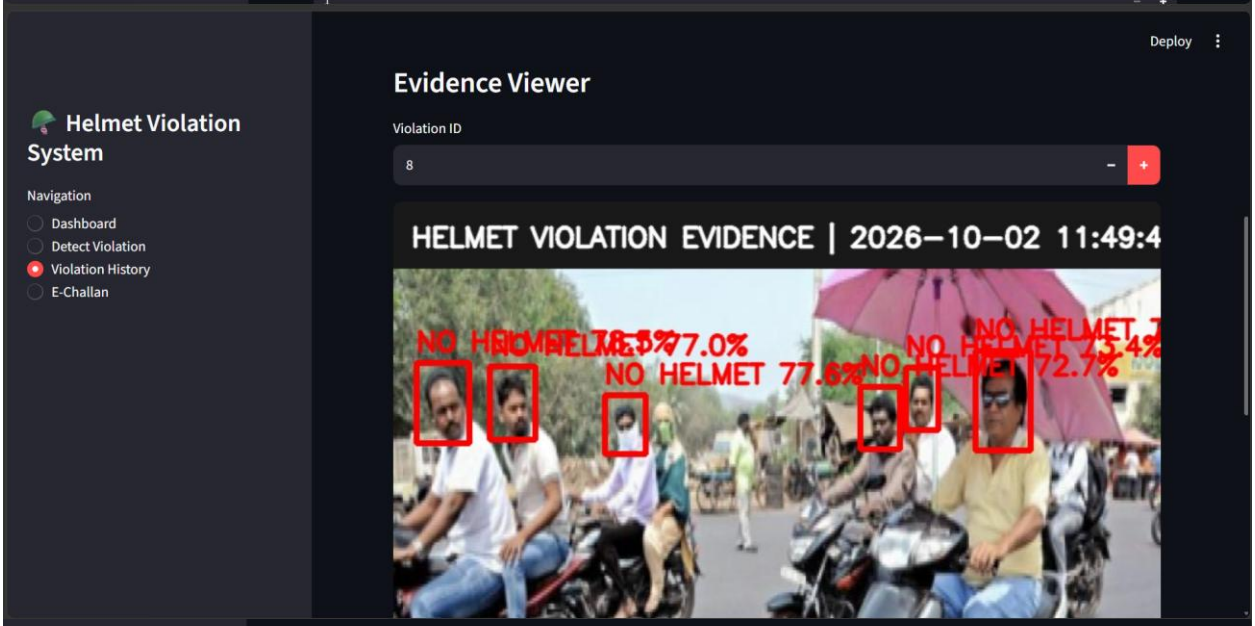
10. **NVIDIA CUDA / GPU Computing Resources** – Referred to for GPU-accelerated deep-learning execution and computational environment configuration.

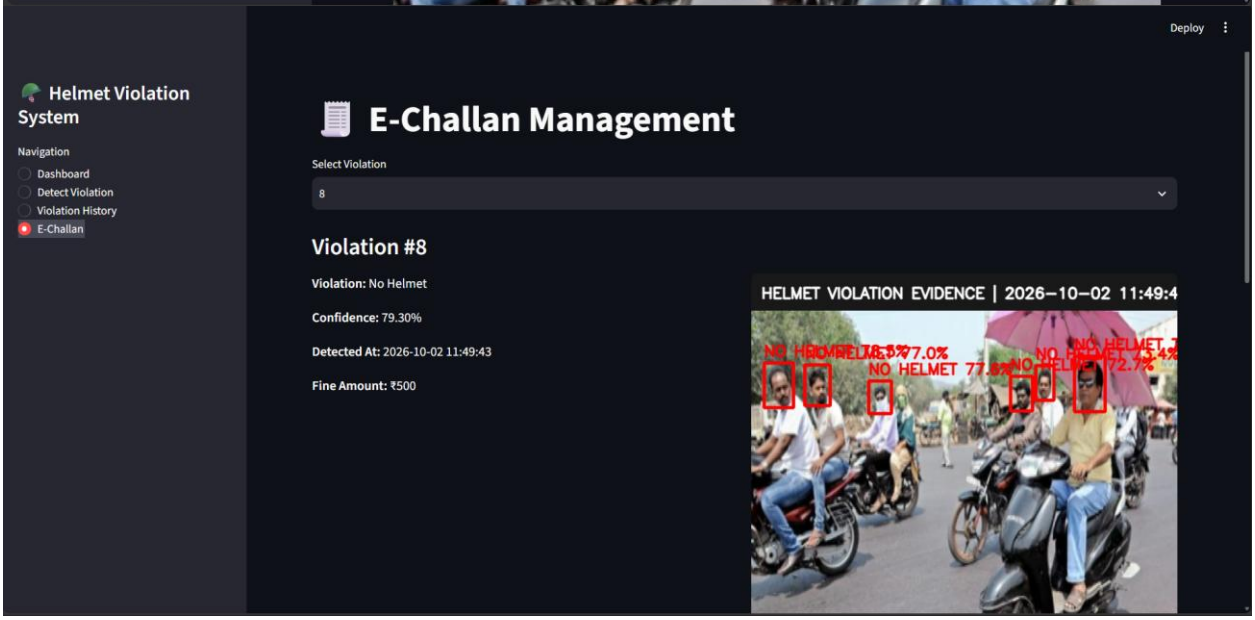
CHAPTER -14 APPENDICES

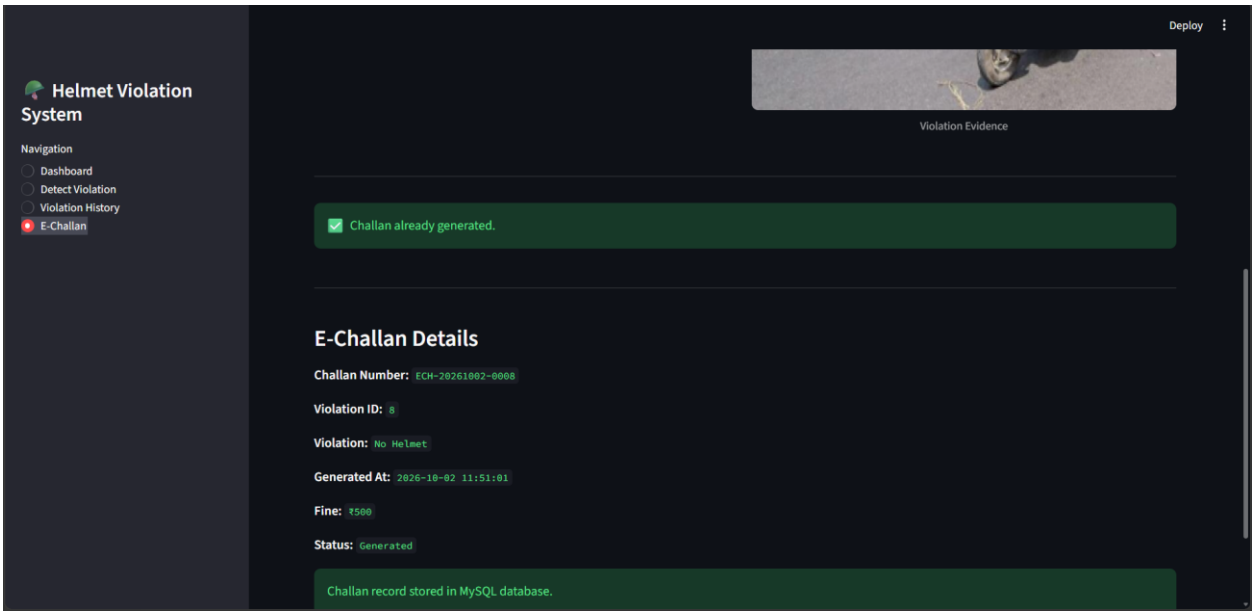
Screenshots of the app




Deploy
Helmet Violation System
Navigation
Dashboard
Detect Violation
Violation History
E-Challan
Violation History
Total violations: 8
ID | Timestamp | Status | Confidence | Evidence
8 | 2026-10-02 11:49:43 | No Helmet | 79.3% | violation_20261002_114943_860798.jpg
7 | 2026-10-01 08:30:43 | No Helmet | 81.86% | violation_20261001_083043_412089.jpg
6 | 2026-09-23 11:35:58 | No Helmet | 82.17% | violation_20260923_113558_845945.jpg
5 | 2026-09-23 11:33:09 | No Helmet | 26.95% | violation_20260923_113309_819876.jpg
4 | 2026-09-23 11:27:49 | No Helmet | 82.65% | violation_20260923_112749_583590.jpg
3 | 2026-09-23 11:26:58 | No Helmet | 80.43% | violation_20260923_112658_325438.jpg
2 | 2026-09-23 10:22:56 | No Helmet | 39.85% | violation_20260923_102256_381070.jpg
1 | 2026-09-15 09:44:00 | No Helmet | 83.28% | violation_20260915_094400_917983.jpg
Evidence Viewer
Violation ID
1

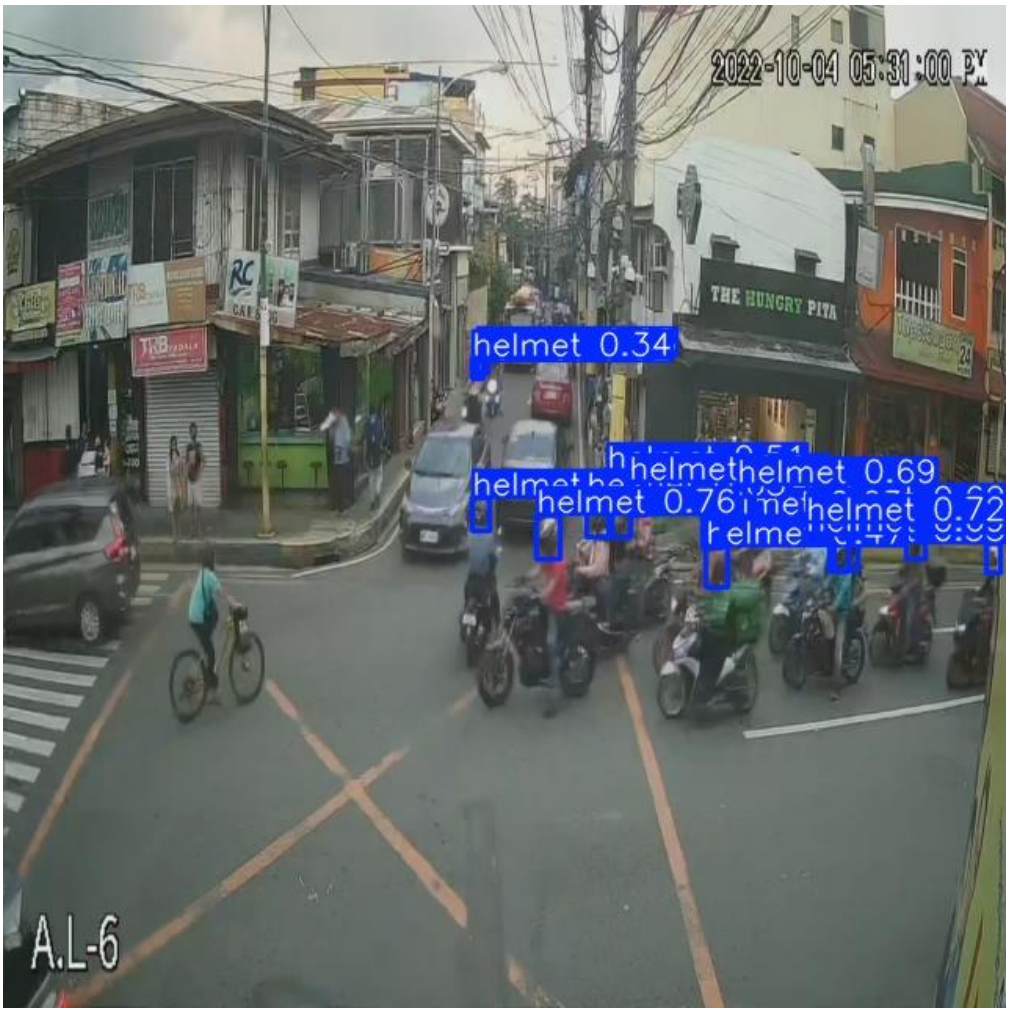

Deploy
Helmet Violation System
Navigation
Dashboard
Detect Violation
Violation History
E-Challan
Evidence Viewer
Violation ID
8
HELMET VIOLATION EVIDENCE | 2026-10-02 11:49:4
NO HELMET 77.6%


Deploy
Helmet Violation System
Navigation
Dashboard
Detect Violation
Violation History
E-Challan
E-Challan Management
Select Violation
8
Violation #8
Violation: No Helmet
Confidence: 79.30%
Detected At: 2026-10-02 11:49:43
Fine Amount: ₹500
HELMET VIOLATION EVIDENCE | 2026-10-02 11:49:4
NO HELMET 77.6%



AI Detection Outputs




2022-10-04 05:31:00 PM
THE HUNGRY PITA
helmet 0.34
helmethelmet 0.69
helmet 0.76
helmet 0.72
A.L-6


motorcycle-rider
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
motorcycle-rider
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
motorcycle-rider
motorcycle-rider
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
motorcycle-rider
motorcycle-rider
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
motorcycle-rider
motorcycle-rider
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Faced
Half-Faced
Full-Invalid

SAM2 Segmentation Outputs



Database Screenshots

id	violation_id	challan_number	generated_at	fine_amount	status
1	1	ECH-20260915-0001	2026-09-15 09:44:31	500	Generated
2	2	ECH-20260923-0002	2026-09-23 10:23:45	500	Generated
3	6	ECH-20260923-0006	2026-09-23 11:36:27	500	Generated
4	5	ECH-20260923-0005	2026-09-23 21:02:05	500	Generated
5	4	ECH-20260923-0004	2026-09-23 21:02:15	500	Generated
6	3	ECH-20260923-0003	2026-09-23 21:02:23	500	Generated
7	7	ECH-20261001-0007	2026-10-01 08:31:24	500	Generated
8	8	ECH-20261002-0008	2026-10-02 11:51:01	500	Generated
NULL	NULL	NULL	NULL	NULL	NULL

violation_id	timestamp	helmet_status	confidence	evidence_image	challan_number	generated_at	fine_amount	status
1	2026-09-15 09:44:00	No Helmet	0.83284	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260915-0001	2026-09-15 09:44:31	500	Generated
2	2026-09-23 10:22:56	No Helmet	0.398501	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0002	2026-09-23 10:23:45	500	Generated
3	2026-09-23 11:26:58	No Helmet	0.804312	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0003	2026-09-23 21:02:23	500	Generated
4	2026-09-23 11:27:49	No Helmet	0.826479	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0004	2026-09-23 21:02:15	500	Generated
5	2026-09-23 11:33:09	No Helmet	0.269471	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0005	2026-09-23 21:02:05	500	Generated
6	2026-09-23 11:35:58	No Helmet	0.821686	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20260923-0006	2026-09-23 11:36:27	500	Generated
7	2026-10-01 08:30:43	No Helmet	0.818649	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20261001-0007	2026-10-01 08:31:24	500	Generated
8	2026-10-02 11:49:43	No Helmet	0.792978	C:\AI Projects\HelmetViolationSystem\outputs\evidence\violation_202...	ECH-20261002-0008	2026-10-02 11:51:01	500	Generated